Parsek — Rewind to Separation

Post-implementation design specification for the mid-mission rewind system. Covers Rewind Points captured at multi-controllable split events (staging, undocking, EVA), the Unfinished Flights virtual group, the append-only supersede relation, narrow kerbal-death tombstone scope, the journaled staged commit, and the load-time sweep that keeps half-finished state bounded — together with the v0.9.1 stable-leaf extension that broadens the Unfinished Flights membership predicate to include orbiting / sub-orbital non-focus siblings and stranded EVA kerbals, adds per-slot Seal and Stash actions, and tightens Re-Fly merge classification with a focus-override auto-seal contract.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to an immutable timeline, and see previously recorded missions play back as ghost vessels alongside new ones. This document extends the flight recorder, timeline, and ledger systems with Rewind-to-Separation. It assumes familiarity with the recording DAG, BranchPoint model, controller identity, ghost chains, and the additive-only invariant (see `parsek-flight-recorder-design.md`) and with the ledger model, immutable ActionId, reservations, and career-state replay (see `parsek-game-actions-and-resources-recorder-design.md`).

Status: shipped. Core feature in v0.9.0; stable-leaf extension (broadened Unfinished Flights predicate, per-slot Seal / Stash actions, Re-Fly merge auto-seal contract, invocation linearization) in v0.9.1. **Pre-implementation specs (archived):**

- `docs/dev/done/parsek-rewind-separation-design.md` — pre-v0.9.0 spec, archived as-is alongside the v0.9 rollout.
- `docs/dev/done/parsek-unfinished-flights-stable-leaves-design.md` — pre-v0.9.1 spec for the stable-leaf extension, archived alongside the v0.9.1 rollout. Promoted from research note `docs/dev/research/extending-rewind-to-stable-leaves.md` (R17, merged in PR #634).

This document supersedes both pre-impl specs as the source of truth for what shipped. **Related docs:** `parsek-flight-recorder-design.md`, `parsek-recording-finalization-design.md`, `parsek-timeline-design.md`, `parsek-game-actions-and-resources-recorder-design.md`, `parsek-logistics-supply-routes-design.md`.

---

1. Introduction

A Parsek career is a committed timeline of missions. The player flies, commits, the ghost plays back, and the next mission begins on top. Before v0.9 the timeline had a sharp edge: once a multi-controllable split happened — a stage decoupled with a probe core on each side, a lander undocked from a station, a kerbal popped out on EVA — whichever half the player did not personally fly was recorded in the background and, if things went badly, committed as a crashed or destroyed sibling. There was no in-game path back to re-fly that half. The successful side was locked in with the failed side, and the only "fix" was to discard the entire tree and re-fly the whole mission.

Rewind to Separation is the feature that unsticks that edge. At every split that produces two or more controllable entities, Parsek writes a transient KSP quicksave — a **Rewind Point** — plus a compact persistent-id-to-slot table captured at save time. If any sibling ends as an **unfinished flight** — a crashed booster, a destroyed lander, a kerbal who fell without a parachute, a probe abandoned in orbit, an upper stage left coasting sub-orbital, a kerbal stranded alive on a surface — the recording appears in a read-only **Unfinished Flights** group in the Recordings Manager. Clicking Rewind on an Unfinished-Flight row, either in that virtual group or on the same recording's normal table row, reloads the quicksave, strips the non-selected siblings (they play back as ghosts from their committed recordings), and hands the player the other half at the exact split moment. When the re-fly ends and the player merges, the new recording supersedes the old one via an **append-only relation** — the original recording is never mutated or deleted, the ghost/claim subsystems just filter it out.

Two membership tiers, one feature. The v0.9.0 release shipped the narrow "rescue from destruction" predicate: only `TerminalKind.Crashed` siblings of multi-controllable splits qualified. The v0.9.1 stable-leaf extension broadens the predicate to also include orbiting / sub-orbital non-focus siblings (the 4-probe deploy case, the upper-stage-left-coasting case) and stranded-alive EVA kerbals, while preserving the focus-continuation exclusion so a routine upper stage that the player flew to orbit does not pollute the list. A per-row **Seal** button lets the player close any slot permanently when they have decided the recording is canonical, and a per-row **Stash** button lets the player promote a default-excluded stable leaf into Unfinished Flights when they want it back later. Re-Fly merges of player-chosen slots that reach a stable terminal auto-seal the slot — the player explicitly engaged the slot to fly it, and the merge concludes that engagement. Terminal-failure outcomes (Destroyed, non-boarded EVA) without a retry-blocking action stay open so the player can retry the failure. Crashed sibling rows from v0.9.0 keep their behaviour exactly.

1.1 Scope

The shipped feature covers:

v0.9.0 core (rescue from destruction):

- **Split detection** at joint-break, undock, and EVA boundaries. `SegmentBoundaryLogic.IsMultiControllableSplit` (`Source/Parsek/SegmentBoundaryLogic.cs`) is the gate; debris-only splits and single-controllable splits do not receive a Rewind Point.
- **Rewind Point capture.** `RewindPointAuthor.Begin` (`Source/Parsek/RewindPointAuthor.cs:52`) synchronously attaches the RP to its `BranchPoint` and to `ParsekScenario.RewindPoints` , then a one-frame-deferred coroutine drops warp to zero, populates both PID-to-slot maps from live vessels, writes a stock KSP save, and atomically moves the result into `saves/<save>/Parsek/RewindPoints/<rpId>.sfs` . The author also captures `RewindPoint.FocusSlotIndex` (the slot index of the active vessel at split time, or `-1` when no live slot matches the active focus) for the v0.9.1 predicate to consume.
- **Unfinished Flights UI group.** A virtual, read-only group in the Recordings Manager (`Source/Parsek/UI/UnfinishedFlightsGroup.cs`) computed per-frame from ERS filtered by `EffectiveState.IsUnfinishedFlight` . Membership updates automatically as flights are flown, sealed, stashed, and merged.
- **Invocation.** `RewindInvoker` (`Source/Parsek/RewindInvoker.cs`) runs a five-precondition gate, captures a pre-load reconciliation bundle, copies the RP quicksave to the save-root (KSP's `LoadGame` does not accept subdirectory paths), triggers `GamePersistence.LoadGame` + `HighLogic.LoadScene(FLIGHT)` , then on the reloaded scene atomically runs Restore → Strip → Activate → provisional + `ReFlySessionMarker` write.
- **Append-only supersede.** On merge, `SupersedeCommit.AppendRelations` (`Source/Parsek/SupersedeCommit.cs:108`) appends one `RecordingSupersedeRelation` per recording in the forward-only merge-guarded subtree closure of the retired sibling. No field on a committed Recording is mutated post-commit.
- **Narrow tombstone scope.** The only ledger retirement on supersede is `KerbalAssignment` -to-Dead actions and `ReputationPenalty` actions bundled with them (paired by same-recording kerbal-death within a 1-second UT window). Contract completions, milestones, facility upgrades, strategies, tech research, science rewards, funds spending, and vessel-destruction rep penalties all remain in the Effective Ledger Set.
- **Crashed re-fly stays rewindable.** `TerminalKindClassifier.Classify` (`Source/Parsek/TerminalKindClassifier.cs`) maps the provisional's terminal state to `Landed` (commits `Immutable`), `Crashed` (commits `CommittedProvisional` so the slot stays an Unfinished Flight), or `InFlight` (same as Landed). The v0.9.1 Re-Fly auto-seal contract (§4.9) tightens this for player-chosen slots reaching stable terminals.
- **Journaled staged commit.** `MergeJournalOrchestrator.RunMerge` (`Source/Parsek/MergeJournalOrchestrator.cs:149`) drives the merge through nine phase

checkpoints, all reflected in `MergeJournal.Phase` (`Source/Parsek/MergeJournal.cs:53`). A load-time finisher rolls back (pre-Durable-1 phases) or drives to completion (post-Durable-1 phases).

- **Load-time sweep.** `LoadTimeSweep.Run` (`Source/Parsek/LoadTimeSweep.cs:51`) validates the re-fly marker's six durable fields via `MarkerValidator.Validate` (`Source/Parsek/MarkerValidator.cs:86`), discards zombie NotCommitted provisionals and session-scoped RPs not referenced by a valid marker, warn-logs orphan supersede/tombstone rows, and clears stray `SupersedeTargetId` fields. Normal staging RPs with `CreatingSessionId == null` are retained across merge-dialog scene loads.
- **Revert-during-re-fly dialog.** `RevertInterceptor` (`Source/Parsek/RevertInterceptor.cs`) prefixes `FlightDriver.RevertToLaunch` via Harmony and routes the player to `ReFlyRevertDialog` (`Source/Parsek/ReFlyRevertDialog.cs`) with three options: Retry from Rewind Point, Discard Re-fly, Continue Flying.

v0.9.1 stable-leaf extension:

- **Broadened predicate.** `EffectiveState.IsUnfinishedFlight` is rewritten to include stable-terminal non-focus controllable leaves (`Orbiting` , `SubOrbital`) and stranded EVA kerbals (`EvaCrewName != null AND terminal != Boarded`) alongside the v0.9.0 `Destroyed` qualification. The structural-leaf gate, controllable-subject gate, and slot-still-open gate are all preserved or tightened. Focus-slot continuation siblings stay excluded so routine upper stages do not pollute the list.
- **One Unfinished Flights group, broader membership.** No new virtual group. The existing tooltip and group affordances stay; the predicate behind it changes.
- **Per-row Seal action.** A new in-table action that closes a slot permanently without touching the underlying recording, so the player can clear over-included rows and let the reaper free disk space. Seal is decoupled from `MergeState` — see §6.24.
- **Per-row Stash action.** A complementary in-table action that promotes a default-excluded stable terminal leaf into Unfinished Flights while the backing Rewind Point still exists. Manual Stash is conservative: only spawnable stable terminals (`Landed` , `Splashed` , `Orbiting` , `SubOrbital`) qualify; `Recovered` , `Docked` , `Boarded` , and recordings carrying retry-blocking `ScienceEarning` rows are not stashable.
- **Re-Fly merge auto-seal (§4.9).** When the player Re-Flies a slot and the merge commits the chain tip at a stable terminal (`Orbiting` , `SubOrbital` , `Landed` , `Splashed` , plus the existing hard-safety set `Recovered` / `Docked` / `Boarded`), or the session authors a structural mutation (decouple / stage / undock / joint break / EVA), or a retry-blocking recording-linked `ScienceEarning` row was credited, the slot seals (`Immutable + slot.Sealed = true`). Terminal-failure outcomes (Destroyed, non-boarded EVA) without a retry-blocking action stay `CommittedProvisional` so the player can retry the failure.
- **New persistent fields.** `ChildSlot.Sealed` (+ `SealedRealTime`), `ChildSlot.Stashed` (+ `StashedRealTime`), `RewindPoint.FocusSlotIndex` , `ReFlySessionMarker.SupersedeTargetId` , and `ReFlySessionMarker.PreSessionBranchPointIds` . All back-compat — legacy ConfigNodes load with safe defaults.
- **Helper extraction.** `TryResolveRewindPointForRecording` and friends move from `UI/RecordingsTableUI.cs` into a new `UnfinishedFlightClassifier` static class so non-UI consumers (`RecordingStore` , `SupersedeCommit`) can call them without a layering inversion.
- **Closure-helper split + invocation linearization.** `EffectiveState.ComputeSessionSuppressedSubtree(marker)` becomes a thin wrapper around `ComputeSubtreeClosureInternal(marker, rootOverride)` . `RewindInvoker.AtomicMarkerWrite` stamps `marker.SupersedeTargetId = priorTip` so chain extension produces a linear graph instead of a star. Required prerequisite for the v0.9.1 Site B-1 merge classifier (see §6.22 and §10.10).

1.2 Out of scope

The shipped feature deliberately does NOT attempt:

- Re-emitting retired contract completions, milestone flags, tech research, facility upgrades, strategies, or the KSP subsystems that own them (`ContractSystem` , `ProgressTracking` , `ScenarioUpgradeableFacilities` , `StrategySystem` , `ResearchAndDevelopment`). These are "sticky" — KSP will not re-emit a contract completion after `ContractSystem` marks it complete, so tombstoning the event and hoping the re-fly emits a fresh one produces zero credit. The feature sidesteps this by leaving all non-kerbal-death ledger actions in the Effective Ledger Set regardless of whether their source recording was superseded. See §2, §3, §8, §10 for the exact framing.
- Nested re-fly sessions. The precondition gate rejects a rewind invocation while another session is active (`RewindInvoker.cs:108`).
- Cross-tree supersedes. The feature does not produce them; `EffectiveState.EffectiveRecordingId` does not yet halt at cross-tree boundaries but has a TODO marker for when cross-tree supersedes become producible (see Known Limitations).
- Auto-purge policies for long-lived reap-eligible RPs. Monitoring via the disk-usage diagnostic (with a v0.9.1 split into live-crashed / stable-open / sealed-pending buckets) is the answer; a TTL-based reaper or "Wipe All Sealed RPs" button is deferred.
- A recovery-snapshot mechanism for corrupt quicksaves. The pre-impl `SplitTimeSnapshot` concept was dropped before v0.9.0 shipped; the KSP quicksave is the sole source of truth.
- Voluntary-action heuristics for stable-leaf surfacing (orbit-shift, mid-chain-surface, body-change classifiers). The R1-R3 exploration in the stable-leaves research note explicitly rejected this; over-inclusion of the broadened predicate is handled by the per-row Seal button instead.
- Migration sweep for legacy star-shaped supersede graphs. The v0.9.1 closure-helper split (§6.22) tolerates legacy stars and grows linearly off whichever leaf the walker reaches; the existing star portions in pre-v0.9.1 saves are not flattened.
- An in-game un-Seal path. The Seal action is intentionally one-way; tree-scoped Full-Revert is the only undo. A complementary Un-Seal affordance is deferred unless playtest demands it.

1.3 Who benefits

- Players who launch rockets with recoverable boosters or stages and whose booster crashed instead of landing — they want to re-fly the booster back to the pad while the upper stage's ascent to orbit continues to play back as a ghost.
- Players whose kerbal died during an EVA that turned out to be avoidable (e.g. a fall without a parachute) — they want to replay the EVA so the kerbal survives, while the vessel the kerbal came from retains whatever stable state it actually reached (orbit, dock).
- Players who lost a lander on descent while the orbiting mothership survived — they want to re-fly the lander.
- (v0.9.1) Players running constellation-deploy missions (probes, satellites, ground stations) who want to come back and fly individual deployed objects later. The 4-probe mothership is the canonical case (§4.4): the player flies the mothership home, and the four probes appear in Unfinished Flights so any of them can be flown to a real mission later.
- (v0.9.1) Players whose EVA kerbal got stranded on a surface alive (suit ran out before reboard, fell off ladder during return). v0.9.0 only handled the death case; v0.9.1 handles the alive-but-unreboarded case.
- (v0.9.1) Players who left an upper stage in a sub-orbital arc they intended to circularize but never got around to.
- (v0.9.1) Players who want a cleanup tool: the per-row Seal button closes any UF slot permanently when the player has decided the recording is canonical, freeing the rewind point's quicksave on the next reaper sweep.

What this feature is **not** for: focus-continuation slots (your active mission's upper stage that reached orbit safely), routine dockings, and safe returns are excluded by default — they do not auto-populate Unfinished Flights. The v0.9.1 manual Stash button is the explicit escape hatch for stable terminals the player wants re-flyable later, and only while the backing Rewind Point still exists.

1.4 Relationship to prior features

Rewind to Separation is layered on top of, not inside, the existing subsystems:

- **Flight recorder:** the recording DAG, segment boundary rule, controller identity, ghost chains, background recording, terminal kinds. Rewind to Separation does not change any of these. It adds new persistent state — Rewind Points, supersede relations, tombstones, a session marker, a journal — stored alongside the existing recording tree in `ParsekScenario`.
- **Recording finalization:** Rewind-to-Separation assumes each sibling recording has a trustworthy terminal state and endpoint. The finalization reliability contract in `parsek-recording-finalization-design.md` is the upstream dependency that prevents Unfinished Flights from depending on stale last-sample inference when KSP unloads, deletes, or destroys a vessel before scene exit.
- **Timeline / ledger:** the immutable `ActionId`, the recalculation engine, the resource modules. The feature adds `GameAction.ActionId` as a hard precondition (legacy migration generates a deterministic hash on first load) and introduces `LedgerTombstone` as an append-only retirement filter, but the recalculation walk itself is unchanged — `LedgerOrchestrator.Recalculate*` now feeds from `EffectiveState.ComputeELS()` (the tombstone-filtered view) instead of raw `Ledger.Actions`.
- **Game actions & resources:** contracts, milestones, facilities, strategies, tech, science, funds, kerbals. v1 tombstones only kerbal deaths (plus bundled rep). Everything else sticks. This is the central design decision; see §2 and §10.

---

2. Design Philosophy

These principles governed every design and implementation decision. They are listed up front because they inform every section that follows.

2.1 Correct visually, minimal, efficient

Borrowed from the project-wide recording-design principle. A Rewind Point is 1–2 MB of quicksave. Many splits do not end in regret. Therefore: RP creation is cheap, deferred, and speculative; reap is eager (as soon as no slot can be re-flown, the file is deleted); cached ERS/ELS views rebuild only when source state changes; supersede filtering is a derived lookup, not a stored field. The v0.9.1 Unfinished Flight predicate runs per-frame on every committed recording in ERS; it is structured as cheap structural gates first, expensive terminal-state-and-focus checks last, with single-line shortcut returns. The closure helper is cached.

2.2 Append-only history; slot-level close signal

The recording tree never shrinks. Supersede is a relation stored in a separate list, not a mutation on a recording. Tombstones are append-only. No field on a committed Recording is written after its `MergeState` flips out of `NotCommitted`. `EffectiveState.IsVisible` (`Source/Parsek/EffectiveState.cs:129`) is computed by walking `RecordingSupersedeRelation` entries; the Recording itself carries no `SupersededByRecordingId` field. Tree-scoped rewind-state purge (triggered by merge-dialog Discard or any other whole-tree discard path, §6.17) is the only path that ever removes supersede relations or tombstones, and it clears all such rewind artifacts for the tree atomically. The Revert-during-re-fly dialog's Discard Re-fly option is NOT a tree purge — it is session-scoped (§6.14) and preserves the tree's supersede / tombstone / RP state. The tree's committed recordings themselves are never rewritten — separate pending-tree discard handles removal of pre-commit recordings, and committed recordings stay in the timeline indefinitely.

The v0.9.1 `ChildSlot.Sealed` flag is the slot-level close signal — set once on player invocation (or auto-set by the merge-time auto-seal contract, §4.9), never cleared in-game. Sealed is decoupled from `MergeState`: a recording's MergeState reflects the merge journal's outcome; a slot's Sealed reflects the player's choice (or the merge gate's verdict) to close that slot permanently. They serve different purposes — v0.9.0's existing legacy-Immutable-crash UF

rows continue to qualify because nobody will have Sealed them. The same decoupling applies to `ChildSlot.Stashed`: Stash is the player's intent signal that a default-excluded stable leaf should appear in Unfinished Flights, and never modifies the recording itself.

2.3 Narrow tombstone semantics; Seal as the player override

The central design decision. Supersede is a **physical-visibility / claim-tracker mechanism**, not a ledger or career-state eraser. The tombstone-eligible type list is deliberately small: `KerbalAssignment`-to-Dead and the `ReputationPenalty` actions bundled with them. Everything else — contracts, milestones, facilities, strategies, tech, science, funds, vessel-destruction rep — remains in the Effective Ledger Set after supersede. This produces behavior the player can reason about: "my career state from the failed attempt stays; only the kerbals come back." It also produces behavior the mod can reliably deliver, because re-emitting retired KSP events is not generally possible.

The same narrowness governs the v0.9.1 broadened predicate. The classifier auto-includes obvious-feeling cases (Crashed, Orbiting non-focus, SubOrbital non-focus, EVA-stranded). Over-inclusion is handled by the player Sealing the row. Under-inclusion (rover drove 20m and the player wants it re-flyable) is handled by Stashing the row while its backing Rewind Point still exists. **No heuristic predicates** beyond the simple terminal-state-plus-focus rule. The R1-R3 voluntary-action heuristic exploration was explicitly rejected.

2.4 Crash-recoverable merge

Every staged-commit merge step is journaled through `MergeJournal.Phase` (`Source/Parsek/MergeJournal.cs:53`). Nine phase strings mark the merge's progress; on load, the finisher reads the phase and either rolls back (pre-Durable-1 phases: disk still holds the pre-merge snapshot, in-memory changes evaporate) or drives to completion (post-Durable-1 phases: disk holds the merged state, the remaining steps resume). The journal IS the durability barrier; the next natural `ScenarioModule.OnSave` flushes each phase's in-memory state to disk. Tests inject a synchronous save stub to exercise every window (see `MergeCrashRecoveryMatrixTests.cs`).

2.5 Atomic provisional + marker write

The provisional re-fly Recording and its `ReFlySessionMarker` must land in the scenario in the same synchronous block; no KSP `OnSave` may capture the first without the second. `RewindInvoker.AtomicMarkerWrite` (`RewindInvoker.cs:463`) enforces this: the provisional is added, the marker is constructed and assigned, and a `try/catch` rolls back both if anything throws. There is no coroutine yield, no deferred save, and no frame gap between `CheckpointA:BeforeProvisional` and `CheckpointB:AfterMarker` — the `CheckpointHookForTesting` observer (`RewindInvoker.cs:47`) is the only thing that can see in between.

2.6 Player-visible opt-in

A re-fly never happens behind the player's back. It requires an explicit click on the Rewind button for an Unfinished Flight row; both the virtual group copy and the normal recordings-table copy route through `RewindInvoker` instead of the legacy tree-root launch rewind. The confirmation dialog names the canonical semantics (`RewindInvoker.ShowDialog` body at `RewindInvoker.cs:157`). Rewind Points are written automatically on every multi-controllable split, but they are invisible until a sibling ends up Unfinished — the player sees the UI entry only when it is actionable.

2.7 Observable from logs alone

Every decision point in §6 and every RP / marker / journal / sweep / reap state transition emits a log line. The tag catalog in §12 is what a KSP.log reader needs to reconstruct a session's history: the `[Rewind]`, `[RewindSave]`, `[RewindUI]`, `[ReFlySession]`, `[Supersede]`, `[LedgerSwap]`, `[MergeJournal]`, `[LoadSweep]`, `[UnfinishedFlights]`, `[CrewReservations]`, `[RevertInterceptor]`, `[Recording]`,

`[ReconciliationBundle]`, `[ERS]`, `[ELS]` tags appear exactly where §6 and §12 say they do. The v0.9.1 predicate gates emit `[UnfinishedFlights] Verbose` lines with structured reasons (`crashed`, `stableLeafUnconcluded`, `strandedEva`, `slotSealed`, `downstreamBp`, `noFocusSignalOrbiting`, `stableTerminalFocusSlot`, `recordingAction:*`); Site A and Site B-1 emit `[UnfinishedFlights] Info` on promotion with the resolved slot+RP; Seal accept/cancel logs; the reaper logs `sealedSlotsContributing=<n>`. A KSP.log reader can reconstruct why every row appeared or didn't, and what the Seal action did to each slot.

2.8 Predicate is shared; safety gates cap retry intent

The Unfinished-Flight membership predicate is shared between Site A (original tree commit) and Site B-1 (re-fly merge classification) for diagnostics and crash/stranded-EVA detection. Manual Stash is the player's intent signal for a stable leaf, but retry intent is capped by hard safety gates: no recovered/docked/boarded terminal, no downstream structural/world interaction, and no retry-blocking recording-linked action. Site A keeps `stableLeafUnconcluded` and `stashedStableLeaf` rows open so the player can come back to them — the original-tree commit does not represent player intent to conclude a slot. **Site B-1 (re-fly merge) is stricter:** the slot the player chose to Re-Fly is treated as the merge-time focus regardless of `RewindPoint.FocusSlotIndex`, and reaching a stable terminal on that slot now seals it, because the player explicitly engaged the slot to fly it themselves and the merge concludes that engagement. A Re-Fly that authors a structural branch point during the session (decouple / stage / undock / joint break / EVA) also seals on merge regardless of where the chain tip lands, since those are explicit player-driven shape mutations the player wants preserved. Terminal failure outcomes (`Destroyed`, or an EVA kerbal that did not board) remain retryable unless the recording has a retry-blocking `ScienceEarning` row; automatic consequence rows, KSC-scene player decisions (tech unlock, contract accept/cancel, hire, facility upgrade/repair, strategy toggle, vessel build cost), and tombstoneable kerbal-death ledger rows all stay retryable because a successful re-fly can retire them and a revert/retag preserves their effect. Background protection still applies for vessels the player merely controlled and then switched away from at scene exit (handled by Site A's natural-promotion path, which does not flow through the focus override). See §4.9 for the full Re-Fly auto-seal contract and §6.21 for the algorithm.

2.9 Layering: classifier owns the helpers, UI consumes

`TryResolveRewindPointForRecording`, `IsUnfinishedFlightCandidateShape`, `IsVisibleUnfinishedFlight` live in `UnfinishedFlightClassifier` (extracted in v0.9.1 from `UI/RecordingsTableUI.cs`). `RecordingStore`, `SupersedeCommit`, and the Seal handler all call them from a non-UI namespace. UI continues to consume the classifier; the classifier never reaches into UI.

2.10 Forward-only for vessels, retroactive for stranded kerbals

Pre-feature saves with legacy live RPs whose Orbiting siblings are Immutable do NOT retroactively populate UF — the `FocusSlotIndex == -1` short-circuit suppresses Orbiting/SubOrbital qualification on legacy RPs (no focus signal to discriminate routine upper stages from probe deploys). Stranded EVA kerbals from legacy saves DO retroactively appear (the EVA branch returns before the focus short-circuit). Stranded kerbals are unambiguous (the player wants them back); orbital siblings are ambiguous (intent unclear). This asymmetry is the only retroactive-surfacing exception in the v0.9.1 migration story; see §9.6 for the CHANGELOG split.

2.11 Career ledger is sticky across Seal and supersede

Unfinished Flights is a recording/slot affordance, not a career-ledger rewrite. Seal closes only the `ChildSlot` (see §4.7); it must not prune ledger actions, stored game-state events, or Effective Ledger Set (`EffectiveState.ComputeELS`) input. Re-fly merge appends supersede relations for ERS/ghost visibility and may append narrow action tombstones only for kerbal death/lost assignment actions plus their paired reputation penalty (`SupersedeCommit.CommitTombstones`). Science, milestones, contract completions, funds/rep rewards (except the rep penalty bundled with a kerbal death), facility upgrades, strategies, tech research, and spendings from superseded recordings remain in ELS. A later re-fly may append new actions, but duplicate or once-ever credit is resolved by the

recalculation modules, not by deleting the old action. This avoids paradoxes where KSP will not re-emit sticky career progress after the old recording is superseded.

---

3. Terminology

This section fixes the vocabulary. Throughout the doc, "recording" (lowercase) is the general concept; **Recording** (capitalized, code-font) is the class (`Source/Parsek/Recording.cs`). "Session" always means a Rewind-to-Separation re-fly session unless qualified.

- **Split event** — a `BranchPoint` whose type is `JointBreak` , `Undock` , or `EVA` (see `parsek-flight-recorder-design.md`). The common case is staging (joint break on the decoupler), but lander undocks and EVAs are the same shape.
- **Controllable entity** — a vessel with at least one `ModuleCommand` part, or an EVA kerbal. The classifier `SegmentBoundaryLogic.IsMultiControllableSplit` gates RP creation on `count >= 2` .
- **Multi-controllable split** — a split event that produces two or more controllable entities. Only these get a Rewind Point.
- **Rewind Point (RP)** — the durable object at a multi-controllable split. Holds the quicksave filename, the save UT, the ChildSlots, the PidSlotMap, the RootPartPidMap, plus bookkeeping fields. Defined in `Source/Parsek/RewindPoint.cs` . "Session-provisional" during re-fly; "persistent" after session merge.
- **Child slot** — a stable, ordered entry on an RP identifying one controllable output of the split. Carries `OriginChildRecordingId` (immutable) and derives `EffectiveRecordingId` via a forward walk through `RecordingSupersedeRelation` . Defined in `Source/Parsek/ChildSlot.cs` .
- **PidSlotMap** — `Dictionary<uint, int>` mapping `Vessel.persistentId` to `ChildSlot.SlotIndex` . Populated at quicksave-write time by `RewindPointAuthor.ExecuteDeferredBody` (`Source/Parsek/RewindPointAuthor.cs:202`). The authoritative primary match for the post-load strip.
- **RootPartPidMap** — `Dictionary<uint, int>` mapping root-part `Part.persistentId` to `ChildSlot.SlotIndex` . Populated at the same time as PidSlotMap. Fallback match when KSP has reassigned a vessel-level persistentId between save and load. `Part.persistentId` is the stable cross-save/load identity for a physical part; `Part.flightID` is session-scoped and unstable.
- **Finished recording** — a Recording with `MergeState == Immutable` .
- **Unfinished Flight** — a Recording matching `EffectiveState.IsUnfinishedFlight` (see §5). Visible in the ERS, terminal state classified as Crashed, parent BranchPoint carries a non-null `RewindPointId` , and a live RewindPoint entry exists for that BranchPoint.
- **Re-fly session** — the period between a Rewind Point invocation and its merge / discard. Identified by a `SessionId` GUID generated at invocation (`RewindInvoker.cs:230`). Ends on merge, discard, retry (new SessionId), full-revert, or load-time validation failure.
- **Session-provisional RP** — an RP whose `SessionProvisional` flag is `true` . Newly created RPs are session-provisional; the flag flips to `false` during the merge's `TagRpsForReap` step (`MergeJournalOrchestrator.cs:441`) for RPs whose `CreatingSessionId` matches the merging session. Load-time sweep discards session-provisional RPs only when they are session-scoped (`CreatingSessionId` non-empty) and not referenced by a valid marker; normal staging RPs created outside a re-fly session have `CreatingSessionId == null` , survive merge-dialog scene loads, and are promoted to persistent when the owning tree commits.
- **Supersede** — an append-only relation `RecordingSupersedeRelation { RelationId, OldRecordingId, NewRecordingId, UT, CreatedRealTime }` stored in `ParsekScenario.RecordingSupersedes` . No Recording is mutated; `EffectiveState.IsVisible` filters via the list.
- **Supersede chain** — the forward walk through `RecordingSupersedes` from a `ChildSlot.OriginChildRecordingId` . Chains can extend through multiple re-fly attempts (AB → AB' → AB'' if AB' was merged as a crashed `CommittedProvisional`).

- **Session-suppressed subtree** — during an active re-fly, a second narrow carve-out applied to physical-visibility subsystems only (ghost walker, chain tip, map presence, watch mode). Forward-only closure from the marker's `OriginChildRecordingId`, halting at mixed-parent BranchPoints. Each member also expands to its chain siblings (same `ChainId` AND same `ChainBranch`) so a `RecordingOptimizer.SplitAtSection` env split between HEAD and TIP is suppressed atomically — without this, the merge-time supersede would only retire the BP-linked HEAD and leave the terminal-Destroyed TIP visible. Different `ChainBranch` values stay independent (parallel ghost-only continuations). Computed by `EffectiveState.ComputeSessionSuppressedSubtree` (called from `Source/Parsek/SessionSuppressionState.cs`). Career and ledger subsystems do NOT consume this view — they read ERS / ELS directly.
- **Tombstone** — an append-only `LedgerTombstone { TombstoneId, ActionId, RetiringRecordingId, UT, CreatedRealTime }` in `ParsekScenario.LedgerTombstones`. Keyed by the retired `GameAction.ActionId`. Multiple tombstones for the same ActionId are tolerated; the ELS filter is "at least one tombstone exists." Defined in `Source/Parsek/GameActions/LedgerTombstone.cs`.
- **ReFlySessionMarker** — the singleton marker persisted while a session is live. Seven fields total; six are validated on load (`MarkerValidator.Validate`), `InvokedRealTime` is informational and never fails a load. Defined in `Source/Parsek/ReFlySessionMarker.cs`.
- **Provisional re-fly Recording** — the live Recording created at invocation with `MergeState == NotCommitted`, `CreatingSessionId` set to the session GUID, `SupersedeTargetId` set to the retired sibling's id, `ProvisionalForRpId` set to the invoked RP's id. Becomes `Immutable` or `CommittedProvisional` at merge.
- **MergeJournal** — the singleton journal that carries the merge through nine phase strings (`Begin`, `Supersede`, `Tombstone`, `Finalize`, `Durable1Done`, `RpReap`, `MarkerCleared`, `Durable2Done`, `Complete`). Its presence on load triggers the finisher. Defined in `Source/Parsek/MergeJournal.cs`.
- **Effective Recording Set (ERS)** — the derived, read-only view of committed recordings that count right now. Computed by `EffectiveState.ComputeERS` (`Source/Parsek/EffectiveState.cs:236`). Filters out `NotCommitted`, superseded, and (during an active session) session-suppressed recordings.
- **Effective Ledger Set (ELS)** — the derived, read-only view of ledger actions that count right now. Computed by `EffectiveState.ComputeELS` (`EffectiveState.cs:310`). Filters out actions whose `ActionId` appears in any `LedgerTombstone`. **The ONLY filter on ELS is the tombstone check.** A superseded recording's non-eligible actions (contract completions, milestones, facility upgrades, etc.) remain in ELS.
- **UF** — Unfinished Flight. A recording that satisfies `IsUnfinishedFlight(rec)` and is therefore visible in the Unfinished Flights virtual group.
- **Stable terminal** — a terminal state in `{ Orbiting, SubOrbital, Landed, Splashed, Recovered, Docked, Boarded }`. The opposite is `Destroyed` (Crashed) and the absent terminal (no `TerminalStateValue` set).
- **Stable leaf** — a recording whose chain TIP has a stable terminal AND whose `ChildBranchPointId` is null OR equals the matched RP's `BranchPointId` (the breakup-survivor case).
- **Focus slot** — the `ChildSlot` in a `RewindPoint` whose vessel was the active focus at the moment the multi-controllable split fired. Identified by `RewindPoint.FocusSlotIndex` (default `-1` = no focus signal).
- **Non-focus slot** — any slot other than the focus slot. `slot.SlotIndex != RP.FocusSlotIndex AND RP.FocusSlotIndex >= 0`.
- **No-focus-signal RP** — a RewindPoint with `FocusSlotIndex == -1`. Either a legacy RP that predates the v0.9.1 field, OR a new RP where no slot was focused at split time (rare: e.g. the player was focused on an unrelated vessel outside the split).
- **Sealed slot** — a `ChildSlot` with `Sealed == true`. Either the player explicitly closed this slot via the Seal button, or the Re-Fly merge auto-seal gate (§4.9) flipped it after a committed re-fly produced a hard-close signal such as a stable terminal on the player-chosen slot, a credited `ScienceEarning` (Crew Report / EVA Report / Surface Sample / Transmit / Recover), a structural mutation (decouple / stage / undock / EVA

/ joint break), or a hard safety terminal (Recovered / Docked / Boarded). The row drops from UF and the reaper treats it as equivalent-to-Immutable for reap eligibility. Persistence stores `Sealed` / `SealedRealTime`; the auto-seal reason is logged at merge time, while persisted `SealedBy` / `SealedReason` metadata is a deferred schema follow-up tracked in `docs/dev/todo-and-known-bugs.md`.

- **Stashed slot** — a `ChildSlot` with `Stashed == true`. The player explicitly promoted a default-excluded stable terminal leaf into Unfinished Flights via the Stash button. Only spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) qualify; `Recovered`, `Docked`, `Boarded`, and rows carrying retry-blocking `ScienceEarning` are rejected by the resolver. Stash does not resurrect already-reaped RP quicksaves.
- **Stranded EVA** — an EVA kerbal recording whose chain TIP has `EvaCrewName != null` AND a non-`Boarded` terminal (typically `Landed` for surface strands, `Orbiting` for drift strands; `Destroyed` is a dead kerbal and routes through the Crashed branch).
- **Prior tip** — the slot's current effective recording id at re-fly invocation time, before any new supersede relation has been appended for the current re-fly. Equals `slot.OriginChildRecordingId` on the first re-fly into a slot; equals the previous re-fly's recording id on chain extension. Stamped into `ReFlySessionMarker.SupersedeTargetId` by `RewindInvoker.AtomicMarkerWrite` (v0.9.1, see §6.22).
- **Site A** — `RecordingStore.ApplyRewindProvisionalMergeStates`. The MergeState-promotion call site that runs at original tree commit. Reads the broadened predicate; promotes stable-leaf siblings to `CommittedProvisional` so they appear in Unfinished Flights.
- **Site B-1** — `SupersedeCommit.FlipMergeStateAndClearTransient`. The MergeState-promotion call site that runs at re-fly merge. Applies the v0.9.1 auto-seal contract using the merge-time slot as `focusSlotOverride`.
- **Site B-2** — `MergeDialog.TryCommitReFlySupersede`'s in-place continuation path. Consumes Site B-1's verdict; clean stable Re-Fly outcomes commit `Immutable` + auto-seal at Site B-1 and Site B-2 lets the reaper remove the RP. Safety-closed outcomes follow the same path. Terminal-failure outcomes (`Destroyed`, non-boarded EVA) without a retry-blocking action stay `CommittedProvisional` so the player can retry the failure.

---

4. Mental Model

The tree diagrams below use `(V)` for "visible in ERS" and `(H)` for "hidden (superseded)". `Immutable` is abbreviated `I`; `CommittedProvisional` is `CP`; `NotCommitted` is `NC`.

4.1 A mission with one staging split, one side crashes, player re-flies and lands

Launch: one vessel, `NotCommitted`, recording live.

```
ABC (NC, live)   (V)
```

Staging: `ABC` splits into upper stage `AB` (continues to orbit) and booster `A` (falls back). Both are controllable (probe cores on both). `SegmentBoundaryLogic.IsMultiControllableSplit` returns true; `RewindPointAuthor.Begin` creates RP-1 with two ChildSlots and schedules the deferred quicksave. On the next frame the quicksave is written and atomically moved into `Parsek/RewindPoints/rp_<guid>.sfs`; `PidSlotMap` and `RootPartPidMap` are populated from live vessels.

```
ABC (NC, terminates at split-1)   (V)
 |
 +-- [slot-0: AB]  (NC, live, focus)        (V)
```

```
 +-- [slot-1: A]   (NC, BG-recording)       (V)
     RP-1 (SessionProvisional, maps captured)
```

Orbit + merge: player flies AB to orbit, then returns to Space Center. Merge dialog appears. Booster A crashed in the background. On "Merge to Timeline" the recorder commits ABC and AB as `Immutable`; A's terminal kind classifies as `Crashed`, so its `MergeState` becomes `CommittedProvisional`. RP-1's slots: slot-0's effective recording is AB (`Immutable`), slot-1's effective recording is A (`CommittedProvisional`). RP-1 is NOT reap-eligible (slot-1 is still open), so the quicksave and scenario entry persist.

```
ABC (I)                                    (V)
 |
 +-- AB  (I, Orbited)                      (V)
 +-- A   (CP, Crashed)                     (V)  <- Unfinished Flight
     RP-1 persistent
     Unfinished Flights = {A}
```

`A` now satisfies `IsUnfinishedFlight`: visible, terminal kind classified as Crashed, parent BranchPoint has a live RP. The Recordings Manager shows it in the Unfinished Flights virtual group with a Rewind button.

Invocation: player clicks Rewind on row A. `RewindInvoker` runs through preconditions, captures the reconciliation bundle, copies the RP quicksave to the save-root as `Parsek_Rewind_<sessionId>.sfs`, triggers `GamePersistence.LoadGame` + `HighLogic.LoadScene(FLIGHT)`. On the reloaded scene, `ParsekScenario.OnLoad` calls `RewindInvoker.ConsumePostLoad`, which atomically:

1. Restores the pre-load reconciliation bundle over the quicksave-loaded state (recordings, supersedes, tombstones, reservations, etc. preserved).
2. Runs `PostLoadStripper.Strip(rp, slotIdx=1)` — AB is matched via `PidSlotMap` and stripped (`Vessel.Die()`); A is matched and kept; unrelated vessels are left alone; ghost ProtoVessels are skipped.
3. Calls `FlightGlobals.SetActiveVessel(A)`.
4. Creates the provisional re-fly Recording `A'` with `MergeState = NotCommitted`, `CreatingSessionId = sessionId`, `SupersedeTargetId = A.Id`, `ProvisionalForRpId = rp.Id`, and writes the `ReFlySessionMarker` — same synchronous block, no yield.
5. Recalculates the ledger via `LedgerOrchestrator.RecalculateAndPatch()`.

During the re-fly: ERS = `{ABC, AB, A}`. A' is `NotCommitted` so it is not in ERS. The ghost walker further filters ERS by `SessionSuppressedSubtree(marker)`, which contains `{A}` — so A does not render as a ghost. AB and ABC play back as ghosts. A' is the live active vessel, handled by the normal active-vessel path.

```
ABC (I)                                    (V)
 |
 +-- AB  (I)                               (V)  <- ghost playback
 +-- A   (CP, Crashed)                     (V)  <- in SessionSuppressedSubtree (invisible)
 +-- A'  (NC, live, CSId=sess1,            (V)  <- active vessel
          SupersedeTargetId=A)
     ReFlySessionMarker active
```

Successful re-fly + merge: player lands A' on the launch pad. Recording terminates with `TerminalKind = Landed` (via `TerminalKindClassifier`). Merge dialog fires `MergeJournalOrchestrator.RunMerge`. The orchestrator:

1. Writes the journal at `Phase = Begin`.
2. `SupersedeCommit.AppendRelations`: subtree closure for A is `{A}`; one `RecordingSupersedeRelation { OldRecordingId = A.Id, NewRecordingId = A'.Id, UT = now }` appended.

3. `SupersedeCommit.CommitTombstones` : scans ledger actions in-scope; no kerbal deaths in A's scope → zero tombstones.
4. `SupersedeCommit.FlipMergeStateAndClearTransient` : A' becomes `Immutable` (Landed). `SupersedeTargetId` cleared. Supersede state version bumped. Marker preserved for step 7.
5. Durable Save #1 (deferred — journal phase string is the durable barrier).
6. `TagRpsForReap` : RP-1's `SessionProvisional` was already false (from merge-1); its `CreatingSessionId` does not match sess1 (it was `null` when A's tree committed), so nothing is tagged here. `RewindPointReaper.ReapOrphanedRPs` : RP-1's slot-0 effective is AB (`Immutable`), slot-1's effective is A' (`Immutable`). Both slots are `Immutable` → RP-1 is reap-eligible. Quicksave file deleted, scenario entry removed, BranchPoint.RewindPointId cleared.
7. Marker cleared. Supersede state version bumped again.
8. Durable Save #2 (deferred).
9. Journal set to `Complete` , then cleared. Durable Save #3 (deferred).

After merge: A is hidden (superseded), A' is the effective representative of slot-1. Unfinished Flights is empty.

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, Crashed, SUPERSEDED)       (H)
 +-- A'  (I, Landed)                     (V)  effective slot-1
     RP-1 reaped (quicksave deleted)
     RecordingSupersedes: {A -> A'}
     Unfinished Flights = {}
```

4.2 Crashed re-fly stays rewindable (chain extension)

Same setup as §4.1 except A' also crashes. The player wants to try again.

On merge: `TerminalKindClassifier.Classify(A')` returns `Crashed` . `SupersedeCommit.FlipMergeStateAndClearTransient` assigns `A'.MergeState = CommittedProvisional` . Supersede relation `{A -> A'}` still appends. A is hidden; A' is the new effective slot-1 representative AND a new Unfinished Flight (it satisfies the predicate: visible, CP, Crashed, parent BP has RP-1).

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, Crashed, SUPERSEDED)       (H)
 +-- A'  (CP, Crashed)                   (V)  <- NEW Unfinished Flight
     RP-1 still persistent (slot-1 still open)
     RecordingSupersedes: {A -> A'}
     Unfinished Flights = {A'}
```

The player invokes RP-1 again. `PidSlotMap` still carries the original vessels' PIDs from the quicksave — that's the same quicksave, nothing about it changed. After strip + activate, a new provisional `A''` is created with `SupersedeTargetId = A'.Id` (note: targets `A'` , not `A` ; the UI's "effective recording for slot-1" is A' now).

A'' lands safely. Merge. `TerminalKind = Landed` , so `A''.MergeState = Immutable` . Supersede relation `{A' -> A''}` appends. The chain is now `A -> A' -> A''` ; `ChildSlot.EffectiveRecordingId(supersedes)` walks forward: start at A, find `{A -> A'}` , step to A', find `{A' -> A''}` , step to A'', no further relation → return A''. Slot-1's effective recording is A''. Both slots immutable → RP-1 reap-eligible → quicksave deleted, scenario entry removed.

```
ABC (I)                                  (V)
 |
 +-- AB  (I)                             (V)
 +-- A   (CP, H)                         (H)
 +-- A'  (CP, H)                         (H)
 +-- A'' (I, Landed)                     (V)  effective slot-1
     RP-1 reaped
     RecordingSupersedes: {A -> A'}, {A' -> A''}
     Unfinished Flights = {}
```

Key invariants shown:

1. The tree never shrinks. Every provisional stays around; only relations accumulate.
2. Supersede chains can extend arbitrarily through crashed re-flies.
3. Only `Immutable` closes the rewind opportunity; `CommittedProvisional` keeps the slot rewindable.
4. Career state reads ELS, which is unchanged by supersede in this scenario (no kerbal deaths).

4.3 Session-suppressed subtree with mixed-parent halt

The session-suppressed subtree is the physical-visibility carve-out applied during an active re-fly. It is a **forward-only** walk from the marker's `OriginChildRecordingId`, stopping at any descendant whose parents include a recording outside the subtree.

Suppose mission `ABC` decouples into `AB` and `C` (split-1). `AB` then decouples into `A` and `B` (split-2). Later, `B` docks with a station `S` from an unrelated tree; the dock merges B with S into a new recording `BS`.

```
ABC
 |
 +-- AB
 |    |
 |    +-- A
 |    +-- B ----+
 |              |
 +-- C          v
               BS  (parents = {B, S-tree-tip})
 S-tree:
   S (own lineage)
```

Player invokes RP at split-1 targeting C (C crashed). SessionSuppressedSubtree is the forward closure of `OriginChildRecordingId = C.Id`, which is just `{C}` because C has no descendants — that's fine.

Now suppose the player invokes RP at split-1 targeting AB (AB crashed — hypothetical). The closure starts at AB, walks to its descendants {A, B}, and would then consider BS. But BS has parents `{B, S-tree-tip}`, and `S-tree-tip` is NOT in the subtree — the closure halts here. `BS` stays visible in ERS during the session; the mixed-parent constraint means a descendant that has any outside parent is not session-suppressed. Without this halt, suppressing BS would also suppress S-tree activity that the player may be relying on.

Chain-sibling expansion. Each member added to the closure also expands to its chain siblings — committed recordings sharing both `ChainId` and `ChainBranch`. Merge-time `RecordingOptimizer.SplitAtSection` splits a single live recording at env boundaries (atmo↔exo) into a `ChainId`-linked HEAD + TIP where the HEAD keeps the parent-branch-point link and the TIP carries the terminal state. Without this expansion, walking by `ChildBranchPointId` alone would dequeue the HEAD, find no BP descendant (its `ChildBranchPointId` was moved to the TIP at split time), and stop — leaving the terminal-Destroyed TIP visible alongside the new "kerbal

lived" provisional after merge. Different `ChainBranch` values stay independent (parallel ghost-only continuations are not auto-suppressed together). Same dual-key contract as `EffectiveState.IsChainMemberOfUnfinishedFlight` and `ResolveChainTerminalRecording`.

4.4 The 4-probe canonical example (v0.9.1 stable-leaf extension)

The motivating gameplay for the v0.9.1 broadened predicate:

```
Mun orbit, mothership M with 4 attached probes P1-P4.
Player triggers staging; all 4 decouplers fire simultaneously.
Multi-controllable split: M + P1 + P2 + P3 + P4 = 5 controllables.
RewindPointAuthor.Begin captures RP-1 with 5 ChildSlots.
RP-1.FocusSlotIndex = (slot index for M)  // the player's active vessel.

Player flies M back to Kerbin. P1-P4 background-record their parking coast.
Tree commits.
```

Under v0.9.0 alone: all 5 recordings commit `Immutable` (none crashed). `RewindPointReaper.IsReapEligible` sees all-slots-Immutable → reap RP-1. Quicksave deleted, BranchPoint.RewindPointId cleared. P1-P4 are inert rows in the Recordings table forever.

Under the v0.9.1 extension:

```
Site A (ApplyRewindProvisionalMergeStates) walks tree.Recordings:
  M:   focus slot, terminal Orbiting (Kerbin) -> not UF -> Immutable
  P1:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P2:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P3:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional
  P4:  non-focus, terminal Orbiting (Mun)     -> UF        -> CommittedProvisional

RewindPointReaper.IsReapEligible: 4 slots are CP (open) -> RP-1 stays alive.
Recordings Manager Unfinished Flights group shows P1, P2, P3, P4 with Fly + Seal buttons.
```

Player can pick any probe individually, hit Fly, rewind to the staging UT, and fly that probe to a real mission (land it, transfer it elsewhere, etc.). The other three probes ghost-play-back from their committed coast. Merge produces a re-fly that supersedes the original probe. Per the v0.9.1 auto-seal contract (§4.9): if the re-fly reaches a stable terminal (`Orbiting`, `SubOrbital`, `Landed`, `Splashed`) on the chosen slot, OR authors a structural branch point during the session (decouple / stage / undock / joint break / EVA), the merge commits `Immutable` and auto-seals the slot — the player engaged this slot to fly it themselves and the merge concludes that engagement. Recovers, docks, boards, downstream structural/world interaction, and retry-blocking recording-linked actions all still trigger the existing safety-close path. Destroyed / non-boarded-EVA outcomes stay `CommittedProvisional` for another retry unless they also contain a retry-blocking action. If the player decides one probe is "done" before re-flying it, they hit Seal and the row drops.

4.5 Focus exclusion: routine upper stages don't pollute the list

```
Two-stage rocket: booster B (probe core, parachutes) + upper stage U (player's mission).
Player flies U to orbit. B BG-records, parachutes auto-deploy, B lands safely.
Tree commits.

RP-1.FocusSlotIndex = (slot index for U)
B:  non-focus, terminal Landed   -> not UF (Landed is "stable conclusion")
```

```
U:  focus, terminal Orbiting     -> not UF (focus exclusion)

All slots Immutable -> RP reaps. Same as v0.9.0 behavior.
```

Inversely (the "spaceplane that recovers a strap-on booster" case):

```
Player flies B (the booster) to recover it. U (upper stage) BG-records, ends Orbiting.

RP-1.FocusSlotIndex = (slot index for B)
B:  focus, terminal Landed       -> not UF (focus exclusion + Landed)
U:  non-focus, terminal Orbiting -> UF -> CommittedProvisional
```

The booster is excluded as the focus mission; the upper stage is the unfinished off-mission sibling. RP stays alive while U's slot is open.

4.6 EVA carve-out: stranded kerbals bypass focus, not safety

The kerbal branch in `TerminalOutcomeQualifies` runs BEFORE the focus short-circuit. A stranded EVA kerbal can qualify as UF whether the player flew the EVA actively or not, and whether the RP is post-feature (with FocusSlotIndex set) or legacy (FocusSlotIndex == -1). Stranded kerbals are unambiguous (player wants them back); orbital siblings are ambiguous (intent unclear). This asymmetry is the only retroactive-surfacing exception in the migration story (see §9.6). It is still capped by the retry-blocking recording-action gate from §2.8: a stranded-EVA recording that already produced retry-blocking recording-linked actions closes with `recordingAction:<type>:<actionId>`, while automatic consequence rows, kerbal-death rows, and their paired reputation penalty stay retryable.

4.7 Seal closes a slot, not a recording

The player Seals a slot via the per-row Seal button. The seal flips `slot.Sealed = true` on the matching `ChildSlot`. It does NOT touch `Recording.MergeState`. The recording continues to play back as a ghost on any future rewind, exactly as before; only the re-fly opportunity for that slot is closed.

The reaper (§6.23) treats `slot.Sealed == true` as equivalent-to-Immutable for close-eligibility, BUT requires the effective recording to be in a committed MergeState (Immutable or CommittedProvisional). NotCommitted is unconditional no-reap regardless of Sealed — defends against load-time race states where a journal finisher rolled MergeState back to NotCommitted while the slot's Sealed bit is still on disk.

4.8 Tree-branching parent that ends Destroyed

A tree-branching split can capture a RewindPoint with both the continuing parent vessel and the new side-off child as controllable slots without closing the parent's Recording row. In that model, the parent keeps flying under the same `RecordingId` after the BranchPoint, so `Recording.ParentBranchPointId` and `Recording.ChildBranchPointId` can both stay empty even though the RP slot's `OriginChildRecordingId` points at the parent. If the parent later ends `Destroyed`, it is still an unfinished flight when the retry-blocking recording-action gate is clean: destruction is conclusive and focus exclusion does not apply, but retry-blocking recording-linked actions still close the retry row. The same origin-only route can surface a non-focus tree-branching parent that later ends `Orbiting` or `SubOrbital`, matching the stable-leaf focus-exclusion rules from §4.4-§4.5.

Resolution order is part of the contract. `UnfinishedFlightClassifier.TryResolveRewindPointForRecording` first walks the legacy child/parent BranchPoint links so chain-continuation recordings keep their existing precedence. Only after those walks fail does it scan live RewindPoints and call `EffectiveState.ResolveRewindPointSlotIndexForRecording` to match by slot origin id, choosing the latest matching RP by UT when the same parent recording appears in multiple origin slots. `TryQualify` labels this fallback `side=origin-only` when the matched RP is justified by the slot origin rather than by a parent/child

BranchPoint match. The log line therefore distinguishes the new tree-branching-parent path from the existing `side=active-parent-child` chain-continuation parent and regular `side=child` branches.

The downstream-BranchPoint guard is naturally a no-op for this shape because the still-open parent is its own chain tip and has no `ChildBranchPointId`. The controllable gate still applies through the RP slot and `rec.IsDebris`, so debris breakup rows do not become Re-Fly candidates merely because they share a parent id. Site B-1 merge classification uses the same origin-slot idea against the marker's supersede target for separate provisional re-flights; in-place continuations keep their existing terminal-kind fallback policy.

4.9 Re-Fly auto-seal contract (v0.9.1)

The original v0.9.0 design held that a clean stable Re-Fly outcome stayed `CommittedProvisional` so the player could keep synchronizing the slot against other recordings. v0.9.1 playtesting showed this conflicts with the player's mental model: when the player explicitly *picks a slot to Re-Fly* and *flies it themselves*, the merge concludes their engagement with that slot. Leaving the slot open meant a successful re-fly to orbit still showed up as Unfinished and offered a "Fly" button the player did not want. Worse, it caused confusion about whether the original or the re-flown timeline was authoritative.

Revised contract. Site B-1 (re-fly merge) seals the slot — `Immutable` recording + `slot.Sealed=true` — when ANY of these is true:

1. **Player-chosen slot reached a stable terminal** (`Orbiting`, `SubOrbital`, `Landed`, `Splashed`, plus the existing hard-safety set `Recovered` / `Docked` / `Boarded`). The player engaged the slot to fly it; the merge concludes the engagement.
2. **Structural mutation during the session** — the Re-Fly authored a new `BranchPoint` of type `Breakup`, `Undock`, `EVA`, or `JointBreak` in the provisional's tree. Decouple, stage, undock, joint break, kerbal EVA — these are explicit player-driven shape mutations the player wants preserved, regardless of where the chain tip ends.
3. **Retry-blocking recording-linked action** — credited science (`ScienceEarning` from `Crew Report` / `EVA Report` / `Surface Sample` / `Transmit` / `Recover`). This is the only action type that auto-seals via the recording-action gate. Automatic consequences (`MilestoneAchievement`, funds/rep earnings, contract complete/fail, facility destruction, kerbal assignment/rescue/stand-in) do not close a retry slot by themselves. Other player decisions (`ScienceSpending`, `FundsSpending`, `ContractAccept` / `Cancel`, `KerbalHire`, `FacilityUpgrade` / `Repair`, `StrategyActivate` / `Deactivate`) also do not close a retry slot — they emit from KSC scenes with no flight-recording tag and so cannot reach the gate in practice; the one rollout-adoption case (`FundsSpending(VesselBuild)` retroactively tagged via `TryAdoptRolloutAction`) is paid once and survives revert/retag, so sealing on it would punish retries for spending the player already accepted.
4. **Downstream structural/world interaction** — the chain tip's `ChildBranchPointId` resolves to a deeper RP. (Existing rule, unchanged.)

The slot **stays open** (`CommittedProvisional`, `slot.Sealed=false`) only on terminal-failure outcomes — `Destroyed`, or an EVA kerbal that did not board — so the player can retry the failure. Automatic gameplay/career consequence rows and tombstoneable kerbal-death + bundled reputation penalty still keep the slot open because a successful retry does not need to undo sticky history and can retire death rows.

Site A is unaffected. The original-tree-commit predicate that surfaces stable-leaf siblings into Unfinished Flights does not know about Re-Fly intent; it continues to promote `stableLeafUnconcluded` and `stashedStableLeaf` rows so the player can come back to them. The auto-seal revision lives entirely in the Re-Fly merge call site (Site B-1 / Site B-2). Background vessels the player merely controlled and then switched away from at scene exit still take the Site A path and stay re-flyable.

Mechanics.

- *Focus override.* `UnfinishedFlightClassifier.TryQualify` accepts an optional `int? focusSlotOverride`. `SupersedeCommit.ClassifyMergeStateOrThrow` populates it with the merge-time `slotListIndex`. The override path runs **early** in `TerminalOutcomeQualifiesInternal` — after the Destroyed and EVA branches (so terminal-failure outcomes still take their existing retry-keep-open path) but **before** the stashed-keep-open branch and **before** the static-focus / `noFocusSignalOrbiting` checks. When the override matches the resolved slot AND the chain tip is in `{Orbiting, SubOrbital, Landed, Splashed}`, the classifier runs the retry-blocking recording-action gate first (so player-action `recordingAction:*` reasons still win, while automatic consequence rows are ignored) and otherwise returns `reason=stableTerminalFocusSlot`. Recovered/Docked/Boarded fall through to the existing `stableTerminal` close + `IsHardSafetyTerminal` auto-seal path because they are already correctly handled there. The verdict log emits `focusSlot=N focusSlotOverride=M` (separate fields) so the source of the focus decision is greppable in both `[Supersede]` and `[UnfinishedFlights]` lines.
- *Override-before-stashed.* The override precedes the stashed-keep-open branch by design: a player who Re-Flies a stashed slot to a stable conclusion is concluding the stash, not extending it. Otherwise stashed slots would always escape the override and remain re-flyable indefinitely.
- *Override-before-noFocusSignal.* The override precedes the `rp.FocusSlotIndex < 0` early return because the override IS the focus signal for this merge. Without this ordering, a Re-Fly merge from an RP captured with no static focus (player was controlling an unrelated vessel at split time) would log `noFocusSignalOrbiting` and never reach the override seal.
- *`stableTerminalFocusSlot` auto-seals slot.* `SupersedeCommit.ShouldAutoSealReFlySlotAfterMerge` flips `slot.Sealed=true` for `closeReason.Detail == "stableTerminalFocusSlot"` on both the static-focus and override paths.
- *Structural-mutation gate.* `SupersedeCommit.HasReFlySessionStructuralMutation(provisional, marker)` walks the provisional's tree (resolved via `marker.TreeId`) for any `BranchPoint` with `Type ∈ {Breakup, Undock, EVA, JointBreak}` whose `UT > rp.UT` (the resolved rewind-point UT, NOT `marker.InvokedUT` — the latter is the live UT at click time, typically much greater than rp.UT and would reject normal post-rewind decouples as if pre-rewind). The gate excludes any BP whose id is in the marker's new `PreSessionBranchPointIds` baseline (snapshot at marker creation, prevents `SpliceMissingCommittedRecordingsIntoLoadedTree` false positives from re-grafted pre-RP BPs). The gate also scopes detection to BPs whose `ParentRecordingIds` intersects the **Re-Fly target's lineage set** — the provisional's recording id plus every committed recording sharing `(TreeId, ChainId, ChainBranch)` with the provisional. Without this lineage scope, a background sibling vessel in the same tree that staged / undocked during the session would author a structural BP whose parent is the sibling, not the player-chosen slot, and the gate would auto-seal a slot the player never mutated. Legacy markers with `PreSessionBranchPointIds=null` skip the gate conservatively. Detection fires inside `ShouldKeepReFlySlotOpenAfterMerge` after the existing `IsTerminalFailureReFlyOutcome` keep-open branch, so crashed retries are unaffected.
- *Reachability under the focus override.* The override path closes every stable-terminal Re-Fly outcome on the player-chosen slot before the structural gate runs, so the structural-mutation gate is now a defensive backstop rather than the primary seal mechanism for Re-Fly merges. It remains testable at the helper level and would still fire if a future call site ever invoked `ShouldKeepReFlySlotOpenAfterMerge` from a path that does not pass the override.

What does NOT count as structural mutation. Detection is on `BranchPointType`, not on `Recording.PartEvents`. Solar panel deploy, ladder extend, landing leg toggle, cargo bay open/close, control surface deflect, animation toggle, gear lower/raise — none of those author a `BranchPoint` and none of them auto-seal. Branch points are only authored when KSP changes vessel topology (decouple / stage / undock / joint break / EVA / dock / board / launch / terminal); only the structural-split subset closes the slot.

Reap consequence. An auto-sealed slot may make its rewind point reap-eligible. `MergeDialog.TryCommitReFlySupersede`'s post-merge reap call deletes the RP's quicksave file when all slots in that RP are closed. A successful Re-Fly to stable orbit, or a Re-Fly that staged debris, cannot be re-flown again afterward — the on-disk file is gone. This is the intended outcome ("player is done with that line of flight") but is irreversible for that session.

---

5. Data Model

Every class in this section lives in `Source/Parsek/`. ConfigNode VALUE keys are listed alongside the node name so the save-file layout is self-documenting.

5.1 MergeState

File: `Source/Parsek/MergeState.cs`.

```
public enum MergeState
{
    NotCommitted = 0,
    CommittedProvisional = 1,
    Immutable = 2
}
```

Tri-state replacement of the legacy binary `committed` flag. Legacy saves migrate on load: `committed = True` → `Immutable`; `committed = False` → `NotCommitted`; absent field → `Immutable` (the pre-feature invariant — every recording reachable from a committed tree was already sealed). Serialized as `mergeState = <value>` on the RECORDING node.

- `NotCommitted`: recording is still being produced (recorder live, or re-fly provisional with `SupersedeTargetId` set).
- `CommittedProvisional`: session merge has stamped the recording but the rewind slot remains available (BG-recorded children under an RP; re-fly that ended in a crash and is still re-rewindable).
- `Immutable`: recording is sealed. Never mutated after this point; the rewind slot is closed.

5.2 Recording additions

File: `Source/Parsek/Recording.cs:193-208`.

Field	Type	Purpose
`MergeState`	`MergeState`	Tri-state commit state (§5.1).
`CreatingSessionId`	`string`	Session GUID for recordings produced during an active re-fly. `null` outside sessions. Used by the load-time spare-set logic (§6.16) when a session crashed.
`SupersedeTargetId`	`string`	Transient — set only on `NotCommitted` provisional re-fly recordings to signal the intended supersede target. Cleared at merge when a concrete `RecordingSupersedeRelation` is appended. A non-empty value on `Immutable` / `CommittedProvisional` recordings triggers a Warn log and is treated as cleared (load-path legacy safety; see §6.16 step 5 and `LoadTimeSweep.ClearStraySupersedeTargets`).

`ProvisionalForRpId`	`string`	Back-pointer to the invoked RP's id. Set at invocation by `RewindInvoker.BuildProvisionalRecording` (`RewindInvoker.cs:527`). Null outside sessions.

`SupersededByRecordingId` is **NOT** a field on Recording. Visibility is a derived lookup through `RecordingSupersedeRelation`.

5.3 BranchPoint additions

File: `Source/Parsek/BranchPoint.cs:47`.

Field	Type	Purpose
`RewindPointId`	`string`	The id of the RP attached to this BranchPoint, or `null` if no RP was written (fewer than two controllable children, scene-guard abort, save failure). Stamped synchronously by `RewindPointAuthor.Begin` (`RewindPointAuthor.cs:138`).

Serialized as `rewindPointId = <value>` on the BRANCH_POINT node when non-null.

5.4 GameAction additions

File: `Source/Parsek/GameActions/GameAction.cs`.

Field	Type	Purpose
`ActionId`	`string`	Stable, immutable `act_<Guid-N>` id. Precondition for tombstoning. Legacy migration at load generates a deterministic hash from `UT + Type + RecordingId + Sequence` (`GameAction.cs:541`), so the same legacy action always yields the same id on repeated loads — tombstones written on one load resolve cleanly on the next.

Migration log on first-post-upgrade load: `[LegacyMigration] Info: Assigned deterministic ActionId to <N> legacy actions`. Idempotent — replaying the migration on a save that already carries ActionIds is a no-op.

5.5 RewindPoint

File: `Source/Parsek/RewindPoint.cs`.

Field	Type	Purpose
`RewindPointId`	`string`	Stable id in the format `rp_<Guid-N>`. Must satisfy `RecordingPaths.ValidateRecordingId`.
`BranchPointId`	`string`	Weak link back to the `BranchPoint`.
`UT`	`double`	`Planetarium.UT` at quicksave write.
`QuicksaveFilename`	`string`	Relative path under the save dir; format `Parsek/RewindPoints/<rpId>.sfs`.
`ChildSlots`	`List<ChildSlot>`	One entry per controllable sibling at split time.
`PidSlotMap`	`Dictionary<uint, int>`	`Vessel.persistentId` → `ChildSlot.SlotIndex`. Primary match for post-load strip.

`RootPartPidMap`	`Dictionary<uint, int>`	Root `Part.persistentId` → `ChildSlot.SlotIndex`. Fallback when KSP reassigned a vessel persistentId between save and load.
`SessionProvisional`	`bool`	`true` while the creating session still owns the RP. Normal split RPs created outside a re-fly session remain `true` with `CreatingSessionId == null` only until their owning tree is accepted; load-time sweep retains them during merge-dialog scene loads, and the tree-commit path promotes them to persistent. Session-scoped RPs flip `false` at `TagRpsForReap` during merge.
`Corrupted`	`bool`	`true` when quicksave validation failed (file missing at deferred save, all slots undetectable, PartLoader deep-parse failed). RP kept for diagnostic visibility; Rewind button disabled.
`CreatingSessionId`	`string`	Session GUID when the RP was created inside an active session; `null` otherwise.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC timestamp.
`FocusSlotIndex`	`int`	(v0.9.1) Slot index (0-based, into `ChildSlots`) of the vessel that was the active focus at split time. `-1` means "no focus signal available" — either a legacy RP that predates this field, OR a new RP where no slot was focused at split time (rare: e.g. background joint break, or a split where the player was focused on an unrelated vessel outside the split). Used by `IsUnfinishedFlight`'s `TerminalOutcomeQualifies` to gate stable-terminal qualification: focus-continuation slots are excluded from auto-UF for `Orbiting` / `SubOrbital`. `Crashed` and EVA-stranded terminals qualify regardless of focus. `FocusSlotIndex == -1` short-circuits Orbiting/SubOrbital to false (the conservative choice when no focus signal is available), preserving v0.9.0 Crashed-only behavior for legacy RPs and avoiding retroactive flooding from background-only splits. The Re-Fly merge auto-seal contract (§4.9) treats the merge-time slot as `focusSlotOverride`, which routes around this static field.

Node layout (emitted by `RewindPoint.SaveInto`, `RewindPoint.cs:75`):

```
POINT
{
    rewindPointId = rp_...
    branchPointId = bp_...
    ut = 1742538.43
    quicksaveFilename = Parsek/RewindPoints/rp_...sfs
    sessionProvisional = True|False
    corrupted = True             # omitted when False
    creatingSessionId = sess_... # omitted when empty
    createdRealTime = 2026-04-17T21:35:12Z
    focusSlotIndex = 2           # (v0.9.1) omitted when -1

    CHILD_SLOT { slotIndex = 0  originChildRecordingId = rec_... controllable = True }
    CHILD_SLOT { slotIndex = 1  originChildRecordingId = rec_... controllable = True }
```

```
    PID_SLOT_MAP      { pid = 12345678  slot = 0 }
    PID_SLOT_MAP      { pid = 87654321  slot = 1 }

    ROOT_PART_PID_MAP { pid = 11223344  slot = 0 }
    ROOT_PART_PID_MAP { pid = 99887766  slot = 1 }
}
```

5.6 ChildSlot

File: `Source/Parsek/ChildSlot.cs`.

Field	Type	Purpose
`SlotIndex`	`int`	Zero-based position within `RewindPoint.ChildSlots`.
`OriginChildRecordingId`	`string`	Immutable id of the recording originally created for this slot at split time.
`Controllable`	`bool`	Reserved for future classifier churn; currently always `true` (only controllable entities produce slots).
`Disabled`	`bool`	`true` if the Rewind button for this slot must be grayed out (PidSlotMap lookup failed for this slot at split time, or a loader sanity check failed).
`DisabledReason`	`string`	Human-readable reason shown in the UI tooltip. Current reasons: `no-live-vessel`.
`Sealed`	`bool`	(v0.9.1) `true` if the player invoked the per-row Seal action on this slot, OR the Re-Fly merge auto-seal gate (§4.9) flipped it. Closes the slot permanently — excluded from `IsUnfinishedFlight`; treated as equivalent-to-Immutable by `RewindPointReaper` for close-eligibility, BUT NotCommitted effective recordings always block reap regardless of this flag (defends against load-time race states). Default `false`; legacy saves load with `false` (existing crash UF rows continue to qualify). Player has no in-game un-seal path; tree-scoped Full-Revert is the only undo.
`SealedRealTime`	`string`	(v0.9.1) Wall-clock ISO-8601 UTC timestamp the Seal was applied. Diagnostic only; null when `Sealed` is false.
`Stashed`	`bool`	(v0.9.1) `true` when the player invoked the per-row Stash action on this slot, promoting a default-excluded stable terminal leaf into Unfinished Flights. Only spawnable stable terminals (`Landed`, `Splashed`, `Orbiting`, `SubOrbital`) qualify; `Recovered`, `Docked`, `Boarded`, and rows with retry-blocking `ScienceEarning` actions are rejected by the resolver. Default `false`; legacy saves load with `false`. Stash is only possible while the backing RewindPoint still exists; it does not resurrect already-reaped RP quicksaves.
`StashedRealTime`	`string`	(v0.9.1) Wall-clock ISO-8601 UTC timestamp the Stash was applied. Diagnostic only; null when `Stashed` is false.

Derived:

- `EffectiveRecordingId(IReadOnlyList<RecordingSupersedeRelation>)` delegates to `EffectiveState.EffectiveRecordingId` (`EffectiveState.cs:78`). Forward walk from

`OriginChildRecordingId`, cycle-guarded via visited set, returns the last-reached id when no further relation matches.

Node layout:

```
CHILD_SLOT
{
    slotIndex = 0
    originChildRecordingId = rec_...
    controllable = True
    disabled = True              # omitted when False
    disabledReason = no-live-vessel  # omitted when null
    sealed = True                # (v0.9.1) omitted when False
    sealedRealTime = 2026-...    # (v0.9.1) omitted when null
    stashed = True               # (v0.9.1) omitted when False
    stashedRealTime = 2026-...   # (v0.9.1) omitted when null
}
```

5.7 RecordingSupersedeRelation

File: `Source/Parsek/RecordingSupersedeRelation.cs`.

Field	Type	Purpose
`RelationId`	`string`	Stable id in the format `rsr_<Guid-N>`.
`OldRecordingId`	`string`	The superseded recording (stays in `RecordingStore`, hidden from ERS).
`NewRecordingId`	`string`	The superseding recording (the re-fly provisional promoted on merge).
`UT`	`double`	`Planetarium.UT` at which the merge occurred.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC.

Stored in `ParsekScenario.RecordingSupersedes`. Append-only during normal play. Removed by whole-tree discard (`TreeDiscardPurge.PurgeTree`, §6.17), and by load-time cleanup when a row is fully orphaned (both endpoint ids are missing from `RecordingStore`). One-sided orphan relations stay in place with a Warn log on every load (`LoadTimeSweep.cs`) because an old-present/new-missing row still suppresses the retired old recording; the forward walk in `EffectiveState.EffectiveRecordingId` handles an unresolved `NewRecordingId` as a chain terminator.

Node layout (emitted as `ENTRY` children of the `RECORDING_SUPERSEDES` parent node):

```
ENTRY
{
    relationId = rsr_...
    oldRecordingId = rec_...
    newRecordingId = rec_...
    ut = 1742800.00
    createdRealTime = 2026-04-17T22:10:00Z
}
```

5.8 LedgerTombstone

File: `Source/Parsek/GameActions/LedgerTombstone.cs`.

Field	Type	Purpose
`TombstoneId`	`string`	Stable id in the format `tomb_<Guid-N>`.
`ActionId`	`string`	The superseded action's immutable `GameAction.ActionId`. Aliases the design-spec's `SupersededActionId`; the code uses `ActionId` to match the field it refers to.
`RetiringRecordingId`	`string`	The new recording whose merge caused this tombstone to be written.
`UT`	`double`	`Planetarium.UT` at which the merge occurred.
`CreatedRealTime`	`string`	Wall-clock ISO-8601 UTC.

Stored in `ParsekScenario.LedgerTombstones`. Append-only. Multiple tombstones for the same `ActionId` are tolerated — the ELS filter is "at least one tombstone exists." Removed only by whole-tree discard when the underlying action's recording is in the discarded tree.

Node layout (emitted as `ENTRY` children of `LEDGER_TOMBSTONES`):

```
ENTRY
{
    tombstoneId = tomb_...
    actionId = act_...
    retiringRecordingId = rec_...
    ut = 1742800.00
    createdRealTime = 2026-04-17T22:10:00Z
}
```

5.9 ReFlySessionMarker

File: `Source/Parsek/ReFlySessionMarker.cs`.

Singleton marker present in `ParsekScenario` iff a re-fly session is live. The v0.9.0 release defined seven fields (six validated, one informational); v0.9.1 adds `SupersedeTargetId` (weakly validated) and `PreSessionBranchPointIds` (informational baseline for the structural-mutation gate).

Field	Validated?	Purpose
`SessionId`	yes	Unique GUID per invocation / retry.
`TreeId`	yes	RecordingTree this re-fly belongs to. Must exist in `RecordingStore.CommittedTrees`.
`ActiveReFlyRecordingId`	yes	The `NotCommitted` provisional re-fly recording. Must resolve in `CommittedRecordings` with `MergeState == NotCommitted`.
`OriginChildRecordingId`	yes	The slot's IMMUTABLE origin id. Must resolve in `CommittedRecordings`. Existing consumers (`RevertInterceptor.FindSlotForMarker`, in-place continuation, ghost suppression) key off this immutable origin.

`SupersedeTargetId`	weak (v0.9.1)	The slot's CURRENT EFFECTIVE recording id at invocation time — equals `slot.EffectiveRecordingId(supersedes)` (= prior tip on chain extension; = slot origin on first re-fly). Used by `SupersedeCommit.AppendRelations` as the root of the subtree closure walk so chain extension produces a linear graph instead of a star. Always written by post-feature invocations; legacy markers load with null and `AppendRelations` falls back to `OriginChildRecordingId`. `MarkerValidator.Validate`: when non-null, must resolve in `CommittedRecordings` or the matching pending tree; failure logs `[ReFlySession] Warn: Marker invalid field=SupersedeTargetId; clearing` and clears the field. Null is always valid. See §6.22.
`RewindPointId`	yes	The invoked RP's id. Must resolve in `ParsekScenario.RewindPoints`.
`InvokedUT`	yes	`Planetarium.UT` at invocation. Must not be strictly greater than the current UT.
`InvokedRealTime`	**no**	Wall-clock ISO-8601 UTC. Informational; purely for human-readable logs and diagnostics.
`PreSessionBranchPointIds`	**no** (v0.9.1)	Snapshot of every `BranchPoint.Id` present in the marker's tree at the moment the marker was created. Consumed by `SupersedeCommit.HasReFlySessionStructuralMutation` as a session-local baseline so post-RP structural BPs that the load-time `SpliceMissingCommittedRecordingsIntoLoadedTree` path re-grafts back into the loaded tree are excluded from the gate. Without it, any old structural BP from the original future would auto-seal a stashed slot the player never mutated. Populated by `RewindInvoker.SnapshotTreeBranchPointIds(treeId)` called inline in `AtomicMarkerWrite`. Legacy markers persisted before this field shipped load with `null`; the structural-mutation gate skips on those markers so in-flight sessions at upgrade time preserve the pre-fix keep-open behavior on merge.

Persisted as a single `REFLY_SESSION_MARKER` ConfigNode on `ParsekScenario`:

```
REFLY_SESSION_MARKER
{
    sessionId = sess_...
    treeId = tree_...
    activeReFlyRecordingId = rec_...
    originChildRecordingId = rec_...
    supersedeTargetId = rec_...                # (v0.9.1) always written post-feature
    rewindPointId = rp_...
    invokedUT = 1742800.00
    invokedRealTime = 2026-04-17T22:10:00Z
    preSessionBranchPointIdsPresent = true     # (v0.9.1) presence sentinel
    preSessionBranchPointId = bp_...           # (v0.9.1) repeated value entries
    preSessionBranchPointId = bp_...
}
```

The `preSessionBranchPointIdsPresent` sentinel distinguishes "post-fix marker on a tree that had no BPs at invocation" (sentinel set, list empty — gate runs and treats every current BP as session-authored) from "legacy marker from before this field shipped" (sentinel absent — gate conservatively skips). The reader populates the list only when the sentinel is set.

Cleared on: re-fly merge (after Durable Save #1), re-fly discard (return-to-Space-Center), retry (a fresh marker with a new `SessionId` replaces the old one), full-revert (tree discard clears it if scoped to the discarded tree), load-time validation failure (cleared + Warn-logged).

5.10 MergeJournal

File: `Source/Parsek/MergeJournal.cs`.

Singleton journal present on `ParsekScenario` only during a staged-commit merge.

Field	Type	Purpose
`JournalId`	`string`	Per-run id `mj_<Guid-N>`. Addition to the pre-impl design spec for log readability.
`SessionId`	`string`	Merge session GUID (matches the retiring marker's `SessionId`).
`Phase`	`string`	One of the `Phases` constants (below).
`StartedUT`	`double`	`Planetarium.UT` at merge start.
`StartedRealTime`	`string`	Wall-clock ISO-8601 UTC.

The `Phases` vocabulary (`MergeJournal.cs:53`):

Phase string	What has happened on disk	Recovery on load
`Begin`	Journal written. Nothing else.	Rollback.
`Supersede`	Supersede relations half-written or fully written in memory.	Rollback.
`Tombstone`	Tombstones half-written or fully written in memory.	Rollback.
`Finalize`	MergeState flipped + transient cleared in memory.	Rollback.
`Durable1Done`	First durable save flushed: supersedes + tombstones + MergeState on disk. Marker + RPs still live.	Complete from here (tag RPs → reap → clear marker → ...).
`RpReap`	Session-prov RPs tagged + reaped. Marker still live.	Complete (clear marker → ...).
`MarkerCleared`	Marker cleared in memory.	Complete (Durable Save #2).
`Durable2Done`	Second durable save flushed.	Complete (clear journal → Durable Save #3).
`Complete`	Journal set to `Complete`; Durable Save #3 interrupted before clear.	Idempotent clear.

`MergeJournal.IsPreDurablePhase(phase)` returns true for `Begin` / `Supersede` / `Tombstone` / `Finalize`; `IsPostDurablePhase` returns true for `Durable1Done` / `RpReap` / `MarkerCleared` / `Durable2Done` / `Complete`.

Persisted as a single `MERGE_JOURNAL` ConfigNode on `ParsekScenario`:

```
MERGE_JOURNAL
{
    journalId = mj_...
    sessionId = sess_...
    phase = Begin
    startedUT = 1742800.00
    startedRealTime = 2026-04-17T22:10:00Z
}
```

5.11 ParsekScenario additions

The scenario module owns five new persistent collections, each wrapped in its own parent node on save (`ParsekScenario.OnSave`; verified at `Source/Parsek/ParsekScenario.cs:716-825`):

```
PARSEK
{
    ...
    REWIND_POINTS        { POINT { ... } POINT { ... } ... }
    RECORDING_SUPERSEDES { ENTRY { ... } ENTRY { ... } ... }
    LEDGER_TOMBSTONES    { ENTRY { ... } ENTRY { ... } ... }
    REFLY_SESSION_MARKER { ... }  # present only during active session
    MERGE_JOURNAL        { ... }  # present only during staged-commit merge
    ...
}
```

All five sections are additive — a save written before v0.9 simply has none of them, and `OnLoad` treats the absence as empty lists / null singletons. Saves written by v0.9 and loaded on v0.8.x would fail cleanly because the pre-v0.9 scenario does not know these keys; no downgrade path is provided.

Runtime-only state counters on the scenario drive the ERS / ELS cache invalidation:

Counter	Bumped by
`SupersedeStateVersion`	Every mutation to `RecordingSupersedes`, marker identity change, MergeState flip.
`TombstoneStateVersion`	Every mutation to `LedgerTombstones`.
`RecordingStore.StateVersion`	Every mutation to `CommittedRecordings`.
`Ledger.StateVersion`	Every mutation to `Actions`.

5.12 Directory layout

```
saves/<save>/
    persistent.sfs                          (KSP standard)
    Parsek/
        Recordings/
            <recordingId>.prec              (existing, unchanged)
            <recordingId>_vessel.craft      (existing, unchanged)
            <recordingId>_ghost.craft       (existing, unchanged)
            <recordingId>.pcrf              (existing, unchanged)
```

```
        RewindPoints/
            <rewindPointId>.sfs                (NEW; KSP-format quicksave)
    Parsek_Rewind_<sessionId>.sfs              (NEW; transient, root-level)
```

The transient `Parsek_Rewind_<sessionId>.sfs` at save-root exists only between `RewindInvoker.StartInvoke` (which copies `Parsek/RewindPoints/<rpId>.sfs` to this location) and the post-load cleanup in `RewindInvoker.ConsumePostLoad` (`RewindInvoker.cs:453`). The copy is required because `GamePersistence.LoadGame(fileName, folder, ...)` does not accept subdirectory paths. The cleanup runs in a `finally` block regardless of success, so a crash mid-invocation leaves at most one stale copy that the next invocation overwrites.

5.13 Unfinished Flights virtual UI group

File: `Source/Parsek/UI/UnfinishedFlightsGroup.cs` .

Not stored in `GroupHierarchyStore` . Membership derived per frame from ERS filtered by `EffectiveState.IsUnfinishedFlight` . As a system group it **cannot be hidden** and **cannot be a drop target** for manual group assignment. Rename on an individual member row still persists through the standard `Recording.VesselName` path; hide on a member row warns and refuses via `ParsekLog` → `ScreenMessages` advisory.

The virtual group is not the only safe affordance. The same recording can also appear in the normal recordings table, where its tree root still carries a legacy launch rewind save. RP-backed unfinished-flight rows therefore preempt the normal `RecordingStore.CanRewind` / `InitiateRewind` path and resolve the row's child slot before any legacy fallback is considered.

---

6. Behavior

6.1 Effective Recording Set (ERS)

File: `Source/Parsek/EffectiveState.cs:236` .

```
ERS = { r in RecordingStore.CommittedRecordings
      : r.MergeState != NotCommitted
        AND IsVisible(r, ParsekScenario.RecordingSupersedes)
        AND (activeMarker == null OR r.RecordingId not in
SessionSuppressedSubtree(activeMarker)) }

IsVisible(r, supersedes) :=
    walk forward from r.RecordingId via supersedes,
    return true iff the walk terminates at r.RecordingId itself.
```

Cached. Rebuild triggered by any change to `RecordingStore.StateVersion` , `ParsekScenario.SupersedeStateVersion` , or the active `ReFlySessionMarker` identity. Every rebuild emits a single `[ERS]` Verbose line:

```
[Parsek][Verbose][ERS] Rebuilt: <N> entries from <raw> committed
(skippedNotCommitted=<k> skippedSuperseded=<s> skippedSuppressed=<u>
marker=<sessionId|none>)
```

6.2 Effective Ledger Set (ELS)

File: `Source/Parsek/EffectiveState.cs:310`.

```
ELS = { a in Ledger.Actions : a.ActionId not in { t.ActionId for t in
ParsekScenario.LedgerTombstones } }
```

One filter only: tombstones. There is no recording-level filter. A superseded recording's non-eligible ledger actions (contracts, milestones, facilities, strategies, tech, science, funds, non-kerbal-death rep) remain in ELS. The only way an action exits ELS is via an explicit `LedgerTombstone` carrying its `ActionId`.

Cached. Rebuild triggered by any change to `Ledger.StateVersion` or `ParsekScenario.TombstoneStateVersion`. Every rebuild emits a single `[ELS]` Verbose line with the count and skipped-tombstoned counter.

6.3 Session-suppressed subtree (physical-visibility only)

Computed by `EffectiveState.ComputeSessionSuppressedSubtree(marker)`. Forward-only closure from `marker.OriginChildRecordingId`:

```
closure(r) = { r }
  for each child c whose ParentRecordingIds contain r:
    if c.ParentRecordingIds is a subset of the current closure:
      include c (and recurse)
    else:
      stop — c has a parent outside the subtree (mixed-parent halt)
```

Consumed by physical-visibility subsystems:

- **Ghost playback engine** (`GhostPlaybackEngine.UpdatePlayback`) — per-frame filter with a `sessionSuppressed=<n>` counter in the frame summary log; destroys leftover ghost visuals on session entry.
- **Ghost chain walker** (`GhostChainWalker`) — suppressed recordings do not claim chain tips. Aggregated skip log.
- **Ghost map presence** (`GhostMapPresence`) — create-entry-points gate + per-tick prune of suppressed presence.
- **Watch mode** (`WatchModeController`) — refuses entry on suppressed anchor; exits on session-start if the current anchor is suppressed.

Subsystems that read ERS / ELS directly and do NOT apply the session-suppressed filter:

- Ledger recalculation (`LedgerOrchestrator`) reads ELS directly via `EffectiveState.ComputeELS()`.
- Career-state UI (Contracts / Strategies / Facilities / Milestones tabs).
- Crew reservation manager (`CrewReservationManager`), with a narrow live-re-fly-crew carve-out (see §6.4).
- Contract / milestone / facility state derivation (KSP-owned; Parsek never touches on supersede).
- Timeline view.
- Unfinished Flights membership.
- RewindPoint reap logic.

Exactly one `[ReFlySession] Start` / `End` log line is emitted per transition via `SessionSuppressionState` (`Source/Parsek/SessionSuppressionState.cs`), which observes marker-identity changes and logs once per transition.

6.4 Kerbal dual-residence carve-out (during active session)

`CrewReservationManager.IsLiveReFlyCrew(pcm, marker)` exempts kerbals physically embodied in the live re-fly vessel from reservation lock. Without this carve-out, a kerbal whose ledger `KerbalAssignment`-to-Dead event is still in ELS (tombstone not yet written — that happens at merge) would be reservation-locked as dead, silently blocking EVA or crew transfer during the re-fly.

Other reservation effects (kerbals reserved to unrelated still-ghost recordings, stand-in generation for other unreserved slots) remain active.

6.5 Multi-controllable split detection

Every joint-break, undock, and EVA event funnels through `SegmentBoundaryLogic.ClassifyJointBreakResult` and its siblings. `SegmentBoundaryLogic.IdentifyControllableChildren` (`SegmentBoundaryLogic.cs:235`) enumerates the post-split vessels and counts the ones that have a `ModuleCommand` part or are EVA kerbals. `SegmentBoundaryLogic.IsMultiControllableSplit(count)` is a one-line gate: `count >= 2` .

When the gate passes, `ParsekFlight.CreateSplitBranch` / `ProcessBreakupEvent` / the EVA handler invokes `RewindPointAuthor.Begin` with the new `BranchPoint` , the list of `ChildSlot` s (one per controllable child), and the list of controllable child PIDs. For BG-recorded splits that originate outside the focus vessel, `BackgroundRecorder` takes the same path with a cached PID payload.

6.6 Rewind Point capture

File: `Source/Parsek/RewindPointAuthor.cs` .

`Begin` runs synchronously (same frame as the split):

1. **Scene guard**: if `HighLogic.LoadedScene != GameScenes.FLIGHT` , abort with `[RewindSave] Warn: Aborted: scene=<loaded>` . Return null.
2. **Scenario guard**: verify `ParsekScenario.Instance` is live (using `ReferenceEquals` to bypass Unity's overloaded `== null` for test fixtures).
3. Generate `rpId = "rp_" + Guid.NewGuid().ToString("N")` and validate via `RecordingPaths.ValidateRecordingId` .
4. Build the RP stub with empty `PidSlotMap` / `RootPartPidMap` , `SessionProvisional = true` , `CreatingSessionId` set from the active marker if one is live.
5. Append the RP to `ParsekScenario.RewindPoints` and stamp `BranchPoint.RewindPointId = rpId` . This is the synchronous wiring — an `OnSave` triggered between `Begin` and the deferred coroutine will already see the in-progress RP.
6. Emit `[Rewind] Info: RewindPoint begin: rp=<id> bp=<bpId> slots=<N> controllablePids=<M> ut=<x>` .
7. Start the deferred coroutine (or, in tests, the `SyncRunForTesting` hook).

`RunDeferred` / `ExecuteDeferredBody` (one-frame-deferred):

1. Re-check scene guard; if we left FLIGHT between `Begin` and the deferred frame, `RollbackBegin` removes the RP from the scenario and clears the BranchPoint back-reference.
2. Drop high warp if `CurrentRateIndex > 1` . Wrapped in a `finally` so a mid-save throw still restores the player's rate.
3. Populate `PidSlotMap` and `RootPartPidMap` by iterating ChildSlots. For each slot, resolve the expected `Vessel.persistentId` via the injected `RecordingResolver` (or `CommittedRecordings` fallback), look up the live vessel via `FlightGlobalsProvider.TryGetVesselSnapshot` , and record `{pid → slotIdx}` and `{rootPid → slotIdx}` . A slot with no live vessel is marked `Disabled = true` with `DisabledReason = "no-live-vessel"` .
4. Log `[Rewind] Info: RewindPoint slot capture: rp=<id> populated=<p>/<total> disabled=<d>` .

5. If every slot failed, mark the RP `Corrupted = true` but continue with the save so the on-disk quicksave is still available for diagnostics.
6. Call `GamePersistence.SaveGame(tempName, saveFolder, SaveMode.OVERWRITE)` to `Parsek_TempRP_<rpId>.sfs` in the save root. KSP always writes to the save root per its own convention.
7. `FileIOUtils.SafeMove` the temp file to `<saveDir>/Parsek/RewindPoints/<rpId>.sfs` (atomic tmp + rename). `RecordingPaths.EnsureRewindPointsDirectory` creates the directory on demand.
8. Log `[RewindSave] Info: Wrote rp=<id> path=<relPath> bytes=<N> ms=<t>`. On any failure in steps 6-7, call `RollbackBegin` so a partial-write never leaves a half-registered RP.
9. Restore warp rate in the `finally`; log `[RewindSave] Info: Warp restored to <rate> for rp=<id>` or `[RewindSave] Warn: Warp restore to <rate> failed for rp=<id>: <reason>`.

Failure modes: disk full, permissions, save exception, scene transition between `Begin` and deferred frame. All paths either roll back cleanly or keep a Corrupted RP for diagnostics; the split itself proceeds normally either way.

6.7 Between split and merge

The recorder keeps recording normally: the focused vessel is actively sampled, unfocused controllable siblings are BG-recorded. All have `MergeState = NotCommitted`.

On session merge (the NON-re-fly merge that commits the tree that just ran):

- Focused vessel's recording → `Immutable`.
- BG-recorded controllable children whose parent BranchPoint has `RewindPointId != null` → `CommittedProvisional`. (They might become Unfinished Flights.)
- Other BG children under a BranchPoint without an RP → `Immutable`.
- Session-provisional RPs promote to persistent during `TagRpsForReap` when their `CreatingSessionId` matches the merging session — but for tree merges outside an active re-fly session, `CreatingSessionId` is null on the RPs written during flight, so the session promote path doesn't apply. `LoadTimeSweep` retains those normal staging RPs during the scene load that presents the merge dialog; `RecordingStore.CommitTree` promotes the accepted tree's normal staging RPs to persistent. Nested/session-created RPs with a dead `CreatingSessionId` are the ones purged. Unfinished Flights membership recomputes.

Note: the distinction between "session merge of a normal flight" and "session merge of a re-fly session" is driven by whether `ActiveReFlySessionMarker` is non-null. Only re-fly merges go through `MergeJournalOrchestrator.RunMerge`.

6.8 Invoking a Rewind Point

File: `Source/Parsek/RewindInvoker.cs`.

Precondition gate (`CanInvoke`, `RewindInvoker.cs:63`). Five checks, each emitting a reason string on failure:

1. Scene is `FLIGHT` / `SPACECENTER` / `TRACKSTATION` (not a transition).
2. `RewindInvokeContext.Pending` is false (no in-flight invocation).
3. RP is not `Corrupted`.
4. Quicksave file exists on disk at the expected path (`ResolveAbsoluteQuicksavePath`).
5. No active re-fly session (`ParsekScenario.Instance.ActiveReFlySessionMarker == null`).
6. Deep-parse precondition passes: the quicksave is loaded as a ConfigNode and every `PART` node's `name` must resolve via `PartLoader.getPartInfoByName`. Cached per RP for 60 seconds via `PreconditionCache`. Missing parts mark the RP `Corrupted`.

Confirmation dialog (`ShowDialog`, `RewindInvoker.cs:138`). A `PopupDialog.SpawnPopupDialog` with a MultiOptionDialog. The message names the UT, the selected slot / origin recording, and the narrow supersede

semantics:

```
Rewind to rewind point <rpId> at UT <utText>?
Spawning the selected child (slot <N>, origin=<rec>) live; merged siblings will play as
ghosts.

Career state during this attempt stays as it is now. Supersede on merge
retires only kerbal-death events; contract / milestone / facility / strategy
/ tech / science / funds state is unchanged.
```

Buttons: `Rewind` (fires `StartInvoke`), `Cancel` (logs and dismisses).

Pre-load phase (`StartInvoke` , `RewindInvoker.cs:206`). All synchronous, no yield:

1. Generate `sessionId = "sess_" + Guid.NewGuid().ToString("N")` .
2. `ReconciliationBundle.Capture()` snapshots every in-memory state that must survive the KSP scene reload: committed recordings, committed trees, ledger actions, ledger tombstones, scenario lists (RPs, supersedes, marker, journal), crew reservations, group hierarchy, milestone store, ResourcesApplied flags. Any exception aborts with `[Rewind] Error` and a user toast.
3. `CopyQuicksaveToSaveRoot` copies `saves/<save>/Parsek/RewindPoints/<rpId>.sfs` to `saves/<save>/Parsek_Rewind_<sessionId>.sfs` (root). Required because `GamePersistence.LoadGame` does not support subdirectory paths.
4. Store the session id, bundle, RP, selected slot, temp path, and slot metadata in the static `RewindInvokeContext` . This is the only state that survives the scene reload.
5. `HighLogic.CurrentGame = GamePersistence.LoadGame(tempLoadName, HighLogic.SaveFolder, true, false)` + `HighLogic.LoadScene(GameScenes.FLIGHT)` . From here, Unity tears down the current scenario; only the static context survives.

Any failure between steps 2 and 5 triggers `TryRestoreBundle` + `HandleQuicksaveMissing` (clears filename + marks Corrupted if the file really is missing) + toast + context clear.

Post-load phase (`ConsumePostLoad` , `RewindInvoker.cs:335`). Called by `ParsekScenario.OnLoad` exactly once per invocation. All synchronous:

1. `ReconciliationBundle.Restore(bundle)` re-applies the pre-load in-memory state over the quicksave-loaded state. The quicksave's `RecordingStore` , `Ledger.Actions` , scenario lists, crew reservations, etc. are discarded; the bundle's are restored. `Planetarium.UT` remains at the quicksave's UT (that's the whole point).
2. `PostLoadStripper.Strip(rp, selectedSlotIndex)` strips the non-selected siblings (§6.9).
3. `FlightGlobals.SetActiveVessel(selectedVessel)` .
4. `AtomicMarkerWrite(rp, selected, stripResult, sessionId)` — the critical section (§6.10).
5. `LedgerOrchestrator.RecalculateAndPatch()` — non-fatal; a throw logs Warn but does not fail the invocation.
6. `finally` : delete the root-level temp quicksave and clear `RewindInvokeContext` .

6.9 Post-load strip (authoritative matching)

File: `Source/Parsek/PostLoadStripper.cs` .

For each live `Vessel` in the enumeration:

1. **Ghost ProtoVessel guard**: if `GhostMapPresence.IsGhostMapVessel(pid)` , skip. Do not strip — it is a Parsek-spawned map presence. Increment `GhostsGuarded` .

2. **Primary match** via `RewindPoint.PidSlotMap`: if `pid` is in the map, record the match with the slot index.
3. **Fallback match** via `RewindPoint.RootPartPidMap` using the vessel's root-part `Part.persistentId`: if primary miss but rootPart is present and the map has a matching entry, record the match and increment `FallbackMatches`; log `[Rewind] Warn: Fallback match via root-part v=<pid> rootPart=<rootPid> slotIdx=<idx>`.
4. **No match**: `LeftAlone` ++. Log `[Rewind] Verbose: Strip leaveAlone: unrelated v=<pid> name=<name>`.

After enumeration: walk the matches. The vessel matching the selected slot is kept as `SelectedVessel`; all other matches are stripped via `Vessel.Die()`. If multiple vessels match the selected slot (shouldn't happen in practice), the first is kept and the rest are stripped with a `[Rewind] Warn: Multiple vessels match selectedSlot=<n>`.

Final log: `[Rewind] Info: Strip stripped=[<indices>] selected=<idx> ghostsGuarded=<N> leftAlone=<M> fallbackMatches=<f>`.

If the stripper returns `SelectedPid == 0u` (selected vessel not present on reload), `ConsumePostLoad` toasts an error and aborts — the quicksave is stale relative to the RP's expectations.

6.10 Atomic provisional + marker write

File: `RewindInvoker.cs:463` (`AtomicMarkerWrite`).

The critical section. Runs synchronously; throws MUST leave global state untouched. NO yield, NO await, NO deferred save.

1. `CheckpointHookForTesting("CheckpointA:BeforeProvisional")`.
2. Build the provisional Recording (`BuildProvisionalRecording`, `RewindInvoker.cs:527`): fresh `RecordingId = "rec_" + Guid.NewGuid().ToString("N")`, `MergeState = NotCommitted`, `CreatingSessionId = sessionId`, `SupersedeTargetId = selected.OriginChildRecordingId`, `ProvisionalForRpId = rp.RewindPointId`, `ParentBranchPointId` cloned from the origin child (or the RP's BranchPoint as fallback), `TreeId` cloned from the origin child, `VesselPersistentId = stripResult.SelectedPid`, `VesselName = stripResult.SelectedVessel.vesselName`.
3. `RecordingStore.AddProvisional(provisional)`.
4. `CheckpointHookForTesting("CheckpointA:AfterProvisional")`.
5. Build the `ReFlySessionMarker` from the provisional, selected slot, and RP.
6. `CheckpointHookForTesting("CheckpointB:BeforeMarker")`.
7. `scenario.ActiveReFlySessionMarker = marker; scenario.BumpSupersedeStateVersion();`
8. `CheckpointHookForTesting("CheckpointB:AfterMarker")`.
9. Log `[ReFlySession] Info: Started sess=<sid> rp=<id> slot=<idx> provisional=<rid> origin=<oid> tree=<tid>`.

A `catch` around steps 5–8 rolls back both: removes the provisional via `RecordingStore.RemoveCommittedInternal(provisional)` and sets `ActiveReFlySessionMarker = null`. Both are idempotent. Rollback is the reason `try/catch` wraps those steps rather than wrapping the whole method — a mid-provisional throw doesn't leave a half-marker behind either.

KSP's `ScenarioModule.OnSave` cannot preempt this synchronous block. The `CheckpointHookForTesting` is the only observer that can see between checkpoints — tests use it to verify no save boundary is crossed.

6.11 Within a re-fly session

Ghost playback engine, chain walker, map presence, watch mode enumerate `ERS \ SessionSuppressedSubtree(marker)`. Career subsystems read ERS / ELS directly.

Nested multi-controllable splits during the session create additional session-provisional RPs with `CreatingSessionId = activeSession.SessionId`. Child recordings born in those splits carry the same `CreatingSessionId`. On session merge, all promote; on discard, all purge (load-time sweep spare-set logic, §6.16).

6.12 Merging a re-fly session (staged commit, journaled)

File: `Source/Parsek/MergeJournalOrchestrator.cs`.

Triggered by `MergeDialog.MergeCommit` when `ActiveReFlySessionMarker` is non-null. The 14-step design-spec merge is consolidated into nine phase transitions:

1. **Journal write, phase = `Begin`** (`MergeJournalOrchestrator.cs:189`). `scenario.ActiveMergeJournal` assigned. Log `[MergeJournal] Info: sess=<sid> phase=Begin`.
2. **Supersede relations** (`AdvancePhase → Supersede`). `SupersedeCommit.AppendRelations` computes the forward-only merge-guarded subtree closure via `EffectiveState.ComputeSessionSuppressedSubtree(marker)`, appends one `RecordingSupersedeRelation { old=subtreeId, new=provisional.Id, UT, CreatedRealTime }` per subtree id. Idempotent — a pre-existing relation with the same old/new pair is skipped with a Verbose log. Log `[Supersede] Info: Added <N> supersede relations for subtree rooted at <originId>`.
3. **Tombstones** (`AdvancePhase → Tombstone`). `SupersedeCommit.CommitTombstones` walks `Ledger.Actions`, groups in-scope actions by RecordingId for bounded rep-pairing, emits `LedgerTombstone`s for every action passing `TombstoneEligibility.IsEligible`. See §6.13. `TombstoneStateVersion` bumped. `CrewReservationManager.RecomputeAfterTombstones` re-derives reservations; death-tombstoned kerbals return to active. Log `[LedgerSwap] Info: Tombstoned <N> actions (KerbalDeath=<d>, repBundled=<r>); <M> actions matched subtree but type-ineligible (breakdown: contract=<c> milestone=<m> facility=<f> strategy=<s> tech=<t> science=<sc> funds=<fn> rep-unbundled=<ru>)` + `[Supersede] Info: Narrow v1: physical playback replaced; career state unchanged except kerbal deaths`.
4. **Finalize** (`AdvancePhase → Finalize`). `SupersedeCommit.FlipMergeStateAndClearTransient` flips `provisional.MergeState` to `Immutable` (Landed / InFlight) or `CommittedProvisional` (Crashed), clears `SupersedeTargetId`, bumps `SupersedeStateVersion`. Marker preserved (`preserveMarker: true`) so it stays live across Durable Save #1.
5. **Durable Save #1** (`AdvancePhase → Durable1Done`). Deferred to the next natural `ScenarioModule.OnSave` in production; tests inject a synchronous hook. This is the durability barrier — on-disk state is now supersedes + tombstones + MergeState all committed; marker + RPs still live.
6. **RP reap** (`AdvancePhase → RpReap`). `TagRpsForReap(marker, scenario)` flips `SessionProvisional = false` on every RP with matching `CreatingSessionId`, and defensively promotes the marker's origin RP when it is a normal staging RP with `CreatingSessionId == null`. Then `RewindPointReaper.ReapOrphanedRPs` walks all RPs; for each non-session-provisional RP, it walks every ChildSlot's `EffectiveRecordingId` and marks the RP reap-eligible if every effective recording has `MergeState == Immutable`. Eligible RPs get their quicksave file deleted (`File.Delete` is best-effort), scenario entry removed, BranchPoint back-reference cleared. Log `[Rewind] Info: ReapOrphanedRPs: reaped=<R> remaining=<rem>`.
7. **Marker clear** (`AdvancePhase → MarkerCleared`). `scenario.ActiveReFlySessionMarker = null`; `BumpSupersedeStateVersion`. Log `[ReFlySession] Info: End reason=merged sess=<sid> provisional=<rid>`.
8. **Durable Save #2** (`AdvancePhase → Durable2Done`).

9. **Clear journal** (`ClearJournalAndFinalSave`). `journal.Phase = Complete` ; Durable Save #3; `ActiveMergeJournal = null` . Log `[MergeJournal] Info: Completed sess=<sid>` .

Failure recovery matrix (`RunFinisher` , `MergeJournalOrchestrator.cs:263`). On load, if `ActiveMergeJournal != null` :

Phase on disk	Recovery
`Begin`, `Supersede`, `Tombstone`, `Finalize` (pre-Durable-1)	**Rollback.** The first durable save never happened; disk still holds the pre-merge snapshot. `RollBack` removes the in-memory session-provisional recording (idempotent when already absent), clears marker, clears journal, Durable Save. `[MergeJournal] Info: Rolled back from phase=<p>: session restored sess=<sid> markerCleared=<b> provisionalRemoved=<r>.`
`Durable1Done`, `RpReap`, `MarkerCleared`, `Durable2Done`, `Complete` (post-Durable-1)	**Complete.** `CompleteFromPostDurable` drives the remaining steps. Idempotent: missing file deletions are no-ops, already-cleared marker is a no-op. `[MergeJournal] Info: Completed from phase=<p> sess=<sid> stepsDriven=<n>.`

The finisher is triggered by journal presence, **not** by marker state. This is the v0.5 pre-impl correction that survived into v1.

6.13 v1 tombstone-eligible scope

File: `Source/Parsek/GameActions/TombstoneEligibility.cs` + `TombstoneAttributionHelper.cs` .

An action `a` is in supersede scope when:

- `a.RecordingId` is non-null AND `a.RecordingId` is in the forward-only merge-guarded subtree closure; OR
- (design-spec allowance for null-scoped attribution; v1 does NOT tombstone null-scoped actions per §7.41).

For actions in scope, eligibility is narrow:

`GameAction.Type`	Eligible?	Condition
`KerbalAssignment` (with `Dead = true`)	yes	Direct eligibility.
`ReputationPenalty`	yes	Paired with a same-recording kerbal-death action within a 1s UT window.
`ContractAccept` / `Complete` / `Fail` / `Cancel`	**no**	Sticky (KSP-owned).
`Milestone`	**no**	Sticky (KSP-owned first-time flag).
`FacilityUpgrade` / `Destruction` / `Repair`	**no**	Sticky.
`StrategyActivate` / `Deactivate`	**no**	Sticky.
`TechResearch` / `PartPurchase`	**no**	Sticky.
`FundsSpending` / `ScienceSpending`	**no**	Already-spent.
`ReputationEarning`, other rep	**no**	Not bundled with a death.

Null-scoped actions (action's `RecordingId == null`) are NEVER tombstoned, regardless of type (§7.41).

Eligibility is checked with an idempotence guard: an action that already carries a tombstone (any existing `LedgerTombstone.ActionId == a.ActionId`) is skipped. This makes the whole merge re-runnable if the finisher drives through the Tombstone phase on load.

6.14 Revert-during-re-fly dialog

Files: `Source/Parsek/RevertInterceptor.cs` + `Source/Parsek/ReFlyRevertDialog.cs`.

`RevertInterceptor` is a pair of Harmony prefixes — one on `FlightDriver.RevertToLaunch` (Esc > Revert to Launch and the flight-results dialog's Launch revert button) and one on `FlightDriver.RevertToPrelaunch` (Esc > Revert to VAB / SPH and the flight-results dialog's VAB/SPH revert button). Both prefixes delegate to the shared dispatcher `RevertInterceptor.Prefix(RevertTarget target, EditorFacility facility)` where `RevertTarget ∈ {Launch, Prelaunch}` and `facility` is captured from the stock `RevertToPrelaunch(EditorFacility)` argument (ignored for the Launch path). When `ParsekScenario.Instance?.ActiveReFlySessionMarker == null`, the prefix returns `true` and the stock revert runs. When non-null, it returns `false` (blocking stock revert) and `ReFlyRevertDialog.Show(marker, target, onRetry, onFullRevert, onCancel)` spawns a `PopupDialog` with three buttons:

- **Retry from Rewind Point** — `RetryHandler`: clear marker, generate fresh `Guid`, re-invoke `RewindInvoker.StartInvoke(rp, slot)` with the same RP + slot captured from the marker. The old provisional becomes a zombie for the next load-time sweep to clean up. RP-anchored and scene-agnostic: Retry lands the player back in FLIGHT at the split moment regardless of whether they clicked Revert-to-Launch or Revert-to-VAB/SPH. Log `[ReFlySession] Info: End reason=retry sess=<old> rp=<rp> slot=<i> target=<Launch|Prelaunch>`.
- **Discard Re-fly** — `DiscardReFlyHandler`: session-scoped cleanup. Removes the provisional re-fly recording from `CommittedRecordings` (by-id lookup in the raw committed list, then `RecordingStore.RemoveCommittedInternal`; ERS-exempt because NotCommitted is invisible to ERS), promotes the origin RP (the RP named by `marker.RewindPointId`) by flipping `SessionProvisional=false` and clearing `CreatingSessionId` so the post-load `LoadTimeSweep` treats it as persistent (the sweep's RP-discard pass skips every `SessionProvisional=false` RP and every normal staging RP with `CreatingSessionId == null`), clears `ActiveReFlySessionMarker` and `ActiveMergeJournal`, bumps `SupersedeStateVersion` (not `TombstoneStateVersion` — no tombstone rows are touched), defensively deletes the prior session temp quicksave `saves/<save>/Parsek_Rewind_<sessionId>.sfs` if still present, then stages the RP quicksave through the same save-root copy path `RewindInvoker` uses and calls `GamePersistence.LoadGame(tempLoadName, SaveFolder, true, false)` to restore the RP's UT / career / vessels. On success, for the Prelaunch context pre-sets `EditorDriver.StartupBehaviour = EditorDriver.StartupBehaviours.START_CLEAN` (NOT `LOAD_FROM_CACHE` — the cached `ShipConstruct` is stale after `LoadGame` swaps `HighLogic.CurrentGame`) and `EditorDriver.editorFacility = facility`; then `HighLogic.LoadScene` dispatches to `GameScenes.SPACECENTER` for Launch or `GameScenes.EDITOR` for Prelaunch. Stock `FlightDriver.RevertToLaunch` / `RevertToPrelaunch` are NOT used — decompilation (via `ilspycmd` on `Assembly-CSharp.dll`) shows they restore the current flight's cached `PostInitState` / `PreLaunchState`, which are tied to the current flight's launch moment, not the RP's UT. `TreeDiscardPurge.PurgeTree` is NOT called: the tree's other Rewind Points, supersede relations, and tombstones are preserved. The origin split's crash recording plus its promoted RP still resolve, so `EffectiveState.IsUnfinishedFlight` stays true for `Immutable` legacy crashes and `CommittedProvisional` current crashes, together with the RP and a crashed terminal, and the Unfinished Flights row stays visible. Log `[ReFlySession] Info: End reason=discardReFly sess=<sid> target=`

`<Launch|Prelaunch> facility=<VAB|SPH|-->` (the `facility=--` sentinel is logged in the Launch variant for grep-friendliness; the handler prepends `dispatched=false` and suppresses the scene call when the RP quicksave cannot be staged so the player is left in flight with a clear toast).
- **Continue Flying** — `CancelHandler` : pure logging; no state changes. Log `[ReFlySession] Info: Revert dialog cancelled sess=<sid> target=<Launch|Prelaunch>` .

Dialog body copy branches on `target` : the Launch variant says "returns you to the Space Center" as the Discard Re-fly destination; the Prelaunch variant says "returns you to the VAB or SPH" and explicitly clarifies that Retry still returns the player to FLIGHT at the split moment regardless of which Revert button they clicked. Title ("Revert during re-fly") and button labels are shared. When `ActiveMergeJournal` is non-null the Discard Re-fly button is omitted entirely (primary UX signal to the player that discard mid-merge is not allowed); the handler also refuses defensively with a `"merge in progress — retry in a moment"` toast + Warn log if called anyway. A third body hook (`ReFlyRevertDialog.BodyHookForTesting`) plus a buttons hook (`ReFlyRevertDialog.ButtonsHookForTesting`) let unit tests assert the variant + the journal gate without spawning a live popup.

The dialog takes an input lock (`DialogLockId = "ParsekReFlyRevertDialog"`) while visible so the player cannot interact with the flight scene during the decision.

`GameEvents.OnRevertToLaunchFlightState` / `OnRevertToPrelaunchFlightState` both fire from inside the respective stock body (after the state roll-back, before the scene unload). Because the interceptor's prefix returns `false` to block the body, neither event fires during an active session — `RevertDetector.PendingKind` stays `None` and `ParsekScenario.OnLoad` 's revert-aware branch is correctly skipped. `DiscardReFlyHandler` never re-calls the stock revert methods (it drives `LoadGame` + `LoadScene` directly), so `OnRevertToLaunchFlightState` stays un-fired across the whole revert flow; any post-load `RevertDetector` state has already been consumed (or is irrelevant) by the time the new scene's scenario loads.

6.15 Session end without merge

- **Return to Space Center** (without clicking Merge): `ParsekScenario.OnDestroy` / the merge-dialog-discard path clears the marker + discards the provisional + purges session-provisional RPs.
- **Quit without returning to Space Center**: marker stays on disk. Load-time validation (§6.16) decides what to do with it.

6.16 Load-time sweep

File: `Source/Parsek/LoadTimeSweep.cs` .

Runs from `ParsekScenario.OnLoad` AFTER `MergeJournalOrchestrator.RunFinisher` and `RewindInvoker.ConsumePostLoad` , and BEFORE `RewindPointReaper.ReapOrphanedRPs` . Single-pass gather-then-delete (eligibility decisions are made against a stable snapshot before any mutation):

1. **Marker validation** (`MarkerValidator.Validate` , `MarkerValidator.cs:86`). Six durable fields checked in order: `SessionId` non-empty; `TreeId` present in `RecordingStore.CommittedTrees` ; `ActiveReFlyRecordingId` resolves in `CommittedRecordings` with `MergeState == NotCommitted` ; `OriginChildRecordingId` resolves in `CommittedRecordings` ; `RewindPointId` resolves in `ParsekScenario.RewindPoints` ; `InvokedUT` is not strictly greater than current UT. The seventh field (`InvokedRealTime`) is informational only and does not fail a load. On any failure, `[ReFlySession] Warn: Marker invalid field=<f>; clearing` + marker cleared before the sweep continues (nested cleanup below depends on the cleared marker's session id).
2. **Spare set.** If marker valid: `ActiveReFlyRecordingId` and `RewindPointId` are in the spare set; additionally, any Recording with `CreatingSessionId == marker.SessionId` and any RP with `CreatingSessionId == marker.SessionId` join the spare set.

3. **Discard set.** Any Recording with `MergeState == NotCommitted` and not in the spare set is a zombie. Any RP with `SessionProvisional == true`, non-empty `CreatingSessionId`, and not in the spare set is a dead session-provisional RP. RPs with `CreatingSessionId == null` are normal staging RPs and are retained.
4. **Orphan supersede relations.** Walk `RecordingSupersedes`; log Warn for each relation whose `OldRecordingId` or `NewRecordingId` doesn't resolve. NOT removed — invariant 7.
5. **Orphan tombstones.** Walk `LedgerTombstones`; log Warn for each `ActionId` that doesn't resolve in `Ledger.Actions`. NOT removed.
6. **Stray `SupersedeTargetId`.** Walk committed recordings; for each `Immutable` or `CommittedProvisional` recording with non-empty `SupersedeTargetId`, log `[Recording] Warn: Stray SupersedeTargetId on committed rec=<id>; treating as cleared` and clear the field.
7. **Nested session-prov cleanup** (§7.11). When step 1 cleared the marker, count the discard-set entries that carried the cleared marker's session id and emit `[ReFlySession] Info: Nested session-prov cleanup sess=<sid> recordings=<n> rps=<m>`. (The discard set already caught them — this is an observational log for the §7.11 contract.)
8. **Delete.** Remove discard-set recordings from `RecordingStore`. Remove discard-set RPs from `ParsekScenario.RewindPoints` (and delete their quicksave files best-effort; a lock / IO failure Warns but still clears the scenario entry).
9. **Summary log.** Single `[LoadSweep] Info: Marker valid=<b>; spare=<n> discarded=<r> orphanSupersedes=<os> orphanTombstones=<ot> strayFields=<s> discardedRps=<rps>`.
10. **Cache invalidation.** `BumpSupersedeStateVersion` + `BumpTombstoneStateVersion` + `EffectiveState.ResetCachesForTesting` so ERS / ELS rebuild on next access.

6.17 Tree-scoped rewind-state purge

File: `Source/Parsek/TreeDiscardPurge.cs`.

Triggered by merge-discard (`RecordingStore.DiscardPendingTree`) or by any other path that discards a whole tree. Note: the Revert-during-re-fly dialog's Discard Re-fly button does NOT reach this path — design §6.14's `DiscardReFlyHandler` is session-scoped and leaves the tree's supersede / tombstone / RP state untouched. `TreeDiscardPurge.PurgeTree(treeId)` removes every Rewind-to-Separation artifact tied to the tree:

- Remove every RP whose `BranchPointId` is in the tree; delete each RP's quicksave file on disk.
- Remove every `RecordingSupersedeRelation` with either endpoint in the tree.
- Remove every `LedgerTombstone` whose target `GameAction.RecordingId` is in the tree.
- Clear `ActiveReFlySessionMarker` if it references the tree.
- Clear `ActiveMergeJournal` if it references the tree.
- Recompute crew reservations via `CrewReservationManager.RecomputeAfterTombstones` when tombstones shrank.

What this does NOT do: it does not remove the tree's committed recordings themselves and does not delete the tree from `RecordingStore.CommittedTrees`. Removal of the recordings (for a pending tree being discarded before commit) is the caller's responsibility — `TreeDiscardPurge.PurgeTree` is invoked by `RecordingStore.DiscardPendingTree` BEFORE the recordings are removed from `CommittedRecordings` so the purge can still resolve recording ids to tree membership. The Revert-during-re-fly dialog's Discard Re-fly button is explicitly NOT a caller of this purge — it only clears the session artifacts and reloads the origin RP's quicksave; the tree's other RPs / supersede relations / tombstones are preserved (design §6.14).

Purging a tree's rewind-state is the **only** path that deletes supersede relations or ledger tombstones (invariant 7). Orphans left by partial crash states stay in place and are logged on every load.

6.18 Disk-usage diagnostic

File: `Source/Parsek/RewindPointDiskUsage.cs`.

Surfaced in Settings → Diagnostics as `Rewind point disk usage: <size> (<N> files)`. Backed by a 10-second snapshot cache — the directory is walked once per cache miss, so the Settings window doesn't thrash the filesystem on every repaint. The v0.9.1 follow-up adds split buckets next to the byte/file total: live crashed-open RPs, live stable-open RPs, and sealed-pending RPs. Buckets are explanatory and may overlap when a single RP contains both sealed and still-open slots; this is monitoring only, not an auto-purge trigger.

6.19 The Unfinished Flights predicate (v0.9.1)

`EffectiveState.IsUnfinishedFlight(Recording rec)` is rewritten in v0.9.1 to use the shared classifier. The full evaluation:

```
IsUnfinishedFlight(rec) :=
    // Filter 1: visible, committed recording
    rec is in ERS                                                  // §1
    AND rec.MergeState in { Immutable, CommittedProvisional }       // §2

    // Filter 2: controllable subject at chain HEAD
    AND chainHead.IsDebris == false                                 // §3

    // Filter 3: matching RP with an open slot, with per-RP-context leaf check
    AND exists RP, exists slot in RP.ChildSlots such that:
        // Slot resolution -- v0.9.0 logic, unchanged shape
        slot.EffectiveRecordingId(supersedes) == rec.RecordingId
        AND (rec.ParentBranchPointId == RP.BranchPointId
             OR rec.ChildBranchPointId == RP.BranchPointId)
                                                                    // §4
        // Slot-close gate
        AND slot.Sealed == false                                    // §5

        // Per-RP leaf gate
        AND let chainTip = ResolveChainTerminalRecording(rec)
            (chainTip.ChildBranchPointId == null
             OR chainTip.ChildBranchPointId == RP.BranchPointId)    // §6

        // Outcome gate
        AND TerminalOutcomeQualifies(chainTip, slot, RP)            // §7

TerminalOutcomeQualifies(chainTip, slot, RP) :=
    let kerbal   = !string.IsNullOrEmpty(chainTip.EvaCrewName)
    let terminal = chainTip.TerminalStateValue
    let isFocus  = (slot.SlotIndex == RP.FocusSlotIndex)
    let noFocusSignal = (RP.FocusSlotIndex == -1)

    if !terminal.HasValue:
        return false                          // no terminal recorded

    // Crashed always qualifies, regardless of kerbal/focus/everything.
    // This branch runs FIRST so a dead EVA kerbal (EvaCrewName != null
    // AND terminal == Destroyed) routes here -- not through the kerbal
    // branch below -- so the reason logging says reason=crashed and the
```

```
    // dead-EVA-kerbal narrative matches the code.
    if terminal.Value == Destroyed:
        return true                          // Crashed

    if kerbal:
        // At this point terminal is guaranteed not Destroyed (handled above).
        // Any non-Boarded stable terminal is a stranded EVA: surface
        // (Landed/Splashed), drift (Orbiting/SubOrbital), or Docked
        // edge cases. Returns true. Boarded would mean the kerbal was
        // reboarded -- but the BoardBP makes the recording non-leaf via
        // the structural gate before we ever reach here, so this branch's
        // "!= Boarded" is effectively redundant. Listed for completeness.
        //
        // EVA branch returns BEFORE the noFocusSignal short-circuit:
        // stranded kerbals surface even from legacy / no-focus-signal RPs.
        // Intentional retroactive carve-out (see §9.5).
        return terminal.Value != Boarded

    // Stable in-flight terminals: only non-focus slots on RPs with a
    // defined focus signal qualify. noFocusSignal short-circuits to false.
    if noFocusSignal:
        return false

    if terminal.Value == Orbiting   && !isFocus: return true
    if terminal.Value == SubOrbital && !isFocus: return true
        // Vacuum-arc SubOrbital. Atmospheric SubOrbital is reclassified
        // to Destroyed by BallisticExtrapolator before commit (see
        // BallisticExtrapolator.cs SubSurfaceStart short-circuit +
        // IncompleteBallisticSceneExitFinalizer terminal stamping).

    // Landed / Splashed / Recovered / Docked: stable surface or recovered
    // terminal. The universe gave the vessel a stable conclusion. Default
    // does not include the row regardless of focus.
    return false
```

The implementation lives in a new `UnfinishedFlightClassifier` static class (§6.20). The signature matches the v0.9.0 `IsUnfinishedFlight(Recording rec)` for backwards-compatible callers, but the implementation routes through `UnfinishedFlightClassifier.Qualifies(rec, slot, rp, considerSealed: true)` after the slot+RP resolution.

6.20 Classifier extraction (v0.9.1)

A new file `Source/Parsek/UnfinishedFlightClassifier.cs` (or alternatively all-in-one in `EffectiveState.cs`) owns:

- `Qualifies(Recording rec, ChildSlot slot, RewindPoint rp, bool considerSealed)` — the predicate body.
- `TerminalOutcomeQualifies(Recording chainTip, ChildSlot slot, RewindPoint rp)` — the outcome gate.

The following helpers move from `UI/RecordingsTableUI.cs` into this file (or `EffectiveState.cs`):

- `TryResolveRewindPointForRecording(Recording rec, out RewindPoint rp, out int slotListIndex)`
- `IsUnfinishedFlightCandidateShape(Recording rec)`
- `IsVisibleUnfinishedFlight(Recording rec, out string reason)`

`RecordingsTableUI` becomes a consumer of the moved helpers. Other consumers (Site A and Site B-1 predicate-call paths in §6.21, plus the Seal handler in §6.24) call them from non-UI namespaces. Site B-2 reads Site B-1's verdict downstream and does not call the predicate directly; it lives in `MergeDialog` and consumes the marker + the just-classified provisional state.

6.21 MergeState promotion: Site A display predicate, Site B merge-close policy (v0.9.1)

Site A decides whether a newly committed original-tree recording appears in Unfinished Flights. Site B decides what happens after the player re-flies one of those slots and commits the attempt back to the timeline. The policies share the same predicate but Site B also applies merge-time conclusion and safety gates: chosen stable leaves auto-seal, terminal failures can remain retryable, while recovered/docked/boarded terminals, downstream structural/world interactions, and retry-blocking recording-linked actions close the slot.

Site A: original tree commit. `RecordingStore.ApplyRewindProvisionalMergeStates` (`RecordingStore.cs:715-770`) extends from the v0.9.0 Crashed-only check to the broader predicate:

```
for each rec in tree.Recordings:
    if rec.MergeState != Immutable: continue
    if rec.chainHead.IsDebris: continue

    if NOT TryResolveRewindPointForRecording(rec, out rp, out slotIdx):
        continue
    var slot = rp.ChildSlots[slotIdx]

    // considerSealed=false: a freshly committed slot is never Sealed yet.
    if NOT UnfinishedFlightClassifier.Qualifies(
            rec, slot, rp, considerSealed: false):
        continue

    rec.MergeState = CommittedProvisional
    log [UnfinishedFlights] Info: CommitTree promoted rec=<rid>
        slot=<slotIdx> rp=<rpId> reason=<crashed|stableLeafUnconcluded|strandedEva>
```

The existing v0.9.0 crash-only path is subsumed: `Qualifies` returns true for `terminal == Destroyed` regardless of focus.

Site B-1: re-fly merge classifier flip. `SupersedeCommit.FlipMergeStateAndClearTransient` resolves the RP slot, evaluates the UF predicate for diagnostics with the merge-time slot supplied as `focusSlotOverride` (§4.9), then applies the merge-close gate:

```
// Slot resolution: walk supersedes from each slot's OriginChildRecordingId
// forward; the helper returns the slot whose forward trail contains the
// queried provisional. After Phase 2 (AppendRelations) of the merge journal,
// the new {priorTip -> provisional} relation is in supersedes (per §6.22),
// so the walk reaches the provisional cleanly.
if NOT TryResolveRewindPointForRecording(provisional, out rp, out slotIdx):
    // Hard failure in DEBUG (Debug.Assert + log Error); fall back to the
    // v0.9.0 default in RELEASE for crash safety. A release-build occurrence
```

```
    // indicates a §6.22 prerequisite regression or AppendRelations bug --
    // not a recoverable runtime state.
    log [Supersede] Error: Site B-1 slot lookup failed for provisional=<rid>
        rpId=<marker.RewindPointId> originChildRec=<marker.OriginChildRecordingId>
        supersedeTargetId=<marker.SupersedeTargetId>
    Debug.Assert(false, "Site B-1 slot lookup failed")
    provisional.MergeState = (Classify(provisional) == Crashed)
                               ? CommittedProvisional : Immutable
    return

var slot = rp.ChildSlots[slotIdx]
// focusSlotOverride: the slot the player chose to Re-Fly is the
// merge-time focus regardless of rp.FocusSlotIndex. Inside the
// classifier, slotListIndex == focusSlotOverride takes the
// stableTerminalFocusSlot branch on Orbiting/SubOrbital tips, after
// the existing static-focus and retry-blocking recording-action checks
// have run.
bool qualifies = UnfinishedFlightClassifier.Qualifies(
    provisional, slot, rp, considerSealed: false,
    focusSlotOverride: slotIdx)

// Terminal failure: Destroyed or non-boarded EVA. Crashed retries
// keep their existing keep-open path so the player can retry.
bool terminalFailure =
    terminal == Destroyed
    OR (EvaCrewName != null AND terminal != Boarded)

bool hasRetryBlockingPlayerAction =
    HasRetryBlockingRecordingAction(provisional)
    // credited ScienceEarning (Crew Report / EVA Report / Surface Sample /
    // Transmit / Recover) — the only ledger row that auto-seals via this
    // gate. Automatic consequence rows, KSC-scene player decisions, and
    // tombstoneable kerbal-death rows do not close the retry path.

// Structural mutation during this Re-Fly session: any BranchPoint with
// Type in {Breakup, Undock, EVA, JointBreak} authored at UT > rp.UT
// AND not already in marker.PreSessionBranchPointIds (the snapshot
// captured at marker creation, persisted on the marker, used to filter
// out spliced-back pre-session BPs). See §4.9.
bool hasStructuralMutation =
    HasReFlySessionStructuralMutation(provisional, marker)

bool keepOpen =
    qualifies
    AND NOT hasRetryBlockingPlayerAction
    AND terminalFailure                  // ONLY terminal failure keeps open
    AND NOT hasStructuralMutation        // ... and no structural mutation

provisional.MergeState = keepOpen
    ? CommittedProvisional
    : Immutable
```

```
// Auto-seal triggers: stableTerminalFocusSlot (player-chosen slot
// reached stable terminal), structuralMutation (session authored a
// new structural BP), recordingAction (retry-blocking action),
// downstreamBp (chain tip's child BP resolves to a deeper RP), and the
// existing hard-safety terminals (Recovered/Docked/Boarded).
if (!keepOpen AND closeReason is an auto-seal reason)
    slot.Sealed = true

log [Supersede] Info: provisional=<rid> mergeState=<state> qualifies=<b>
    slot=<slotIdx> rp=<rpId> focusSlot=<rp.FocusSlotIndex> focusSlotOverride=<slotIdx>
    classifierReason=<reason> autoSeal=<b> autoSealReason=<reason>
```

Under the v0.9.1 auto-seal revision (§4.9), a `stableLeafUnconcluded` reason can no longer come out of Site B-1 because `focusSlotOverride: slotIdx` reroutes the player-chosen-slot Orbiting/SubOrbital path to `stableTerminalFocusSlot` and the `!keepOpen` branch. `stableLeafUnconcluded` still appears at Site A and from non-Re-Fly callers (UI display, reap eligibility), where the override is `null`. Stable retry-keep-open at merge time is therefore deliberately narrowed to terminal-failure outcomes (Destroyed, non-boarded EVA) without a retry-blocking action. If the player wants the stable Re-Fly to remain re-flyable, they unstash via the manual STASH affordance from a separate slot — or they don't merge yet (Discard/Retry the Re-Fly).

Site B-2: in-place continuation merge handling. `MergeDialog.TryCommitReFlySupersede` consumes Site B-1's result. Under the v0.9.1 revision, clean stable in-place outcomes commit `Immutable` + auto-seal at Site B-1, so Site B-2 lets the reaper remove the RP. Safety-closed outcomes follow the same path. Terminal-failure outcomes (`Destroyed`, non-boarded EVA) without a retry-blocking action still stay `CommittedProvisional` and remain retryable from the same RP.

6.22 Closure-helper split + invocation linearization (v0.9.1 prerequisite)

The v0.9.0 invocation stamped `marker.OriginChildRecordingId` and `provisional.SupersedeTargetId` from `selected.OriginChildRecordingId` (the slot's immutable origin). `SupersedeCommit.AppendRelations` then wrote `{slot.OriginChildRecordingId -> provisional}` for every re-fly. Multiple re-flies into the same slot produced a star-shaped graph rooted at the slot's origin. The walker `EffectiveState.EffectiveRecordingId` scans supersedes from the beginning and stops at the first match — on a star, it resolves to the oldest re-fly, missing later ones. Site B-1's slot lookup against the provisional then failed.

Linear semantics + closure-helper split. Marker-write change in `RewindInvoker.AtomicMarkerWrite`:

```
// Compute prior tip ONCE before the in-place vs fresh-provisional branch.
string priorTip = selected.EffectiveRecordingId(scenario.RecordingSupersedes)

// Stamp BOTH marker fields unconditionally in the shared marker-creation
// block (runs on both branches).
marker.OriginChildRecordingId = selected.OriginChildRecordingId   // unchanged
marker.SupersedeTargetId      = priorTip                          // NEW (v0.9.1)

// Guard the provisional overwrite to the fresh-provisional branch only.
// In-place branch sets provisional == null.
if (provisional != null)
    provisional.SupersedeTargetId = priorTip                      // overwrite the
                                                                  // BuildProvisionalRecording
                                                                  // value
```

Closure-helper split in `EffectiveState`:

```
// New: takes an explicit root, parameterized over the existing closure
// algorithm (PID-peer expansion via marker.InvokedUT, mixed-parent halt,
// chain-sibling expansion, all preserved). Cache key includes rootOverride.
internal static IReadOnlyCollection<string> ComputeSubtreeClosureInternal(
    ReFlySessionMarker marker, string rootOverride)
{
    // ... existing closure body, with rootOverride substituted for
    // marker.OriginChildRecordingId at the seed point.
}

// Existing public helper: thin wrapper preserving null-guard +
// cached-null-guard + defensive-copy contracts.
public static IReadOnlyCollection<string> ComputeSessionSuppressedSubtree(
    ReFlySessionMarker marker)
{
    if (marker == null)
        return Array.Empty<string>();
    var cached = ComputeSubtreeClosureInternal(
                    marker, marker.OriginChildRecordingId);
    if (cached == null)
        return Array.Empty<string>();
    return new HashSet<string>(cached, StringComparer.Ordinal);
}
```

`SupersedeCommit.AppendRelations` switches to call:

```
ComputeSubtreeClosureInternal(marker,
    marker.SupersedeTargetId ?? marker.OriginChildRecordingId)
```

`CommitTombstones` continues to receive the closure unchanged; tombstone scope correctly tracks the chain-extension's actual descendants.

Runtime ghost suppression (ghost playback engine, chain walker, ghost map presence, watch mode) continues calling `ComputeSessionSuppressedSubtree(marker)` and gets the same return value as today.

Migration concern. Existing saves with star-shaped supersede graphs from prior v0.9.0 Crashed chain extensions are tolerated as-is. The walker continues to pick the oldest re-fly in the star portion, then walks linearly from there. New chains extend linearly from whichever leaf the walker reached. The hybrid case (legacy star + new linear extension) is covered by §7.72.

6.23 Reaper rule (v0.9.1 extension)

`RewindPointReaper.IsReapEligible` extends one term:

```
SlotIsClosed(slot, effectiveRecording) :=
    effectiveRecording.MergeState != NotCommitted        // unconditional no-reap
    AND
    (effectiveRecording.MergeState == Immutable          // existing v0.9.0 close signal
     OR slot.Sealed == true)                             // new v0.9.1 close signal
```

```
RP is reap-eligible iff every ChildSlot satisfies SlotIsClosed.
```

NotCommitted is unconditional no-reap regardless of `slot.Sealed`. This defends against load-time race states where the journal finisher rolled MergeState back to NotCommitted while the slot's Sealed bit is still on disk; reaping in that state would delete the RP quicksave while an active re-fly is still live.

Logging on reap:

```
[Rewind] Info: ReapOrphanedRPs: reaped=<R> remaining=<rem>
    sealedSlotsContributing=<S>
```

The new `sealedSlotsContributing` counter logs how many of the closed slots reached closure via the Seal path vs the Immutable path. Useful for understanding player behavior during playtest.

6.24 The Seal handler (v0.9.1)

The Seal action lives on each Unfinished Flight row in the Recordings Manager. Visual layout per §6.25.

Handler (in a new `UnfinishedFlightSealHandler` static class or as a method on the classifier):

```
1. Spawn Seal confirmation dialog (PopupDialog.SpawnPopupDialog with
   MultiOptionDialog body, see §6.25.2). Take input lock
   ParsekUFSealDialog.

2. On Cancel: log [UnfinishedFlights] Info: Seal cancelled rec=<rid>;
   release input lock; dismiss.

3. On Accept:
   - Locate slot via TryResolveRewindPointForRecording(rec, out rp, out slotIdx).
     If lookup fails: log [UnfinishedFlights] Error: Seal could not resolve
     slot for rec=<rid>; release input lock; dismiss; show toast
     "Seal failed -- slot not found." This should not happen for a row
     that was rendered as UF.
   - var slot = rp.ChildSlots[slotIdx]
   - slot.Sealed = true
   - slot.SealedRealTime = DateTime.UtcNow.ToString("o")
   - Bump SupersedeStateVersion (so ERS / UF group cache invalidates).
   - Determine reaperImpact: walk all slots of rp, check SlotIsClosed for
     each; "willReap" if all closed, "stillBlocked" otherwise.
   - log [UnfinishedFlights] Info: Sealed slot=<slotIdx> rec=<rid>
       bp=<bpId> rp=<rpId> terminal=<state> reaperImpact=<willReap|stillBlocked>
   - Release input lock; dismiss; row drops from group on next frame
     because the predicate now sees slot.Sealed == true.
```

Seal does NOT touch `Recording.MergeState`. Decoupling is load-bearing — see §2.2.

6.25 UI changes for Seal / Stash (v0.9.1)

6.25.1 Layout

The Recordings table has separate **Rewind** and **Re-Fly** columns. Rewind/Forward remain in the Rewind column so launch and timeline navigation stay visible. Unfinished-Flight actions live in the Re-Fly column: existing UF rows show `Fly` + `Seal`, and stable rows that are eligible for the manual escape hatch show `Stash` + `Seal`.

The separate Re-Fly column is chosen over widening/overloading the Rewind cell, a kebab menu, or right-click because:

- Rewind and Re-Fly are different workflows; keeping them in separate columns prevents stable-row Stash from hiding launch Rewind.
- UF rows need visible `Fly`, `Stash`, and `Seal` affordances. A menu would make the safety/cleanup path too easy to miss.
- KSP's stock UI has no strong right-click table-row precedent, and the mod already uses explicit row buttons for primary actions.

Cascade: every row in the Recordings Manager table gets the Rewind and Re-Fly columns. Non-UF rows leave the Re-Fly cell empty unless the row can be manually Stashed. Tooltip refresh on the UF group header.

6.25.2 Seal confirmation dialog

`PopupDialog.SpawnPopupDialog` with a `MultiOptionDialog`. Title: `Confirm Seal Unfinished Flight`. Dialog id and input-lock id: `ParsekUFSealDialog` (`UnfinishedFlightSealHandler.DialogName` / `DialogLockId`). Body, as emitted by `UnfinishedFlightSealHandler` (`UnfinishedFlightSealHandler.cs:160`):

```
Seal "<vessel-name>" (<terminal-state> at UT <ut>)?

This cannot be undone. After sealing, this entry is permanently merged
to the timeline in its current state.

If you might want to re-fly this later, click Cancel.
```

Buttons: `Seal Permanently` (destructive style, fires the seal handler), `Cancel`.

6.26 Unfinished Flights group tooltip refresh (v0.9.1)

`UI/UnfinishedFlightsGroup.cs` tooltip changes from the v0.9.0 wording to:

> *Vessels and kerbals that ended up in a state where you might want to re-fly them — crashed, abandoned in orbit, stranded on a surface. Click Fly to take control at the separation moment; click Seal to close the slot permanently if you're done with it.*

---

7. Edge Cases

Each entry: scenario / expected behavior / shipped status. Status codes: **Shipped (test)** = dedicated unit / integration test; **Shipped (integration)** = covered by multi-test wiring or in-game test; **Deferred (v1 limitation)** = explicitly not in v1, documented; **N/A** = design evolved out. Test names name `*.cs` test classes in `Source/Parsek.Tests/`.

7.1 Single-controllable split

No RP. Debris path unchanged. **Shipped (test)**: `MultiControllableClassifierTests`.

7.2 Three or more controllable children at one split

One RP with N ChildSlots + N entries in each PID map. Player iterates through the slot rewind buttons. **Shipped (test)**: `RewindPointRoundTripTests` + `RewindPointAuthorTests`.

7.3 Scene-transition during deferred save

§6.6 step 1 of the deferred coroutine re-checks the scene guard; if we left FLIGHT, `RollbackBegin` removes the RP and clears the BranchPoint back-reference. Log `[RewindSave] Warn: Aborted mid-coroutine: scene=<loaded>`. **Shipped (test)**: `RewindPointAuthorTests`.

7.4 Partial PidSlotMap failure at split

One slot's lookup fails at split time; RP still created with partial maps. The affected slot is marked `Disabled = true` with reason `no-live-vessel`; other slots usable. Log `[Rewind] Warn: Slot <i> disabled: no live vessel`. **Shipped (test)**: `RewindPointAuthorTests`.

7.5 Invoking RP while a re-fly is active

UI + CanInvoke gate rejects with `"Another re-fly session is already active"`. No nested sessions in v1. **Shipped (test)**: `RewindTests` (precondition matrix).

7.6 Re-fly target with BG-recorded descendants

SessionSuppressedSubtree includes descendants (forward-only closure). Merge step 2 adds supersede relations for each descendant. **Shipped (test)**: `SessionSuppressionWiringTests` + `SupersedeCommitTests`.

7.7 Cross-tree dock during re-fly

Station `S` from a different tree's Immutable recording is real at its chain tip. The re-fly can physically dock. Dock event is added to the re-fly's recording. No cross-tree merge; S's tree is not modified. **Acceptable v1 limitation**: no dedicated in-game test for cross-tree dock; covered by `InvokeRPStripAndActivateTest` behavior when a docked station is present.

7.8 Non-Parsek stock vessel interaction

Standard — the stripper leaves unrelated vessels alone (§6.9 step 4). **Shipped (test)**: `PostLoadStripperTests`.

7.9 Visual divergence for cross-recording stock vessel

Ghost-B recorded a dock with S at UT=X; the live re-fly also docks with S at UT=X−100 and moves S. Ghost-B's playback tries to dock with S-where-ghost-B-remembers-it. Accepted v1 visual limitation.

7.10 EVA duplicate on load

Strip detects a kerbal present in EVA form AND in a source vessel's crew. Canonical live location is the selected child's crew manifest; dedup via KSP crew removal. **Shipped (integration)**: KSP's own reload logic handles the dedup; the stripper's ghost-ProtoVessel guard + left-alone path cover the remaining cases.

7.11 Nested session-provisional cleanup on parent discard

Nested RP's `CreatingSessionId` matches the discarded session; load-sweep spare-set logic removes it. **Shipped (test)**: `LoadTimeSweepTests`.

7.12 F5 mid-re-fly + quit + load

Marker validates; all session-tagged recordings AND RPs spared; re-fly resumes. Atomic §6.10 write means no save can capture an intermediate state. **Shipped (integration)**: in-game `F5MidReFlyResumeTest`.

7.13 Contract supersede is a no-op on career state

BG-crash completed contract X. Player re-flies, merges. ContractComplete action is NOT tombstoned (not v1-eligible). Contract remains complete in `ContractSystem`. Rep bonus stays. **Shipped (integration)**: `TombstoneEligibilityTests` + in-game `ContractStickyAcrossSupersedeTest`.

7.14 Contract failed by BG-crash; re-fly succeeds

BG-crash failed contract X. v1 does NOT un-fail. Contract stays failed. **Deferred (v1 limitation)**: re-fly is a visual replay, not a contract rescue.

7.15 Milestone earned by superseded recording

First-time flag is KSP-owned and sticky. v1 never un-sets. **Deferred (v1 limitation)**.

7.16 Kerbal death + recovery

`KerbalAssignment`-to-Dead + bundled `ReputationPenalty` tombstoned at merge. `CrewReservationManager.RecomputeAfterTombstones` re-derives; kerbal returns to active. **Shipped (integration)**: `CrewReservationRecomputeTests` + in-game `KerbalRecoveryOnSupersedeTest`.

7.17 Re-fly crashes; player merges

Re-fly commits as `CommittedProvisional` with `TerminalKind = Crashed` per `TerminalKindClassifier`. Supersede relation appends (`A → A'`). A' satisfies `IsUnfinishedFlight`. Player CAN invoke RP again to try once more. **Shipped (integration)**: in-game `MergeCrashedReFlyCreatesCPSupersedeTest`.

7.18 Re-fly reaches stable outcome; player discards at dialog

Provisional discarded; marker cleared; original BG-crash remains Unfinished; RP durable. **Shipped (integration)**: the discard path drops the provisional via `RecordingStore.RemoveCommittedInternal` + `ActiveReFlySessionMarker = null`.

7.19 Simultaneous multi-controllable splits

Two independent RPs; two quicksave writes serial through KSP. **Shipped (integration)**: `RewindPointAuthorTests` (deferred-one-frame ensures serialization through KSP's save pipeline).

7.20 F9 during re-fly

Standard quickload + auto-resume. Marker validates against the loaded state; if consistent, the session continues. **Shipped (integration)**.

7.21 Warp during re-fly

Standard Parsek warp. Not a new code path.

7.22 Rewind click during scene transition

CanInvoke returns `"Scene transition in progress — please wait"`. **Shipped (test)**: `RewindTests`.

7.23 Two children crash at same location

Independent rewinds via independent RPs.

7.24 Re-fly vessel with no engines

Allowed. The rewind only restores a quicksave state.

7.25 Drag Unfinished Flight into manual group

`GroupPickerUI.CanAddToUserGroup` rejects with `[UnfinishedFlights]` Verbose log + ScreenMessages toast. **Shipped (test)**: `UnfinishedFlightsDragRejectTests`.

7.26 Mod-modified joint-break / destruction

Classifier is `ModuleCommand`-based; mods that add alternative controller-like parts may not be seen. **Deferred (v1 limitation)** outside stock parts.

7.27 High physics warp at split

Forced to 0 during save (`RewindPointAuthor.ExecuteDeferredBody` step 2). **Shipped (integration)**: in-game `WarpZeroedDuringSaveTest`.

7.28 Disk usage diagnostics

Settings → Diagnostics shows total RP disk usage with a 10-second cache. **Shipped (test)**: `DiskUsageDiagnosticsTests`.

7.29 Mod part missing on re-fly load

Deep-parse precondition (`RewindInvoker.CanInvoke` step 6) walks the quicksave's PART nodes through `PartLoader.getPartInfoByName`. Any missing part marks the RP `Corrupted` and disables the Rewind button. **Shipped (test)**: `RewindTests` precondition matrix.

7.30 Cannot hide Unfinished Flights group

`GroupHierarchyStore.CanHide` denies. UI hide checkbox is not drawn for the virtual group. **Shipped (test)**: `UnfinishedFlightsGroupCannotHideTests`.

7.31 Non-crashed BG sibling — focus vs non-focus (v0.9.1 contract)

Terminal kind classifies as `Landed` / `Orbiting` / `SubOrbital` / `InFlight` via `TerminalKindClassifier`. v0.9.1's broadened predicate splits this case in two:

- **Focus-continuation slot** (`slot.SlotIndex == RP.FocusSlotIndex`) reaching `Landed` / `Splashed` / `Orbiting` / `SubOrbital`: `IsUnfinishedFlight` returns **false**. The recording does NOT enter the Unfinished Flights group and NO Re-Fly button is drawn — this slot is the player's continued mission, and broadening to it would pollute the list with every routine upper-stage-to-orbit. The recording plays back from its committed trajectory and the end-of-recording spawn puts the real vessel where it actually ended up. Manual Stash (§6.25.1) is the explicit escape hatch when the player decides post-hoc that they want such a slot re-flyable.
- **Non-focus slot** reaching `Orbiting` / `SubOrbital`: `IsUnfinishedFlight` returns **true** under the v0.9.1 stable-leaf contract (the 4-probe deploy case in §4.4, the upper-stage-left-coasting case in §4.5). Site A promotes the recording to `CommittedProvisional` and the row appears in Unfinished Flights with `Fly` + `Seal` affordances. `Landed` / `Splashed` / `Recovered` / `Docked` / `Boarded` non-focus terminals still return false (stable conclusion, no re-fly opportunity by default); manual Stash is available for `Landed` / `Splashed`.
- **Stranded EVA kerbal** (`EvaCrewName != null`, terminal != `Boarded`): qualifies regardless of focus or `FocusSlotIndex` (the kerbal branch runs before the focus short-circuit). See §4.6.
- **Legacy / no-focus-signal RPs** (`FocusSlotIndex == -1`): conservatively short-circuit Orbiting / SubOrbital to false (the focus signal is missing), preserving v0.9.0 Crashed-only behavior on pre-feature

saves. Stranded kerbals still surface (intentional retroactive carve-out, §9.5).

Corollary: if a sibling that **should** have been destroyed shows up with a non-crashed terminal (e.g. a booster left behind on reentry but misclassified `Orbiting`), the bug is in the terminal-state classifier, not the UF predicate. The scene-exit finalizer (`IncompleteBallisticSceneExitFinalizer` / `BallisticExtrapolator`) is the usual suspect; its `[Extrapolator]` log tag traces the decisions. A `Start: body=… alt=-<large>` preceded by `PatchedConicSnapshot: solver unavailable` (NullSolver) means KSP has already invalidated the vessel, and the extrapolator now short-circuits to `TerminalState.Destroyed` via `ExtrapolationFailureReason.SubSurfaceStart` rather than silently horizon-capping to Orbiting. Fixing the classifier upstream restores the Crashed Unfinished Flights entry automatically — do not work around it by further broadening the UI predicate.

7.32 Classifier drift across Parsek versions

Old RPs retained; no reclassify. `[Rewind] Verbose` notes the mismatch.

7.33 Rename / hide on Unfinished Flight row

Rename persists through the generic `Recording.VesselName` path. Hide warns via `ScreenMessages` + refuses. **Shipped (test)**: `RenameOnUnfinishedFlightTests`.

7.34 Corrupted quicksave on invocation

RP marked Corrupted via `HandleQuicksaveMissing` at CanInvoke time (file missing) or via `PartLoaderPrecondition.MarkCorrupted` (parse failure / missing parts). Rewind button disabled with the precondition reason. **Shipped (test)**: `RewindTests`.

7.35 Auto-save concurrent with RP save

One-frame defer + root-save-then-move + KSP's serial save pipeline. Auto-save is blocked during the synchronous `GamePersistence.SaveGame` inside the deferred body. **Shipped (integration)**.

7.36 Rewind while unrelated new mission in-flight

CanInvoke gate rejects if a session is already active; if the player is flying a new unrelated mission outside any session, they must first Revert-to-Launch / return-to-Space-Center that mission. **Shipped (integration)**: `RewindTests`.

7.37 Pre-existing vessel reservation conflict during re-fly

`CrewReservationManager.IsLiveReFlyCrew` carve-out exempts the live re-fly crew from reservation lock for the session duration. **Shipped (test)**: `SessionSuppressionWiringTests` + in-game `KerbalDualResidenceCarveOutTest`.

7.38 Strip encounters ghost ProtoVessel

`PostLoadStripper.Strip` step 1 ghost guard via `GhostMapPresence.IsGhostMapVessel`. **Shipped (test)**: `PostLoadStripperTests`.

7.39 Strip encounters unrelated committed real vessel

Step 4 left-alone path. Log `[Rewind] Verbose: Strip leaveAlone: unrelated v=<pid>`. **Shipped (test)**: `PostLoadStripperTests`.

7.40 SessionSuppressedSubtree with mixed-parent descendant

Closure halts at the mixed-parent node. Descendant stays in ERS. **Shipped (test)**: `EffectiveStateTests` (mixed-parent halt case).

7.41 Null-scoped action in supersede scope

v1 tombstones only when the action's `RecordingId` is in the supersede subtree. Null-scoped actions (deadline `ContractFail`, stand-in generation, etc.) are NEVER tombstoned. They stay in ELS. **Shipped (test)**: `TombstoneEligibilityTests`.

7.42 KSC action during active re-fly

Player doesn't typically leave flight during re-fly. If a KSC action fires (mod or edge case), it's added to the ledger with null `RecordingId`; not in supersede scope; stays ELS. **Shipped (integration)**.

7.43 Chain extends through multiple merged crashes

Each `CommittedProvisional Crashed` merge extends the supersede chain. `EffectiveRecordingId` walks forward through all relations. Unfinished Flights moves along with the chain's tip. **Shipped (test)**: `ChildSlotEffectiveRecordingIdTests` exercises the forward walk through 0/1/2/3-length chains; the chain-extension flow over multiple merges is covered by the in-game `MergeCrashedReFlyCreatesCPSupersedeTest` matrix.

7.44 Vessel-destruction rep penalty

Original BG-crash had a vessel-destruction rep penalty (not kerbal-death; just "vessel blew up"). v1 does NOT tombstone general `ReputationPenalty` actions — only the kerbal-death-bundled rep retires. Vessel-destruction rep stays. **Deferred (v1 limitation)**.

7.45 Merge interruption recovery

Journaled commit + finisher triggered by journal presence on load, regardless of marker state. Five distinct crash windows covered. **Shipped (test)**: `MergeCrashRecoveryMatrixTests` + `MergeJournalOrchestratorTests` + in-game `MergeInterruptionRecoveryTest` / `JournalFinisherMarkerPresentVariantTest`.

7.46 EffectiveRecordingId walk direction + orphan handling

Walks forward via `RecordingSupersedes`. Cycle detection via visited set (`EffectiveState.cs:88`). One-sided orphan relation (new endpoint missing) is treated as a chain terminator at prior node and logs Warn. Fully orphaned rows are removed by load-time sweep before they can warn forever. **Shipped (test)**: `EffectiveStateTests` + `ChildSlotEffectiveRecordingIdTests`.

7.47 Orphan supersede relation on load

Step 4 of load sweep logs Warn and retains one-sided orphans; fully orphaned rows (`oldResolved=false && newResolved=false`) are removed because neither endpoint can affect effective state. Walk treats retained one-sided orphans as terminators. **Shipped (test)**: `LoadTimeSweepTests`.

7.48 Immutable-merged re-fly — player wants to retry anyway

They cannot. `Immutable` seals the slot. To retry, the player must Full-Revert, which clears the rewind point + supersede / tombstone state for the tree (the committed recordings stay in the timeline). **Shipped (integration)**: documented behavior + `TreeDiscardPurgeTests` for the full-revert path.

7.49 4-probe deploy from a Mun mothership (the canonical v0.9.1 stable-leaf case)

Mothership is focus → Immutable. 4 probes are non-focus, terminal Orbiting → 4 CP slots → 4 UF rows. RP stays alive while any probe slot is unsealed. **Shipped (in-game test)**: end-to-end coverage in `RuntimeTests` (Mun-mothership-deploy → fly probe #2 → land → merge → slot closure + supersede chain extension).

7.50 Auto-parachute booster, focus on upper stage

Booster non-focus, terminal Landed → not UF (Landed always returns false from `TerminalOutcomeQualifies`) → Immutable. Upper stage is focus, terminal Orbiting → not UF (focus exclusion) → Immutable. RP reaps cleanly. **Critical regression guard test** (largest-impact failure mode of an over-broad predicate).

7.51 Inverted: focus on booster, upper stage left orbiting

Booster is focus, terminal Landed → not UF → Immutable. Upper stage non-focus, terminal Orbiting → UF → CP. RP stays alive while upper-stage slot is unsealed.

7.52 Stranded EVA kerbal (alive)

EVA kerbal terminal Landed (or Splashed for water) on a body, `EvaCrewName` non-null. Kerbal branch returns true regardless of focus. UF. Player can Fly to attempt reboard. **Shipped (test)**: `UnfinishedFlightClassifierTests` per-state matrix.

7.53 EVA kerbal reboards

Board BP fires. EVA recording's chain TIP `ChildBranchPointId` = boardBp.Id. Per-RP leaf gate: matchingRP.BranchPointId is the original EVA BP, not the Board BP. ChildBranchPointId != null AND != matchingRP.BranchPointId → leaf gate fails. Not UF.

7.54 Dead EVA kerbal

Terminal `Destroyed`. Routes through the Crashed branch (Destroyed always returns true regardless of focus or kerbal status). UF — same predicate path as a crashed vessel. Distinct from §7.16 only in that the EVA branch was never reached because Destroyed short-circuits first. Tombstone + crew-recovery behaviour matches §7.16 once the player merges a successful re-fly.

7.55 Breakup-survivor active parent, terminal Crashed

Survivor V's chain TIP terminal Destroyed. Crashed branch returns true regardless of focus. UF. v0.9.0 behavior preserved exactly.

7.56 Breakup-survivor active parent, terminal Landed

Survivor V's chain TIP terminal Landed. Landed always returns false from `TerminalOutcomeQualifies`. Not UF. RP reaps when other slots close. v0.9.0 behavior.

7.57 Breakup-survivor active parent, terminal Orbiting, focus slot

Survivor V is FocusSlot, terminal Orbiting → not UF (focus exclusion). Player loses access to re-fly the breakup moment via UF by default unless they use Stash (§6.25.1) while the backing RP still exists. The origin-only fallback from §4.8 does not bypass focus exclusion; it only restores the missing RP-slot link when the slot would otherwise qualify.

7.58 Cross-tree dock during stable-leaf re-fly

Re-flown probe docks with another tree's station. Dock BP fires; probe-re-fly's chain TIP gets `ChildBranchPointId = dockBp.Id`. Per-RP leaf gate fails. Site B-1 sees the downstream structural/world interaction →

`TerminalOutcomeQualifies` returns false → Immutable, auto-sealed. AppendRelations closure walk (rooted at SupersedeTargetId) is tree-scoped and halts at the mixed-parent BP; station's tree unaffected.

7.59 Re-fly a stashed probe, end in Mun orbit

Re-fly merge under the v0.9.1 auto-seal revision (§4.9): Site B-1 calls the classifier with `focusSlotOverride: slotIdx`. The chain tip is Orbiting and the slot the player chose to fly equals the override, so the classifier returns `classifierReason=stableTerminalFocusSlot`. `ShouldKeepReFlySlotOpenAfterMerge` exits at `!classifierQualifies` with `closeReason=ClassifierClosed/stableTerminalFocusSlot`; `MergeState=Immutable` and `slot.Sealed=true`. Supersede still appends `{priorTip -> provisional}` so the timeline points at the new recording. The row drops from UF. The player intentionally engaged this slot to fly it; the merge concludes the engagement. If the player wants the stable-orbit retry to remain open, they don't merge — they Discard or Retry the Re-Fly. The variant where the re-fly also credited `ScienceEarning` (Crew Report / EVA Report / Surface Sample / Transmit / Recover) follows the same Immutable + auto-seal path through the existing `recordingAction:*` reason (the retry-blocking action gate fires before the focus override branch). KSC-scene player decisions (`ScienceSpending`, `FundsSpending`, `ContractAccept` / `Cancel`, `KerbalHire`, `FacilityUpgrade` / `Repair`, `StrategyActivate` / `Deactivate`) do not trip this gate — they emit from KSC scenes with no flight-recording tag in practice; nor do automatic rows such as `FundsEarning`, `ReputationEarning`, `ReputationPenalty`, `MilestoneAchievement`, `ContractComplete`, or `ContractFail`.

7.60 Re-fly an auto-included stable-leaf probe, end Landed

The chain tip is Landed (a stable conclusion) and the slot the player chose to fly equals `focusSlotOverride`. Under the v0.9.1 focus-override contract, `IsReFlyOverrideStableTerminal` accepts `{Orbiting, SubOrbital, Landed, Splashed}`, so Site B-1's classifier returns `classifierReason=stableTerminalFocusSlot`; `MergeState=Immutable` and `slot.Sealed=true`. The row drops from UF and the rewind point reaps once all sibling slots are closed. Re-fly merges from non-Re-Fly call sites (`focusSlotOverride=null`) still fall through to the existing `terminal != Orbiting && terminal != SubOrbital` branch and return `stableTerminal` without auto-seal — used by `EffectiveState.IsUnfinishedFlightCandidate`, the natural scene-exit promotion path, and reap-eligibility checks. Retry-blocking `ScienceEarning` recording-linked actions still fire ahead of the override and report `recordingAction:*` + auto-seal. Manual-Stash variant: a stashed slot Re-Fly to Landed routes through the same `stableTerminalFocusSlot` close reason because the override precedes the stashed-keep-open branch by design.

7.61 Re-fly a stashed stranded EVA kerbal, succeed in reboarding

Re-fly merge produces a Board BP. Provisional has `ChildBranchPointId = boardBp.Id`. `TerminalOutcomeQualifies` returns false (Boarded → kerbal branch returns false). Site B-1 → Immutable and auto-sealed. Stranded-kerbal-recovery path complete.

7.62 Player Seals a UF row

`slot.Sealed = true`; row drops from group; reaper runs (RP deleted if all sibling slots also closed, NotCommitted-not-allowed). Recording unchanged; ghost playback unchanged on subsequent rewinds. **Shipped (test)**: Seal handler + reaper integration tests.

7.63 Player Seals a Crashed row (not a stable leaf)

Same Seal handler. The Crashed terminal originally qualified the row; Sealing closes it as "I accept the crash as canonical." Provides v0.9.0 users with a cleanup affordance they didn't have before.

7.64 Sealed row was never Immutable

A CP slot whose recording is in chain extension (e.g. probeReFly1 with subsequent CP attempts). Player Seals. `slot.Sealed = true` ; reaper sees CP+Sealed → equivalent-to-Immutable for close-eligibility. RP reaps when other slots close.

7.65 Sealed slot whose effective recording is NotCommitted

Should not occur in normal operation (NotCommitted recordings are not in ERS, so the row never appears as UF, so the player can't Seal it via the UI). If it occurs through a load-time race state (journal finisher rolled MergeState back, slot.Sealed bit still on disk), reaper's NotCommitted-unconditional-no-reap rule (§6.23) prevents the RP from being deleted. **Defense-in-depth test.**

7.66 Pre-feature save load — Orbiting non-focus sibling from before upgrade

Legacy RP loads with `FocusSlotIndex == -1` . `noFocusSignal` short-circuit returns false for Orbiting/SubOrbital. The legacy Immutable Orbiting sibling stays Immutable; row does not appear in UF. **Forward-only migration for vessels.** See §9.4.

7.67 Pre-feature save load — Crashed sibling from before upgrade

Legacy RP loads with `FocusSlotIndex == -1` . Crashed branch returns true regardless of focus. Row appears in UF as it did in v0.9.0. **Regression guard.**

7.68 Pre-feature save load — stranded EVA kerbal from before upgrade

Legacy RP loads with `FocusSlotIndex == -1` . EVA branch returns BEFORE the noFocusSignal short-circuit. Stranded kerbal qualifies. Row appears in UF post-upgrade. **Intentional retroactive carve-out** — see §9.5 (the carve-out itself) and §9.6 (CHANGELOG split note).

7.69 New post-feature RP, no slot was focused at split time

RP captures with explicit `FocusSlotIndex = -1` (player was focused on an unrelated vessel outside the split). Same `noFocusSignal` behavior as legacy RPs: Orbiting/SubOrbital suppressed; Crashed and EVA-stranded qualify. Manual Stash allows the player to add Orbiting siblings from these rare RPs while the backing RP still exists.

7.70 BG-only multi-controllable split with all controllable Orbiting siblings

RP captures with `FocusSlotIndex = -1` (no focus involved). All sibling slots stay Immutable post-commit. No UF rows. RP reaps. Same outcome as legacy save case.

7.71 In-place re-fly merge ending Orbiting (Site B-2 behavior)

Player drives a re-fly merge through `MergeDialog.TryCommitReFlySupersede` (in-place path). Under the v0.9.1 revision, Site B-1 sees Orbiting on the chain tip with `focusSlotOverride=slotIdx` matching the merge-time slot → `stableTerminalFocusSlot` → `Immutable` + `slot.Sealed=true` . Site B-2 does not override; it lets the reaper remove the RP when all sibling slots are closed. Retry-blocking recording-linked actions still fire ahead of the focus override branch and auto-seal via `recordingAction:*` (unchanged).

7.71b In-place re-fly merge after a session-authored decouple

Player Re-Flies a stashed slot whose chain tip is stable and stages debris during the session. Under the v0.9.1 focus-override contract, the classifier reaches `stableTerminalFocusSlot` before the stashed keep-open branch, so the chosen slot closes as `Immutable` + `slot.Sealed=true` . The structural-mutation helper still detects the new

`Breakup` BP authored at `UT > rp.UT` (and not in `marker.PreSessionBranchPointIds`) as a defensive backstop for safe-stable retry paths that do not close through the override. Pure-ablation part loss without spawning a sibling vessel does NOT trip this gate (no `BranchPoint` is authored). Destroyed / stranded-EVA in-place merges use the same gate: pure terminal failure, plus automatic consequence rows or tombstoneable kerbal-death ledger rows, stays `CommittedProvisional`; any additional retry-blocking action closes the slot.

7.72 Hybrid star+linear supersede graph

Save loaded with legacy star portion `{probeOrig -> probeReFly1, probeOrig -> probeReFly2}` from a pre-§6.22 chain extension. Player invokes a new re-fly into the same slot. New invocation computes `priorTip = slot.EffectiveRecordingId(supersedes)` — the walker picks the oldest member of the star (probeReFly1) and walks linearly from there. New relation appended: `{probeReFly1 -> probeReFly3}`. Resulting graph is hybrid: star portion preserved, linear portion grows from probeReFly1. `TryResolveRewindPointForRecording(probeReFly3, ...)` returns the slot via the forward walk from slot.OriginChildRecordingId → probeReFly1 → probeReFly3. **Tolerated, no migration sweep needed.**

7.73 Crash-quit during a stable-leaf re-fly

Marker validates against on-disk session-provisional + RP. Session resumes. **No new state vs v0.9.0.** `MarkerValidator.Validate` extension weakly validates `SupersedeTargetId`; failure clears the field and AppendRelations falls back to `OriginChildRecordingId`.

7.74 Player reverts a stable-leaf re-fly mid-flight

`RevertInterceptor` 3-option dialog appears. Discard Re-fly preserves the stashed row in UF (origin RP promoted, slot still CP). **Same as v0.9.0** (the merge-close gate in §6.21 doesn't affect Discard Re-fly because the Discard path doesn't reach the merge classifier).

7.75 Tree discard during chain-extension state

`TreeDiscardPurge.PurgeTree` removes every RP, supersede relation, and tombstone scoped to the discarded tree. The new `slot.Sealed` / `slot.Stashed` flags on the tree's RP slots are removed alongside the RPs. `marker.SupersedeTargetId` clears alongside the marker. **No new code paths needed in TreeDiscardPurge** — the discard already removes the parent objects.

7.76 Recording with no terminal state

`TerminalStateValue` is null (finalization didn't run cleanly). `TerminalOutcomeQualifies` returns false → not UF. Logged as `reason=noTerminal`. The upstream finalization-reliability contract in `parsek-recording-finalization-design.md` is responsible for ensuring this is rare.

---

8. What Doesn't Change

- **Flight-recorder principle 10** (additive-only recording tree). Enforced rigorously: supersede is a relation, not a field; tombstones are append-only; no field on a committed Recording is ever written post-commit; orphan supersede relations are never deleted outside tree-discard.
- **Single-controllable split path.** Unchanged.
- **Merge dialog UI shell.** Unchanged; the feature adds a supersede-advisory line when an active session is present.
- **Ghost playback engine.** Reads ERS via `SessionSuppressionState`; `SessionSuppressedSubtree` is upstream of the engine, not inside it. The v0.9.1 closure-helper split (§6.22) preserves the wrapper contract

— runtime ghost suppression continues calling `ComputeSessionSuppressedSubtree(marker)` and gets the same return value as before.

- **Active-vessel ghost suppression.** Natural ERS filter (`NotCommitted` recordings are excluded; the live provisional is `NotCommitted`).
- **Ledger recalculation engine.** Reads ELS via `EffectiveState.ComputeELS` ; no engine change.
- **Rewind-to-launch (the existing T0 quicksave-reload feature).** Unchanged path.
- **Loop / overlap / chain.** Read ERS through the same filter; no per-feature code change.
- **Recording sidecar format.** No changes.
- **Reservation manager internals.** Re-derivation from ERS is existing; the carve-out in `IsLiveReFlyCrew` is a single-method filter.
- **Career state (KSP-sticky subsystems).** The feature never touches `ContractSystem` , `ProgressTracking` , `ScenarioUpgradeableFacilities` , `StrategySystem` , `ResearchAndDevelopment` , or the vessel-destruction rep code paths on supersede. Contracts completed by the superseded recording stay complete. Milestones earned stay earned. Facility upgrades stay upgraded. Tech researched stays researched.
- **Vessel-destruction reputation.** Not a tombstone-eligible type. The `ReputationPenalty` action from a vessel destruction in a superseded subtree stays in ELS; career rep is unchanged by supersede unless a kerbal also died.
- **Science rewards.** `ScienceSpending` and `ScienceEarning` stay in ELS.
- **v0.9.0 Crashed UF behavior (preserved by v0.9.1 extension).** Every recording that was UF under v0.9.0 is still UF under v0.9.1 (preserved by §7.55 + §7.67). Site A's broader predicate subsumes the v0.9.0 Crashed-only check; the Crashed branch returns true regardless of focus / kerbal status / `FocusSlotIndex` short-circuit.
- **Reaper "every slot closed → reap" rule.** Same shape as v0.9.0; the close definition extends by one term (`slot.Sealed == true`) and tightens by one (NotCommitted unconditional no-reap).
- **`marker.OriginChildRecordingId` contract.** Still the slot's immutable origin, even after the v0.9.1 invocation linearization. Existing consumers (`RevertInterceptor.FindSlotForMarker` , in-place continuation, ghost suppression) continue to read it unchanged.
- **CommitTombstones tombstone scope.** Still uses the closure from AppendRelations; the closure now correctly walks the prior-tip's descendants on chain extension (more accurate, not different in shape).
- **Discard Re-fly + Retry from RP semantics.** Both run before the merge classifier; the v0.9.1 Site B-1 / Site B-2 changes don't affect them.
- **TreeDiscardPurge.** New v0.9.1 fields are removed alongside their parent objects; no purge-side changes.
- **ERS / ELS routing rules.** No new raw-read sites; the v0.9.1 classifier reads through `EffectiveState.ComputeERS()` like every other consumer.
- **EVA kerbal-death tombstone scope.** v0.9.0 §6.13 narrow scope is unchanged. The v0.9.1 EVA carve-out affects only the predicate (which rows show), not the tombstone-eligibility classifier.
- **Legacy saves.** `GameAction.ActionId` is auto-generated via deterministic hash on first post-upgrade load; `MergeState` migrated from the binary legacy flag; new ConfigNode sections (`REWIND_POINTS` , `RECORDING_SUPERSEDES` , `LEDGER_TOMBSTONES` , `REFLY_SESSION_MARKER` , `MERGE_JOURNAL`) are optional and absent on pre-v0.9.0 saves. v0.9.1 fields (`ChildSlot.Sealed` / `Stashed` , `RewindPoint.FocusSlotIndex` , `ReFlySessionMarker.SupersedeTargetId` / `PreSessionBranchPointIds`) are absent on pre-v0.9.1 saves; defaults match v0.9.0 behaviour.
- **Standalone ghost playback mod (Gloops).** The feature touches only Parsek-internal code; nothing the engine depends on moves.
- **Existing chain semantics outside a re-fly session.** BranchPoints, chain tips, ghost extension, trajectory walkback, etc. are read-only consumers of the new ERS view; the session-suppressed subtree only takes effect while a marker is live.

---

9. Backward Compatibility

9.1 Pre-v0.9.0 saves on v0.9.0+

- Load cleanly. All five new ConfigNode sections are absent → treated as empty lists / null singletons (`REWIND_POINTS`, `RECORDING_SUPERSEDES`, `LEDGER_TOMBSTONES` → `List<T>()`; `REFLY_SESSION_MARKER`, `MERGE_JOURNAL` → `null`).
- `Recording.MergeState` legacy migration: binary `committed = True` → `Immutable`; `committed = False` → `NotCommitted`; absent → `Immutable` (pre-feature invariant: any recording reachable from a committed tree was already sealed). One-shot `[LegacyMigration] Info` log line on first post-upgrade load.
- `GameAction.ActionId` legacy migration: deterministic hash `act_<hash>` generated from `UT + Type + RecordingId + Sequence` (`GameAction.cs:541`). Idempotent — the same action always yields the same id. One-shot `[LegacyMigration] Info` log line.
- `Recording.SupersedeTargetId` — never set on pre-v0.9.0 saves. Non-empty values on `Immutable` / `CommittedProvisional` recordings (should only happen on a corrupt or hand-edited save) log Warn and are treated as cleared (`LoadTimeSweep.ClearStraySupersedeTargets`, `LoadTimeSweep.cs:326`).
- `BranchPoint.RewindPointId` — absent on pre-v0.9.0 saves. No `IsUnfinishedFlight` predicate matches on pre-feature BG-crashed siblings because the predicate requires a non-null parent `RewindPointId`. The player's existing crashed siblings are NOT retroactively made Unfinished Flights.
- Pre-v0.9.0 recordings have no cargo/crew manifest changes; Rewind to Separation does not read or write those fields.

9.2 Post-v0.9.0 saves on v0.8.x

Not supported. A post-v0.9.0 save carries `REWIND_POINTS` / `RECORDING_SUPERSEDES` / `LEDGER_TOMBSTONES` / `REFLY_SESSION_MARKER` / `MERGE_JOURNAL` nodes that pre-v0.9.0 Parsek does not know. Depending on the KSP version's ConfigNode parser tolerance, the save may:

- Load cleanly but silently drop the new sections on next save (losing supersede relations → superseded recordings reappear in ERS).
- Fail to load with a KSP-side parse error.

No downgrade path is provided.

9.3 Feature removal

All new entities are ignorable as orphans. Removing the feature from a save would leave `REWIND_POINTS` etc. unused; the shipped code does not offer a "strip rewind state" toggle.

9.4 Pre-v0.9.1 saves on v0.9.1+

- Load cleanly. The four new ConfigNode keys (`sealed`, `stashed`, `focusSlotIndex`, `supersedeTargetId`) are absent on disk. Default values match v0.9.0 behavior:
  - `slot.Sealed = false` → no slots are pre-Sealed; existing CP slots stay open.
  - `slot.Stashed = false` → no slots are pre-Stashed; default predicate continues to govern membership.
  - `RewindPoint.FocusSlotIndex = -1` → the `noFocusSignal` short-circuit suppresses Orbiting/SubOrbital qualification on every legacy RP. **Forward-only migration for vessels.**
  - `marker.SupersedeTargetId = null` → AppendRelations coalesces with `?? OriginChildRecordingId` fallback. **No change in supersede behavior for legacy markers** (they continue to produce star-shaped graphs from the prior re-fly invocation).

- `marker.PreSessionBranchPointIds = null` → structural-mutation gate (§4.9) skips conservatively on those markers, preserving pre-fix keep-open behavior on merge for in-flight sessions at upgrade time.

9.5 Stranded EVA kerbals: retroactive carve-out

Stranded EVA kerbals from legacy saves DO retroactively appear in UF — the EVA branch returns BEFORE the noFocusSignal short-circuit. **Intentional asymmetry** vs §9.4's vessel forward-only rule. The asymmetry is the only retroactive-surfacing exception in the v0.9.1 migration story; see CHANGELOG split in §9.6 for the player-facing explanation.

9.6 v0.9.1 CHANGELOG split

The v0.9.1 migration + behavior-change story splits into three notes that must be presented separately to the player:

> ***Vessels: forward-only.*** *Past missions where you deployed probes or stages and left them parked are not retroactively converted to Unfinished Flights. The feature only surfaces splits made after the upgrade. (Why: legacy data has no focus signal, and we'd rather under-include than flood your list with every routine upper stage from your career.)*

> ***Stranded EVA kerbals: retroactive.*** *EVA kerbals who got stranded on a body in past missions DO appear in Unfinished Flights after the upgrade. Click Fly to attempt a rescue. (Why: stranded kerbals are unambiguous — you almost certainly want them back — unlike orbital siblings where intent is unclear.)*

> ***Re-Flying a stashed entry to a stable conclusion seals the slot.*** *When you Fly a UF entry and the merge commits the slot's chain tip at a stable terminal (`Orbiting`, `SubOrbital`, `Landed`, `Splashed`, plus the existing `Recovered` / `Docked` / `Boarded` set), Parsek seals that retry slot — the recording is `Immutable`, the row drops from Unfinished Flights, and the rewind point's quicksave is reaped on the next sweep. The same applies if the Re-Fly authors any structural shape change during the session (decouple, stage, undock, joint break, kerbal EVA — anything that creates a new sibling vessel). The intent: when you actively engage a slot to fly it yourself, the merge concludes that engagement. Destroyed vessels and non-boarded EVA kerbals still stay retryable for another attempt unless they also credited science via Crew Report / EVA Report / Surface Sample / Transmit / Recover during the recording. Automatic milestones / contract complete-fail / funds-rep rewards, KSC-scene player decisions you happened to make while the recording was live (tech unlock, contract accept/cancel, hire, facility upgrade/repair, strategy toggle, vessel build cost), and tombstoneable kerbal death stay retryable. Routine non-structural part actions like deploying solar panels, ladders, gear, cargo bays, or braking a rover do NOT count as structural mutation. If you want to keep a stable Re-Fly retryable, don't merge — Discard the Re-Fly and revisit the slot later.*

9.7 Post-v0.9.1 saves on v0.9.0

Not supported. The four new ConfigNode keys (`sealed`, `stashed`, `focusSlotIndex`, `supersedeTargetId`, plus `preSessionBranchPointIdsPresent` / `preSessionBranchPointId`) would be silently dropped on next save. The player would lose Sealed / Stashed state and the FocusSlotIndex; UF predicate would degrade to the v0.9.0 Crashed-only behavior on the now-modified save. **No downgrade path provided.**

9.8 Hybrid supersede graphs

Per §6.22 migration concern + §7.72: legacy star-shaped supersede portions in pre-v0.9.1 chains stay as-is. New post-feature chains extend linearly from whichever leaf the walker picks in the star portion. No migration sweep.

---

10. Known Limitations / Future Work

10.1 ERS-exempt consumers (index-keyed state)

Thirteen files are on the ERS-audit allowlist (`scripts/ers-els-audit-allowlist.txt`) with inline `[ERS-exempt – Phase 3]` comments and `TODO(phase 6+)` markers. These hold raw index-based state keyed by positions in `RecordingStore.CommittedRecordings` :

- `Source/Parsek/GhostMapPresence.cs`
- `Source/Parsek/WatchModeController.cs`
- `Source/Parsek/ChainSegmentManager.cs`
- `Source/Parsek/ParsekFlight.cs`
- `Source/Parsek/ParsekPlaybackPolicy.cs`
- `Source/Parsek/ParsekTrackingStation.cs`
- `Source/Parsek/Diagnostics/DiagnosticsComputation.cs`
- `Source/Parsek/FlightRecorder.cs`
- `Source/Parsek/ParsekKSC.cs`
- `Source/Parsek/ParsekUI.cs`
- `Source/Parsek/UI/RecordingsTableUI.cs`
- `Source/Parsek/UI/GroupPickerUI.cs`
- `Source/Parsek/UI/SettingsWindowUI.cs`

The follow-up work is a recording-id-keyed refactor so the remaining grep-audit exemptions can be lifted. Each exemption has a call-site comment explaining why the raw index read is currently required. The refactor is structurally large (touches every subsystem that keys on committed-recordings index) and deferred beyond v0.9.

10.2 Background-split RP capture relies on cached PID payload

For BG-recorded splits that originate outside the focus vessel, the RP author consumes a cached PID payload produced by `BackgroundRecorder` . A joint-break during high warp on a truly unloaded vessel may not emit the event, in which case no RP is captured and the BG-crashed sibling cannot be rewound. Covered by `BackgroundSplitRpTests` for the supported paths; edge cases with on-rails breakups remain deferred work.

10.3 Wider tombstone scope (v2)

Candidate types beyond `KerbalAssignment` +Dead and bundled rep:

- Vessel-destruction `ReputationPenalty` (unbundled from kerbal deaths). Needs confirmation that KSP-side rep can be safely refunded without re-emitting a destruction event.
- Science-loss on destruction. Parsek-owned, should be tombstone-safe.
- Contract completion / failure for cases where the KSP-side `ContractSystem` can be scripted to re-emit on re-fly. Probably not possible without invasive Harmony work.
- Milestone flags for `ProgressTracking` categories where the binary flag can be safely cleared and re-emitted.

Each candidate needs per-type confirmation that KSP-side state can either be re-emitted or is purely Parsek-owned.

10.4 Cross-tree supersedes

`EffectiveState.EffectiveRecordingId` has an inline TODO (`EffectiveState.cs:76`) to halt the walk at cross-tree boundaries once cross-tree supersedes become producible. v1 does not produce them; the invariant is enforced by the merge logic (supersede relations are scoped to the session's tree).

10.5 End-to-end automated in-game RunInvoke test

Stubbed. Scene reload is not drivable under xUnit, and the KSP in-game test runner does not yet support the full Restore → Strip → Activate → AtomicMarkerWrite pipeline as a single test. The in-game `InvokeRPStripAndActivateTest` covers the strip + activate portion; the pre-load copy-to-root / scene reload is exercised only via manual playtest.

10.6 Auto-purge policies for reap-eligible RPs

There are none in v1. The disk-usage diagnostic surfaces the current total; the player can Wipe All Recordings to clear them. A scheduled purge for old reap-eligible RPs (TTL-based) is future work.

10.7 Recovery-snapshot feature

The pre-impl doc's `SplitTimeSnapshot` concept (`_split.craft` sidecar, redundant to the quicksave) was dropped before v1. A concrete recovery mechanism for corrupt quicksaves is deferred; v1's Corrupted flag keeps the RP visible for diagnostic purposes only.

10.8 Player-selectable merge mode for crashes

v1 auto-picks `CommittedProvisional` for crash outcomes per `TerminalKindClassifier` . v2 could offer "commit as final (no further rewind)" for players who want to seal the slot and accept the crash.

10.9 SpawnGhost_PrimesFreshGhostToCurrentPlaybackUT

A pre-existing Unity-gated test skip unrelated to Rewind to Separation, but exercised heavily by the feature's in-game tests. Carried forward as an open follow-up from earlier phases.

10.10 v0.9.1 invocation linearization (shipped)

The §6.22 closure-helper split + marker-write change shipped as a v0.9 prerequisite PR before the v0.9.1 stable-leaf extension. Its requirements:

- Adds `marker.SupersedeTargetId` field with weak `MarkerValidator.Validate` extension.
- Modifies `RewindInvoker.AtomicMarkerWrite` to compute `priorTip` and stamp both marker fields + the provisional (guarded for in-place).
- Splits `EffectiveState.ComputeSessionSuppressedSubtree(marker)` into the public wrapper + new `ComputeSubtreeClosureInternal(marker, rootOverride)` .
- Modifies `SupersedeCommit.AppendRelations` to call the internal helper with `marker.SupersedeTargetId ?? marker.OriginChildRecordingId` .

Listed in the limitations section as historical context; the prerequisite is not a future-work item.

10.11 Auto-purge of long-lived sealed RPs (deferred)

No TTL-based reaper extension for sealed-pending RPs. Disk usage is the player's responsibility via the Seal button, surfaced through the Settings → Diagnostics disk-usage line and its v0.9.1 crashed-open / stable-open / sealed-pending RP breakdown (§6.18). A later release could add a TTL-based reaper extension or a "Wipe All Sealed RPs" button, but the v0.9.1 design does not invent destructive cleanup policy.

10.12 SealedBy / SealedReason persisted metadata (deferred schema follow-up)

Persistence stores `slot.Sealed` / `slot.SealedRealTime` only; the auto-seal reason from the merge gate is logged at merge time but not written to the slot. Persisted `SealedBy` (`player` vs `mergeAutoSeal`) and `SealedReason` metadata is a deferred schema follow-up tracked in `docs/dev/todo-and-known-bugs.md` so post-mortem analysis of why a slot closed can read directly from the save.

---

11. Risks

The shipped feature carries a handful of known risks. Each is mitigated in code or playtest practice; this section is the inventory for code reviewers and reviewers of follow-up changes.

- **Disk usage growth.** Stable-leaf RPs persist while they are offered in Unfinished Flights (Site A keeps them open). Under the v0.9.1 auto-seal revision (§4.9), a *successful* Re-Fly merge now seals the slot and the RP becomes reap-eligible — so the disk-usage growth from this feature is bounded by un-Re-Flown rows and crash/death retries, not by repeated re-flies of the same slot. Crash/death retry RPs still persist until the player succeeds, hits a safety gate, or explicitly Seals them. Mitigation: the Settings → Diagnostics RP disk-usage line includes live crashed-open, stable-open, and sealed-pending RP buckets next to the byte/file total (§6.18).

- **Over-inclusion of the stable-leaf predicate.** The simple terminal-state classifier surfaces some cases where the player meaningfully flew or didn't intend to leave a vessel and it ended Orbiting/SubOrbital (Mun-transfer-and-park, briefly-nudged-probe, deep-space probe carried by transfer stage and undocked there). The Seal button is the design's answer. **Do not re-introduce voluntary-action heuristics** — that path was explicitly rejected in the R1-R3 research-note exploration.

- **Predicate-vs-merge policy confusion.** Site A's UF-display predicate and Site B's post-merge policy share the classifier reason but compare against different focus signals. Site A uses the static `rp.FocusSlotIndex`, so non-focus stable leaves stay open. Site B-1 passes `focusSlotOverride: slotIdx` so the merge-time slot becomes the de-facto focus and a successful Re-Fly closes it. Tests must pin both the Site A classifier reason (`stableLeafUnconcluded` for non-Re-Fly callers) and the Site B merge result (`stableTerminalFocusSlot` + `Immutable` + auto-seal) so this contract does not regress into accidental closure or paradox-prone re-fly exposure.

- **Irreversible RP reap on stable Re-Fly.** Site B-1 auto-seal makes the slot reap-eligible immediately at merge, so the post-merge sweep deletes the rewind-point quicksave file. A successful Re-Fly to stable orbit, or one that authored a structural BP, cannot be re-flown again afterward — the on-disk file is gone. This is intended behaviour ("player is done with that line of flight") but it removes the synchronization pass that an earlier draft of the design's keep-open contract was meant to enable. Players who want that retry path must Discard / Retry the Re-Fly instead of merging.

- **Structural-mutation false-positive surface.** Detection scans tree branch points with `UT > rp.UT` and excludes `marker.PreSessionBranchPointIds` (the snapshot taken at marker creation). The session-local baseline closes the obvious false-positive vector — `SpliceMissingCommittedRecordingsIntoLoadedTree` re-grafting pre-Re-Fly post-RP BPs back into the loaded tree. The remaining surface is BPs authored after marker creation but before the player actually engaged the slot (e.g. an RP captured during a F5 quicksave that snapshotted a tree mid-stage). The marker is created synchronously with the RP load, so this is a narrow window; tests must pin the cutoff and baseline together to prevent regression.

- **Reversal of v0.9.0 §7.31 stance.** v0.9.0 said "stable-end splits explicitly not in scope." v0.9.1 says some non-focus stable-end splits ARE in scope (Orbiting/SubOrbital non-focus, EVA-stranded). Focus-continuation stable terminals continue to NOT get a row, preserving the "your mission's upper stage didn't suddenly become unfinished" intuition. The CHANGELOG must be precise about what changed and what didn't (see §9.6).

- **Focus-slot breakup-survivor with stable-orbit terminal still isn't auto-added to UF** (§7.57). Origin-only RP-slot resolution fixes tree-branching parent rows that are otherwise eligible, including crashed parents and non-focus Orbiting/SubOrbital parents. It deliberately does not bypass focus exclusion: a focus-slot

survivor that reaches stable orbit is considered the player's continued mission unless they explicitly use Stash while the backing RP still exists.

- **Site B-2 in-place duplicate-row invariant.** The in-place path has no separate provisional, so the recording remains its own effective slot target. Site B-1 owns the close/open decision under the v0.9.1 revision; Site B-2 lets the reaper clean up after stable Re-Fly outcomes (now `Immutable` + auto-sealed) and preserves CP only for terminal-failure outcomes that still allow retry.
- **Legacy hybrid supersede graphs.** §7.72 — tolerated by the design but the hybrid-graph regression test must run green for confidence. If the walker behavior on hybrids differs from expectations, a migration sweep may need to be added back.
- **EVA stranded edge cases.** What if KSP unloaded the kerbal mid-EVA and the terminal is unreliable? The `parsek-recording-finalization-design.md` work is the upstream contract. This feature inherits whatever finalization reliability that work delivers.
- **Optimizer chain length (RESOLVED).** Earlier drafts flagged eccentric-orbit BG-recorded vessels with periapsis-grazing atmosphere as a possible source of unbounded chains. Resolved by the `optimizer-meaningful-split-rule.md` investigation: `BackgroundOnRailsState` omits `currentTrackSection` / `trackSections` / `environmentHysteresis` entirely (`BackgroundRecorder.cs:157`), and `OnBackgroundPhysicsFrame` early-returns on `bgVessel.packed` . On-rails BG vessels can't generate optimizer-splittable Atmospheric↔ExoBallistic toggles. Guarded by `EccentricOrbitOptimizerInvariantTests` . No action needed for this feature.

---

12. Diagnostic Logging

Every decision point in §6 emits a log line. The tag catalog below enumerates what appears in `KSP.log` for a Rewind-to-Separation session. **All tags are verified against the shipped source** in the files named in §1.1. The pre-impl spec's candidate tags `Reap` , `Strip` , and `Tombstone` did NOT ship as distinct tag names — their log lines fold into `[Rewind]` and `[LedgerSwap]` respectively. `RewindSave` , `ERS` , and `ELS` DID ship as distinct tags and are cataloged below.

12.1 `[Rewind]`

Broad tag for RP lifecycle and strip outcomes. Used by `RewindPointAuthor` , `RewindInvoker` , `PostLoadStripper` , `RewindPointReaper` , `TreeDiscardPurge` , `LoadTimeSweep` , `RewindPointDiskUsage` .

- RP creation: `Info: RewindPoint begin: rp=<id> bp=<bpId> slots=<N> controllablePids=<M> ut=<x>` .
- Slot capture summary: `Info: RewindPoint slot capture: rp=<id> populated=<p>/<total> disabled=<d>` .
- Partial capture warning: `Warn: Slot <i> disabled: no live vessel for rec=<rid> (rp=<id>)` .
- All-slot failure: `Warn: All slots disabled; RP unusable (rp=<id>). Keeping for diagnostic visibility.`
- Rollback: `Info: RewindPoint rolled back: rp=<id> removals=<n>` .
- Strip: `Info: Strip stripped=[...] selected=<idx> ghostsGuarded=<N> leftAlone=<M> fallbackMatches=<f>` .
- Strip fallback match: `Warn: Fallback match via root-part v=<pid> rootPart=<rootPid> slotIdx=<idx>` .
- Strip leave-alone: `Verbose: Strip leaveAlone: unrelated v=<pid> name=<n>` .
- Strip ghost guard: `Verbose: Strip guard: ghost-ProtoVessel v=<pid> name=<n>` .

- Reaper: `Info: ReapOrphanedRPs: reaped=<R> remaining=<rem> sealedSlotsContributing=<S>` (v0.9.1: `sealedSlotsContributing` counter logs how many of the closed slots reached closure via the Seal path vs the Immutable path). `Verbose: ReapOrphanedRPs: reaped=0 remaining=<n>` when nothing to do.
- Tree discard: `Info: TreeDiscardPurge: tree=<id> removedRps=<r> removedSupersedes=<s> removedTombstones=<t>` .
- Load summary: `Info: [LoadSweep] Marker valid=<b>; spare=<n> discarded=<r> ...` (see `[LoadSweep]`).
- Disk usage Warn (directory walk failure): `Warn: RewindPointDiskUsage directory walk failed: <reason>` .

12.2 `[RewindSave]`

Narrower tag for the RP save pipeline. Used by `RewindPointAuthor` .

- Scene abort: `Warn: Aborted: scene=<loaded>` .
- Deferred-coroutine abort: `Warn: Aborted mid-coroutine: scene=<loaded>` .
- Warp drop: `Info: Warp dropped from <prior> to 0 for rp=<id>` .
- Warp restore: `Info: Warp restored to <rate> for rp=<id>` / `Warn: Warp restore to <rate> failed for rp=<id>: <reason>` .
- Save success: `Info: Wrote rp=<id> path=<relPath> bytes=<N> ms=<t>` .
- Save failure: `Error: Failed rp=<id> reason=<…>` .

12.3 `[RewindUI]`

UI-layer decisions. Used by `RewindInvoker.ShowDialog` and `ReFlyRevertDialog` .

- Invoke button clicked: `Info: Invoked rec=<rid> rp=<rpId> slot=<idx> listIndex=<i>` .
- Dialog cancelled: `Info: Cancelled rp=<rpId> slot=<idx> listIndex=<i>` .
- ReFlyRevertDialog input lock: `Verbose: ReFlyRevertDialog input lock set (<lockId>)` / `... cleared` .
- Null callbacks on dialog buttons: `Warn: ReFlyRevertDialog: <Button> button had null callback sess=<sid>` .
- Callback throws: `Error: ReFlyRevertDialog <Button> callback threw: <Type>: <message>` .

12.4 `[ReFlySession]`

Session lifecycle. Used by `RewindInvoker` , `SessionSuppressionState` , `MergeJournalOrchestrator` , `LoadTimeSweep` , `RevertInterceptor` , `ReFlyRevertDialog` , `TreeDiscardPurge` .

- Session start: `Info: Started sess=<sid> rp=<id> slot=<idx> provisional=<rid> origin=<oid> tree=<tid>` .
- Session-suppressed subtree Start/End: `Verbose: SuppressedSubtree entered sess=<sid> ids=[...]` / `... exited` .
- Retry: `Info: Retry sess=<old> → <new>` (issued by `RevertInterceptor.RetryHandler`).
- End (merged): `Info: End reason=merged sess=<sid> provisional=<rid>` .
- End (full revert): `Info: End reason=fullRevert sess=<sid>` .
- Revert dialog shown: `Info: Revert dialog shown sess=<sid>` .
- Revert dialog cancelled: `Info: Revert dialog cancelled sess=<sid>` .
- Marker valid on load: `Info: Marker valid sess=<sid> tree=<tid> active=<rid> origin=<oid> rp=<rpid>` .
- Marker invalid on load: `Warn: Marker invalid field=<f>; clearing` (v0.9.1: includes `field=SupersedeTargetId` for the new weakly-validated field).

- Nested cleanup: `Info: Nested session-prov cleanup sess=<sid> recordings=<n> rps=<m>`.

12.5 `[Supersede]`

Supersede relation writes + ERS-effective observations. Used by `SupersedeCommit`, `EffectiveState`, `LoadTimeSweep`, `TreeDiscardPurge`.

- Relation append: `Info: rel=<relId> old=<oid> new=<nid>`.
- Subtree count summary: `Info: Added <N> supersede relations for subtree rooted at <originId> (subtreeCount=<sc> skippedExisting=<se>)`.
- Flip MergeState: `Info: provisional=<rid> mergeState=<state> terminalKind=<kind> priorTarget=<oid>`.
- Advisory: `Info: Narrow v1: physical playback replaced; career state unchanged except kerbal deaths`.
- Cycle detection: `Warn: EffectiveRecordingId: cycle detected in supersede chain starting from <id> ...`.
- Orphan relation on load: `Warn: Orphan supersede relation=<relId> oldResolved=<b> newResolved=<b>`.
- Fully orphaned relation on load: `Info: Fully orphaned relation=<relId> old=<oid> new=<nid>; removing`.
- Tree-discard purge count: `Info: PurgeTree: removed <N> supersedes with endpoint in tree=<id>`.
- (v0.9.1) Site B-1 verdict: `Info: provisional=<rid> mergeState=<state> qualifies=<b> slot=<slotIdx> rp=<rpId> focusSlot=<rp.FocusSlotIndex> focusSlotOverride=<slotIdx> classifierReason=<reason> autoSeal=<b> autoSealReason=<reason>` — the merge-time classifier verdict including separate `focusSlot` and `focusSlotOverride` fields so the source of the focus decision is greppable.
- (v0.9.1) Auto-seal observation: `Info: Auto-sealed re-fly slot=<slotIdx> rec=<rid> rp=<rpId> terminal=<state> reason=<reason>` (where `reason` is one of `stableTerminalFocusSlot`, `recordingAction:ScienceEarning:<actionId>`, `structuralMutation:<bpType>`, `downstreamBp`, `IsHardSafetyTerminal`).
- (v0.9.1) Site B-1 slot lookup failure: `Error: Site B-1 slot lookup failed for provisional=<rid> rpId=<marker.RewindPointId> originChildRec=<marker.OriginChildRecordingId> supersedeTargetId=<marker.SupersedeTargetId>` — fires when `TryResolveRewindPointForRecording` fails after `AppendRelations`; indicates a §6.22 prerequisite regression or `AppendRelations` bug.

12.6 `[LedgerSwap]`

Tombstone writes + reservation-recompute diagnostics. Used by `SupersedeCommit`, `LoadTimeSweep`, `TreeDiscardPurge`.

- Tombstone batch: `Info: Tombstoned <N> actions (KerbalDeath=<d> repBundled=<r>); <M> actions matched subtree but type-ineligible (breakdown: contract=<c> milestone=<m> facility=<f> strategy=<s> tech=<t> science=<sc> funds=<fn> rep-unbundled=<ru>)`.
- Skip existing: `Verbose: skip existing tombstone action=<aid>`.
- Reservation recompute: `Info: CrewReservation re-derived; kerbals returned active: [names]`.
- Orphan tombstone on load: `Warn: Orphan tombstone=<tid> actionId=<aid>`.

12.7 `[MergeJournal]`

Staged-commit phase transitions + finisher outcomes. Used by `MergeJournalOrchestrator`, `TreeDiscardPurge`.

- Phase start (Begin): `Info: sess=<sid> phase=Begin`.
- Phase advance: `Verbose: sess=<sid> phase=<phase>`.
- Fault injection (tests only): `Warn: Fault injected at phase=<Phase>`.
- Finisher: `Info: Finisher sess=<sid> markerWas=<present|absent> phase=<phase>`.
- Rollback outcome: `Info: Rolled back from phase=<p>: session restored sess=<sid> markerCleared=<b> provisionalRemoved=<r>`.
- Completion outcome: `Info: Completed from phase=<p> sess=<sid> stepsDriven=<n>`.
- Final cleared: `Info: Completed sess=<sid>`. `Verbose: sess=<sid> cleared`.
- TagRpsForReap: `Info: TagRpsForReap: promoted <n> session-provisional RP(s) for sess=<sid>`.
- Rollback recording removal: `Info: Rollback: removed session-provisional recording id=<rid>`.

12.8 `[LoadSweep]`

Single summary line per load after `LoadTimeSweep.Run` completes.

- `Info: [LoadSweep] Marker valid=<b>; spare=<n> discarded=<r> orphanSupersedes=<os> orphanTombstones=<ot> strayFields=<s> discardedRps=<rps>`.
- Pre-summary Verbose: `Verbose: LoadTimeSweep.Run: no ParsekScenario instance — skipping` when no scenario live.

12.9 `[UnfinishedFlights]`

Classifier + UI group decisions. Used by `EffectiveState`, `UnfinishedFlightClassifier` (v0.9.1), `UnfinishedFlightsGroup`, `UnfinishedFlightSealHandler` (v0.9.1).

Predicate verdicts (Verbose; rate-limited per `(rec, reason)` pair):

```
IsUnfinishedFlight=false rec=<rid> reason=mergeState:<state>
IsUnfinishedFlight=false rec=<rid> reason=notControllable headIsDebris=true
IsUnfinishedFlight=false rec=<rid> reason=noParentBp
IsUnfinishedFlight=false rec=<rid> reason=noScenario
IsUnfinishedFlight=false rec=<rid> reason=noMatchingRP bp=<bpId> rpCount=<n>
IsUnfinishedFlight=false rec=<rid> reason=slotSealed slot=<idx>
IsUnfinishedFlight=false rec=<rid> reason=downstreamBp
    chainTipChildBp=<bpId> matchedRpBp=<bpId>
IsUnfinishedFlight=false rec=<rid> reason=stableTerminalFocusSlot
    slot=<idx> focusSlot=<idx> terminal=<state>
IsUnfinishedFlight=false rec=<rid> reason=stableTerminal terminal=<state>
IsUnfinishedFlight=false rec=<rid> reason=noTerminal
IsUnfinishedFlight=false rec=<rid> reason=noFocusSignalOrbiting
    terminal=<state>
IsUnfinishedFlight=true  rec=<rid> reason=crashed terminal=Destroyed
IsUnfinishedFlight=true  rec=<rid> reason=stableLeafUnconcluded
    slot=<idx> terminal=<state>
IsUnfinishedFlight=true  rec=<rid> reason=strandedEva terminal=<state>
```

Site A promotion (Info, one-shot per promotion, v0.9.1):

```
[UnfinishedFlights] Info: CommitTree promoted rec=<rid>
    slot=<slotIdx> rp=<rpId>
    reason=<crashed|stableLeafUnconcluded|strandedEva>
```

Seal handler (Info, v0.9.1):

```
[UnfinishedFlights] Info: Seal cancelled rec=<rid>
[UnfinishedFlights] Info: Sealed slot=<slotIdx> rec=<rid>
    bp=<bpId> rp=<rpId> terminal=<state>
    reaperImpact=<willReap|stillBlocked>
[UnfinishedFlights] Error: Seal could not resolve slot for rec=<rid>
```

UI drag-into rejection: Verbose log + `ScreenMessages` toast (`Unfinished Flights is a system group and cannot receive dropped recordings`).

12.10 `[CrewReservations]`

Tree-discard reservation recomputation. Used by `TreeDiscardPurge` .

- Re-derive after tombstone set shrunk: `Info: Reservation re-derived after tree discard tree=<id>` .
- No-op: `Verbose: PurgeTree: no reservations affected tree=<id>` .

12.11 `[RevertInterceptor]`

Harmony prefix decisions. Used by `RevertInterceptor.Prefix` .

- Pass-through (no session): `Verbose: Prefix: no active re-fly session — allowing stock RevertToLaunch` .
- Block: `Info: Prefix: blocking stock RevertToLaunch sess=<sid> — showing re-fly dialog` .
- Error paths: `Error: ReflyRevertInterceptor internal failure: <reason>` .

12.12 `[ReconciliationBundle]`

Invocation Restore / Capture observations. Used by `ReconciliationBundle` .

- Capture: `Info: Captured bundle: recordings=<n> trees=<m> ledgerActions=<l> supersedes=<s> tombstones=<t> rps=<rps> marker=<b> journal=<b>` .
- Restore: `Info: Restored bundle: <same fields>` .

12.13 `[ERS]` / `[ELS]`

Cache rebuilds. Used by `EffectiveState` .

- ERS rebuild: `Verbose: Rebuilt: <n> entries from <raw> committed (skippedNotCommitted=<k> skippedSuperseded=<s> skippedSuppressed=<u> marker=<sessionId|none>)` .
- ELS rebuild: `Verbose: Rebuilt: <n> entries from <raw> (skippedTombstoned=<k>)` .

12.14 `[Recording]`

Stray-field cleanup on load. Used by `LoadTimeSweep` .

- `Warn: Stray SupersedeTargetId on committed rec=<id>; treating as cleared` .

12.15 `[Boundary]`

Multi-controllable classifier decisions. Used by `SegmentBoundaryLogic`.

- Classification outcome: `Info: ClassifyJointBreakResult: <outcome> originalPid=<pid> postBreakCount=<n>`.

---

13. Testing

13.1 Unit tests

The Rewind-scoped test classes in `Source/Parsek.Tests/`:

- `ActionIdMigrationTests` — deterministic `ActionId` generation from legacy action fields; idempotence.
- `AtomicMarkerWriteTests` — `RewindInvoker.AtomicMarkerWrite` including the `Phase1And2_NoOnSaveBetween` atomicity guard.
- `BackgroundSplitRpTests` — BG-split RP capture with cached PID payload.
- `BranchPointRewindPointIdRoundTripTests` — BranchPoint.RewindPointId serialization.
- `ChildSlotEffectiveRecordingIdTests` — forward walk with cycle guard + orphan terminator.
- `CopyRewindSaveToRootTests` — quicksave copy to save-root for `GamePersistence.LoadGame`.
- `CrewReservationRecomputeTests` — dead kerbal returns active after tombstone; no-op-safe paths.
- `DiskUsageDiagnosticsTests` — directory walk, missing-dir zero fallback, 10s cache TTL.
- `EffectiveStateTests` — ERS/ELS computation, mixed-parent halt, IsVisible, IsUnfinishedFlight, EffectiveRecordingId.
- `GrepAuditTests` — CI gate: verifies the `scripts/grep-audit-ers-els.ps1` allowlist matches the current source.
- `LedgerTombstoneRoundTripTests` — tombstone serialization.
- `LegacyMigrationTests` — MergeState migration + ActionId migration idempotence.
- `LoadTimeSweepTests` — marker validation, spare/discard sets, orphan logs, stray field, nested cleanup, summary.
- `MergeCrashRecoveryMatrixTests` — per-window state-invariant assertions for the 5-point crash matrix.
- `MergeJournalOrchestratorTests` — happy path + rollback + completion + idempotence + null-guard.
- `MergeJournalRoundTripTests` — MergeJournal ConfigNode round-trip.
- `MultiControllableClassifierTests` — `IsMultiControllableSplit` gate coverage.
- `PostLoadStripperTests` — ghost guard, primary match, fallback match, unrelated left alone, multi-slot conflict.
- `ReFlyRevertDialogTests` — Retry / Discard Re-fly / Cancel handlers + null callback paths; Discard Re-fly covers session-artifact clear, origin-RP promotion, journal-gate + defensive refusal, scene dispatch to `SPACECENTER` (Launch) or `EDITOR` with `EditorDriver.StartupBehaviour=START_CLEAN` + facility (Prelaunch), and survival of the origin RP across `LoadTimeSweep`.
- `ReFlySessionMarkerRoundTripTests` — marker ConfigNode round-trip.
- `ReconciliationBundleTests` — capture + restore over every field.
- `RecordingSupersedeRelationRoundTripTests` — relation ConfigNode round-trip.
- `RenameOnUnfinishedFlightTests` — rename persistence + hide-row advisory.
- `RewindCleanupClearTests` / `RewindContextTests` / `RewindLoggingTests` / `RewindTests` / `RewindTimelineTests` / `RewindTreeLookupTests` / `RewindUtCutoffTests` — overall invocation flow, log capture, tree lookup, UT-boundary behaviors.
- `RewindPointAuthorTests` — scene guard, warp drop, PID map capture, partial failure.
- `RewindPointReaperTests` — eligibility matrix, supersede walk, orphan slot, file-delete failure, idempotence.
- `RewindPointRoundTripTests` — RP ConfigNode round-trip including both PID maps.

- `RewindPrelaunchStripTests` — PRELAUNCH guard interactions.
- `SessionSuppressionWiringTests` — predicate + transition logging + chain-walker skip + map-presence gate + crew carve-out.
- `SpawnStateReconciliationTests` — post-strip state reconciliation.
- `SupersedeCommitTests` / `SupersedeCommitTombstoneTests` — commit + tombstone matrix + subtree + mixed-parent halt + idempotence.
- `TombstoneEligibilityTests` — full type matrix + 1s pairing window.
- `TreeDiscardPurgeTests` — RPs + supersedes + tombstones + reservations + marker/journal + sibling-tree isolation + state-version bumps.
- `UnfinishedFlightsDragRejectTests` / `UnfinishedFlightsGroupCannotHideTests` / `UnfinishedFlightsMembershipTests` — UI group invariants.
- `EarningsReconciliationTests` — earnings-ledger interactions with tombstoned rep bundles.

The v0.9.1 stable-leaf extension adds:

- `UnfinishedFlightClassifierTests` — predicate gate matrix: `IsDebris`, per-state `TerminalOutcomeQualifies` (8 `TerminalState` values × 2 `isFocus` × 2 `noFocusSignal` × 2 EVA-vs-vessel), null-terminal, EVA branch returns BEFORE `noFocusSignal` short-circuit, per-RP leaf gate, slot-closed gate. **Fails if:** any predicate branch verdict regresses.
- `ChildSlotSealedRoundTripTests` — `ChildSlot.Sealed` + `SealedRealTime` round-trip through ConfigNode save/load; legacy `CHILD_SLOT` ConfigNodes without the keys load with `Sealed = false`.
- `ChildSlotStashedRoundTripTests` — `ChildSlot.Stashed` + `StashedRealTime` round-trip; legacy ConfigNodes load with `Stashed = false`.
- `RewindPointFocusSlotIndexRoundTripTests` — `RewindPoint.FocusSlotIndex` round-trip; legacy `POINT` ConfigNodes load with `FocusSlotIndex = -1`.
- `ReFlySessionMarkerSupersedeTargetIdRoundTripTests` — `ReFlySessionMarker.SupersedeTargetId` round-trip; legacy markers load with null.
- `ReFlySessionMarkerPreSessionBranchPointIdsRoundTripTests` — presence sentinel + repeated value round-trip; legacy markers load with `null`.
- `ApplyRewindProvisionalMergeStatesTests` (extended) — Site A coverage: stable-leaf non-focus controllable → CP; stable-leaf focus controllable → Immutable; stable-leaf debris → Immutable; crashed-leaf focus → CP (active-parent crash regression guard); crashed-leaf non-focus → CP. **Fails if:** the broader predicate misclassifies any case.
- `SupersedeCommitMergeClassifierTests` — Site B-1 coverage: re-fly ending Orbiting/SubOrbital/Landed/Splashed on chosen slot with no retry-blocking actions → `stableTerminalFocusSlot` + `Immutable` + `slot.Sealed=true`; origin-only marker-target resolution produces the same promoted-focus result and logs `side=origin-only`; `ScienceEarning` → `Immutable` + `slot.Sealed=true` through `recordingAction:*`; Recovered/Docked/Boarded or downstream structural/world interaction → `Immutable` + `slot.Sealed=true`; Destroyed with no retry-blocking actions → CP and not sealed; Destroyed with credited `ScienceEarning` → `Immutable` + sealed; Destroyed with automatic consequence rows / KSC-scene player decisions / tombstoneable kerbal-death rows only → CP and not sealed. **Fails if:** chosen stable outcomes stay open, unsafe `ScienceEarning` outcomes remain re-flyable, or terminal-failure outcomes stop being retryable.
- `SupersedeCommitInPlaceTests` — Site B-2 coverage: in-place re-fly ending Orbiting/SubOrbital/stashed-stable on chosen slot with no retry-blocking actions → `Immutable` + `slot.Sealed=true`, RP reaps; same path with retry-blocking action also closes through `recordingAction:*`; in-place re-fly ending Destroyed / non-boarded EVA → CP unless retry-blocking action present.
- `SupersedeCommitStructuralMutationGateTests` — `HasReFlySessionStructuralMutation` UT cutoff (`> rp.UT` not `> marker.InvokedUT`), `PreSessionBranchPointIds` baseline filter, lineage-set scope, `BranchPointType` matrix.

- `SealHandlerTests` — `slot.Sealed` flips + `SealedRealTime` stamps; `SupersedeStateVersion` bumps; `MergeState` UNCHANGED. `RewindPointReaper.IsReapEligible` matrix: slots Immutable + Sealed → reap; any unsealed CP slot → no-reap; **any NotCommitted → no-reap regardless of `slot.Sealed`** (defends against load-time race states). `reaperImpact=stillBlocked` vs `willReap` log values.
- `RewindPointAuthorFocusSlotIndexTests` — `FocusSlotIndex` set correctly when active vessel matches one of the post-split slots; `FocusSlotIndex == -1` when no slot matches.
- `InvocationLinearizationTests` — marker-write linearization (both branches stamp both fields), wrapper contract (`ComputeSessionSuppressedSubtree(null)` empty, defensive copy, cached-null fallback), closure-equivalence (refactor regression guard), linear chain extension (3-link chain produces `{probeReFly1 -> probeReFly2}`).
- `HybridSupersedeGraphTests` — §7.72 hybrid topology: legacy star + new linear extension; walker resolves to chain-tip; `TryResolveRewindPointForRecording` works on both branches.
- `ManualStashTests` — Stash resolver matrix: spawnable stable terminals (`Landed` , `Splashed` , `Orbiting` , `SubOrbital`) accepted; `Recovered` / `Docked` / `Boarded` / Destroyed rejected; `ScienceEarning` -bearing recordings rejected; already-reaped RPs rejected (no resurrection).

13.2 In-game tests (`Source/Parsek/InGameTests/RuntimeTests.cs` , Ctrl+Shift+T)

- `CaptureRPOnStaging` — asserts both PID maps populated, quicksave exists in `Parsek/RewindPoints/` .
- `SavePathRootThenMove` — no stray file left in save root.
- `WarpZeroedDuringSave` — warp drops to zero during save.
- `InvokeRPStripAndActivate` — strip counts + active-vessel PID unchanged.
- `MergeLandedReFlyCreatesImmutableSupersedeTest` — relation in list, provisional Immutable, RP reaps.
- `MergeCrashedReFlyCreatesCPSupersedeTest` — provisional `CommittedProvisional` , is Unfinished Flight, RP does not reap; re-invoke path exercises chain extension on next merge.
- `ContractStickyAcrossSupersedeTest` — contract completed by BG-crash stays complete after re-fly merge.
- `GhostSuppressionDuringReFlyTest` — no ghost rendering for supersede-target subtree.
- `KerbalRecoveryOnSupersedeTest` — kerbal returns active; reservation re-derived.
- `KerbalDualResidenceCarveOutTest` — live re-fly crew exempt from reservation lock.
- `UnfinishedFlightsRenderingAndNoHideTest` — UI group renders correctly and cannot be hidden.
- `F5MidReFlyResumeTest` — atomic phase 1+2 means marker validates; spare set survives.
- `MergeInterruptionRecoveryTest` — inject exception mid-merge; verify finisher behavior.
- `JournalFinisherMarkerPresentVariantTest` — marker-present variant of the finisher.
- `TreeDiscardRemovesSupersedesAndTombstonesTest` — tree-discard purge covers all scoped collections.
- `RPReapOnLastSlotImmutableTest` — RP reaps when the last slot becomes Immutable.
- `PartPersistentIdStabilityTest` — critical precondition probe: `Part.persistentId` is stable across the save/load cycle (required for `RootPartPidMap` fallback to work).

The v0.9.1 stable-leaf extension adds:

- `MunMothershipFourProbeDeployTest` — the canonical 4-probe deploy: fly mothership home, return to Recordings Manager, verify all 4 probes in UF AND mothership NOT in UF, Fly probe #2, land it, merge, verify slot closure + supersede chain extension. **Fails if:** the canonical case stops working end-to-end.
- `AutoParachuteBoosterFocusUpperStageTest` — booster non-focus + Landed terminal + upper stage focus + Orbiting → both NOT in UF, RP reaps cleanly. **Critical regression guard.**
- `InvertedFocusBoosterUpperStageOrbitingTest` — fly booster, leave upper stage Orbiting, verify upper stage IS in UF and RP stays alive.
- `StrandedEvaKerbalRescueTest` — kerbal stranded + lander Orbiting → both in UF; Fly kerbal, reboard, merge → kerbal slot closes; lander slot still in UF; Seal lander to clean up.

- `BreakupSurvivorMatrixTest` — trigger a breakup, survive, land safely → NOT in UF and RP reaps; trigger another, survive but Orbiting as the focus slot → NOT in UF (intentional focus exclusion); trigger a comparable origin-only non-focus survivor to Orbiting/SubOrbital → IS in UF; trigger a third, crash post-survival → IS in UF (Crashed regardless of focus).
- `CrossTreeDockDuringStableLeafReFlyTest` — probe re-fly docks with another tree's station, merge, verify slot closes Immutable, supersede stays inside probe's tree, station's tree unchanged.
- `ReflyStableTerminalAutoSealTest` — auto-included non-focus probe, re-fly to a different stable orbit, merge, verify the supersede relation points at the new recording and that the chosen slot auto-seals via `stableTerminalFocusSlot` and drops from UF (the v0.9.1 focus override applies to the player-chosen slot regardless of `RewindPoint.FocusSlotIndex`); credit `ScienceEarning` (Crew Report / Transmit / Recover) during a comparable re-fly and verify the slot still auto-seals via `recordingAction:ScienceEarning:*`. Manual-Stash variant: Stash a default-excluded stable row, merge clean Landed/Orbiting outcomes and verify the slot auto-seals via `stableTerminalFocusSlot`; then crash a re-fly with only automatic consequence rows / KSC-scene player decisions / tombstoneable kerbal-death rows and verify the slot stays open for another retry.
- `SealAndNoUnsealTest` — Seal a row, verify there's no in-game un-seal path (Full-Revert is the only undo).

13.3 Grep-audit CI gate

- `scripts/grep-audit-ers-els.ps1` scans every source file outside the allowlist for raw `RecordingStore.CommittedRecordings` or `Ledger.Actions` reads.
- `scripts/ers-els-audit-allowlist.txt` enumerates the 35 permitted exemptions with rationale comments. Each exemption has an inline `[ERS-exempt]` comment at the call site.
- `GrepAuditTests.GrepAudit_AllRawAccessIsAllowlisted` fails the build on any un-allowlisted raw read.

13.4 Crash-recovery reviewer sign-off

The pre-impl spec's v0.5 sign-off condition was "crash recovery matrix covers every window." `MergeCrashRecoveryMatrixTests` asserts per-window state invariants for the 5 crash-point cases; `MergeJournalOrchestratorTests` exercises the rollback and completion paths via `FaultInjectionPoint` + `DurableSaveForTesting` seams. Together they cover every pre-Durable-1 and post-Durable-1 crash window referenced in §6.12's failure-recovery matrix.

13.5 v0.9.1 migration tests

- Legacy save with already-reaped split RPs has empty UF group for those splits (no retroactive surfacing). **Fails if:** the upgrade tries to resurrect deleted RPs.
- Legacy save with live RPs whose Landed siblings are Immutable: row count unchanged from v0.9.0. **Fails if:** Landed terminals start qualifying.
- Legacy save with live RPs whose Orbiting non-focus siblings are Immutable: `ApplyRewindProvisionalMergeStates` does NOT re-promote. **Fails if:** the `noFocusSignal` short-circuit doesn't fire on legacy RPs.
- Legacy save with live RPs whose stranded EVA Landed: row newly appears. Negative variant: same scenario with `EvaCrewName == null` → row does NOT surface. **Fails if:** the EVA carve-out doesn't fire OR vessel terminals get retroactively surfaced.
- Post-upgrade fresh deploy: spawn a multi-controllable split, leave probes orbiting, commit. Verify the new RP has `FocusSlotIndex >= 0`, the probes promote to CP, and the row appears in UF. **Fails if:** the motivating case stops working for new RPs.

13.6 Log-assertion tests (v0.9.1)

- `[UnfinishedFlights] Verbose` reasons for each predicate gate appear with the right structured values. **Fails if:** the diagnostic catalog drifts from the predicate.
- `[UnfinishedFlights] Info: Sealed ... reaperImpact=...` log appears with the correct impact computation.
- `[Supersede] Error: Site B-1 slot lookup failed ...` log appears in the (intentionally-induced) test scenario where the §6.22 prerequisite is broken.
- `[Supersede] Info: provisional=... focusSlot=N focusSlotOverride=M` includes both fields so the source of the focus decision is greppable.
- Shared-classifier diagnostic test: assert that Site B logs the same classifier reason Site A would have produced on the same `(Recording, ChildSlot, RewindPoint)` triple, while separately asserting Site B's stricter merge result. **Fails if:** the diagnostic predicate drifts or the merge-close policy regresses.

Appendix A — Gameplay Scenarios

A.1 Booster recovery (the motivating case)

1. Player launches an AB stack: upper stage B (crewed capsule) on top of booster A (probe core + parachute, intended for recovery).
2. Staging: the decoupler fires. A and B split. Both controllable → RP-1 is captured (one-frame-deferred quicksave under `Parsek/RewindPoints/`). PidSlotMap records B and A by Vessel.persistentId; RootPartPidMap records their root-part PIDs.
3. Player flies B to orbit. A is BG-recorded; the player gets no chance to deploy the chute in time and A crashes into the ocean.
4. Player returns to Space Center. Merge dialog → Merge to Timeline. Tree commits: ABC Immutable, B Immutable (Orbited), A CommittedProvisional (Crashed, terminal kind classifies as Crashed). RP-1 survives the scene load as a normal staging RP (`SessionProvisional=true`, `CreatingSessionId=null`) and is promoted to persistent when the tree commits. Recordings Manager shows a new "Unfinished Flights" virtual group containing A with a Rewind button.
5. Player opens Recordings Manager, finds A under Unfinished Flights, clicks **Rewind**. Dialog names the split UT and advisory about career state / kerbal deaths. Player clicks **Rewind**.
6. Scene reloads into FLIGHT at the staging moment. A is the active vessel (live physics); B has been stripped and now plays back as a ghost from its committed recording. ABC plays back as a ghost up to the split moment, then terminates. Player's career state matches what it was immediately before Rewind (B's orbit entry + all milestones / contracts are intact).
7. Player deploys A's parachute and lands it safely on the launch pad. Recorder finalizes A' with TerminalKind=Landed.
8. Merge dialog → Merge to Timeline. `MergeJournalOrchestrator.RunMerge` drives through the nine phases. A → hidden (superseded); A' → Immutable + effective slot-1 representative. No tombstones (no kerbal died). RP-1 reap-eligible (both slots Immutable) → quicksave deleted, scenario entry removed, BranchPoint back-reference cleared.
9. Unfinished Flights is now empty. The timeline shows B's orbit entry AND A's safe landing (the sealed slot points at `A'` via the supersede relation); both play back on any subsequent rewind-to-launch.

A.2 EVA re-fly

1. Player has a Mun lander in orbit. Kerbal EVAs to plant a flag.
2. Multi-controllable split (EVA): lander + kerbal. RP-2 captured.
3. Kerbal slips off the ladder while re-boarding. BG-recorded; EVA kerbal terminal kind = Crashed; kerbal dies.
4. Player returns to Space Center. Merge. Lander Immutable; EVA recording CommittedProvisional + Crashed. `KerbalAssignment`-to-Dead action emitted; paired rep penalty emitted within 1s → both are tombstone-

eligible but not yet tombstoned (tombstoning happens on re-fly merge, not on the initial tree merge). EVA appears in Unfinished Flights.
5. Player invokes RP-2 targeting the EVA kerbal slot. The lander plays back as a ghost; the kerbal is live on the ladder.
6. Player carefully re-boards the lander. Recording terminates at the re-board event (which Parsek records as a merge boundary). TerminalKind=Landed (or Docked, depending on classifier interpretation — both map to Immutable).
7. Merge. Supersede relation `{EVA → EVA'}` appends. Kerbal-death action tombstoned; bundled rep penalty tombstoned. `CrewReservationManager.RecomputeAfterTombstones` re-derives; the dead kerbal returns to active. EVA' Immutable; both slots Immutable → RP-2 reaps.

A.3 Docking merge + re-fly

1. Player launches a tanker to rendezvous with a station. Lander undocks from the station; both controllable → RP captured.
2. Player flies the tanker. Lander is BG-recorded. Due to an autopilot bug the lander de-orbits and crashes.
3. Merge. Lander CommittedProvisional Crashed → Unfinished Flight. Tanker Immutable.
4. Player rewinds. Lander is live; tanker plays back as a ghost; station's own tree is unrelated and stays physical (or ghost, depending on its own chain state).
5. Player manually flies the lander back up and docks with the station. Recording terminates at dock; `TerminalKind = Docked` → maps to Landed kind → Immutable merge.
6. Supersede `{Lander → Lander'}`. Station's tree unchanged. RP reaps if the tanker slot is also Immutable.

A.4 Revert-during-re-fly (3-option dialog)

1. Player is in the middle of an A.1-style booster re-fly. Rocket barely controllable after re-entry.
2. Player hits Esc → Revert to Launch. `RevertInterceptor.Prefix` sees the active marker → spawns the 3-option dialog.
3. **Retry from Rewind Point**: dialog dismisses. `RetryHandler` clears the marker, generates a fresh session id, re-invokes `RewindInvoker.StartInvoke(rp, slot)`. Scene reloads back to split moment with a new provisional A''. The old A' is now a zombie for the next load-time sweep (or for the current-session load cycle's sweep if another save happens first).
4. **Discard Re-fly**: dialog dismisses. `DiscardReFlyHandler` removes the provisional A' from `CommittedRecordings`, promotes the origin RP to persistent (`SessionProvisional=false`, `CreatingSessionId=null`) so `LoadTimeSweep` does not reap it, clears the marker + journal, bumps the supersede state version, then stages the origin RP's quicksave through the save-root copy path and calls `GamePersistence.LoadGame` + `HighLogic.LoadScene(GameScenes.SPACECENTER)`. The player lands in the Space Center at the RP's UT. The Unfinished Flights entry is still present (the origin crash recording remains visible as an `Immutable` legacy crash or `CommittedProvisional` current crash, and its RP was promoted, so `IsUnfinishedFlight` continues to resolve true). Career state is unchanged. The tree's other committed supersede relations, tombstones, and Rewind Points are preserved — other re-fly attempts already merged into the tree are unaffected.
5. **Continue Flying**: dialog dismisses; nothing happens. Player keeps trying.

The same 3-option dialog appears when the player picks Revert-to-VAB (or Revert-to-SPH for a spaceplane). Retry's behaviour is identical (RP quicksave is FLIGHT-scoped — Retry lands back at the split moment in FLIGHT, not in the editor). Discard Re-fly in that case pre-sets `EditorDriver.StartupBehaviour = EditorDriver.StartupBehaviours.START_CLEAN` and `EditorDriver.editorFacility = facility` before `HighLogic.LoadScene(GameScenes.EDITOR)` so the scene swap lands in the VAB or SPH as originally clicked, with a fresh editor against the just-loaded career.

A.5 Crash-quit-resume mid-re-fly

1. Player is 30 seconds into a re-fly. Home power fails. KSP dies without saving.
2. Player reopens KSP, loads the save. `ParsekScenario.OnLoad` loads the scenario including the `REFLY_SESSION_MARKER` that was on disk from the last F5 quicksave (if one was taken during the session) or from the last auto-save.
3. `MergeJournalOrchestrator.RunFinisher` runs first; no journal present (the player wasn't in the middle of merging), returns false.
4. `RewindInvoker.ConsumePostLoad` runs; no `RewindInvokeContext.Pending`, returns.
5. `LoadTimeSweep.Run` runs. `MarkerValidator.Validate` checks the six durable fields against the current scenario state. If the provisional recording from the session is still in `CommittedRecordings` with `MergeState == NotCommitted` AND the origin recording + RP + tree all resolve AND `InvokedUT` is in the past → marker valid. Spare set = {provisional, RP}. No discard.
6. `RewindPointReaper.ReapOrphanedRPs` runs. The RP is in the spare set and session-provisional; nothing to reap.
7. Player picks up where they left off. The live scene may need them to recover physics state (KSP's own auto-save quirks), but the Parsek-side re-fly session is intact.

Alternative: if the scenario state drifted (e.g. the provisional was corrupted on disk), `MarkerValidator` fails on the first mismatching field, emits `[ReFlySession] Warn: Marker invalid field=<f>; clearing`, clears the marker, and the nested-cleanup pass discards the provisional + session-provisional RPs. The player reverts to the pre-invocation state, with Unfinished Flights still showing the original retired sibling (because its CommittedProvisional stays). They can click Rewind again.

End of Rewind to Separation design doc.